Companion document

MeTTaDinC in Practice

Four Worked Examples

Compacting an agent's belief memory, auditing an access policy without enumeration, sharing skills across camera frames, and market making across regime shifts

Companion to the MeTTaDinC specification draft and the Cetta implementation guide of 4 November 2026

4 November 2026

Developed from the d-calculus and Hyperseed project

Scope. Four complete MeTTaDinC programs, each worked out from the problem statement to the expected result of every request: the model, the modules, the statuses, the witnesses and certificates, the algorithms that produce them, the design lessons the results carry, and the registry additions each program needs. Together the four examples exercise the graded deterministic core, symbolic carriers with proof-logged obligations, transfers and hives with declared gauges, and the stochastic profile with decision processes and monitors.

Validation material. Each example ships as a bundle: its modules, a registry patch that declares every proposed addition, a Python model, an exact-arithmetic oracle that recomputes every number in this document from the module files, the oracle's results, and the static checker that tests the modules against the registry with and without the patch. The bundles of Examples 1 and 2 also carry their proof obligations and the DRAT proofs of every unsatisfiable one, which the oracles replay through drat-trim and the verified checker `cake_lpr`. The four bundles total 305 oracle checks. Where this text and the bundles differ, the bundles are authoritative. No MeTTaDinC runtime exists yet; the examples are written to be the first programs one runs.

Abstract

A language specification says what programs mean. It does not show what it is like to write one for a problem someone cares about, which requests that person would send, what the answers look like, or which parts of the language turn out to be missing. This document works through four MeTTaDinC programs in full. The first compacts the belief memory of an agent with two consumers, a reasoner that reads truth values and a controller that reads a status, and finds that the choice of decay rule decides whether any compact summary can be trusted; it separates three meanings of "store it in fewer bits" into three claims with different numbers. The second audits an administrative access-control policy on forty-bit configurations without enumerating them: two authoring mistakes are found with short canonical witnesses, the corrected policy is proved safe with 80 small proofs whose obligations are generated from the module itself and checked by a verified checker, and a true property comes back Unknown until a stronger invariant is supplied. The third shares navigation skills between three robots whose cameras see the same bay in different frames; a transfer learned from recorded behavior is correct only up to the task's symmetries, and whether the skill library is coherent depends on the gauge declared at each robot. The fourth follows a market maker through a toxic-flow shift and a one-sided-flow shift: the first changes every value and no decision, the second breaks the desk's state compression and its policy, a toxicity signal is priced correctly and incorrectly, a leak of news is shown to be invisible to a desk that only quotes wide, and monitors are shown to be provably blind to one shift each. Every number is recomputed by an exact oracle, and each example ends with implementation notes and the registry additions it needs.

How to read this document

Section 1 explains the purpose of the examples and how they divide the language between them. Section 2 collects the conventions every example uses: the statuses and claim kinds, the defects, the canonical order of witnesses, the registry patches and the oracles. Sections 3–6 are the four examples. Each opens with the problem in its own terms and the background a reader outside formal methods needs, then gives the program, the results with their witnesses and certificates, a table of expected results, notes for implementers, and the proposed registry additions. Section 7 consolidates what the examples ask of the language, and Section 8 concludes. Appendix A lists the four module files in full, Appendix B summarizes the registry patches, and Appendix C explains how to reproduce every result. The examples can be read in any order; the fourth refers to the third in a few places. Paragraphs headed *In plain terms* summarize a technical step for readers who want the idea without the formulas.

References to "the specification" are to the MeTTaDinC specification draft of 4 November 2026 [1], and references to "the guide" are to the Cetta implementation guide of the same date [2].

Contents

1 Introduction **4**

1.1 Why worked examples 4

1.2 The four examples 4

1.3 How each example is organized 5

2 Conventions shared by the examples **6**

2.1 Statuses and claims 6

2.2 Defects in one page 7

2.3 Canonical witnesses 7

2.4 Registry patches and the two-pass check . . . 8
2.5 Oracles . . . 8
2.6 Release scope . . . 9
2.7 Reproducing the results . . . 9

3 Example 1: compacting an agent's belief memory 9
3.1 The problem . . . 9
3.2 Background: what compaction means here . . . 10
3.3 The program . . . 11
3.4 Exact compaction for the controller . . . 13
3.5 Graded summaries under the original decay . . . 14
3.6 The decay rule is a design parameter . . . 14
3.7 Three uses, three claims . . . 15
3.8 The frontier and its certificates . . . 16
3.9 Six beliefs . . . 17
3.10 Forks and merges . . . 18
3.11 Expected results . . . 19
3.12 Notes for implementers . . . 20
3.13 Proposed registry additions . . . 21

4 Example 2: auditing an access policy without enumeration 22
4.1 The problem . . . 22
4.2 Background: proofs about a space that cannot be listed . . . 23
4.3 The program . . . 24
4.4 The policy as written . . . 25
4.5 The corrected policy . . . 26
4.6 Compaction . . . 27
4.7 Expected results . . . 28
4.8 Notes for implementers . . . 29
4.9 Proposed registry additions . . . 30

5 Example 3: sharing skills between robots with different camera frames 31
5.1 The problem . . . 31
5.2 Background: transfers, symmetries, holonomy and gauges . . . 31
5.3 The dihedral picture . . . 33
5.4 The program . . . 33
5.5 Transfers . . . 35
5.6 The hive under four gauge declarations . . . 35
5.7 Derived kernel hives . . . 35
5.8 A skill around the loop . . . 36
5.9 Docking breaks a symmetry . . . 36
5.10 A pallet in C's bay . . . 37
5.11 Expected results . . . 37

5.12 Notes for implementers . . . 38
5.13 Proposed registry additions . . . 39

6 Example 4: a market maker across regime shifts 39
6.1 The problem . . . 39
6.2 Background: decision processes and their quotients . . . 40
6.3 The program . . . 41
6.4 Calm: three risk buckets are enough . . . 43
6.5 Two shifts of different types . . . 43
6.6 Toxic flow as a hidden distinction . . . 44
6.7 Did the news leak? . . . 46
6.8 Monitoring the shift . . . 48
6.9 Expected results . . . 50
6.10 Notes for implementers . . . 51
6.11 Proposed registry additions . . . 52

7 What the examples ask of the language 53
7.1 Registry additions across the examples . . . 53
7.2 Lessons for implementers . . . 54
7.3 Open questions the examples raise . . . 55

8 Conclusion 55

A Module sources 58
A.1 Example 1: `belief_memory.metta` . . . 58
A.2 Example 2: `access_policy.metta` . . . 62
A.3 Example 3: `skill_share.metta` . . . 65
A.4 Example 4: `market_maker.metta` . . . 69

B Registry patches 73
B.1 Example 1 . . . 73
B.2 Example 2 . . . 74
B.3 Example 3 . . . 74
B.4 Example 4 . . . 75

C Reproducing the results 77

1 Introduction

1.1 Why worked examples

The MeTTaDinC specification defines what a program means: carriers, observers and actions; the defects that measure what an encoding loses; the claims that say which notion of sameness has been established; the certificates that let a second program check the first. It supports those definitions with fixtures, and the fixtures are small on purpose. Each isolates one phenomenon – an eight-state machine for the crisp core, a forty-bit machine that cannot be enumerated, a seven-state chain on which behavioral, trace and linear answers disagree – so that an implementation either reproduces the phenomenon or visibly fails to.

Fixtures do not show what it is like to use the language. Someone with a real problem has to decide which machine to declare, which consumer the interface serves, which requests to send in which order, what a refutation is telling them about their design, and what to do when the language cannot yet express the next question. Those decisions are where a language succeeds or fails in practice, and they only become visible on problems large enough to have structure and small enough to be checked exactly. The four examples in this document were chosen with that in mind. Each is a problem that comes up in the systems this work is meant to support – the memory of an Omega agent, the access policy of an organization, a group of robots sharing skills, a trading desk facing a market that changes under it – and each forces the language to say something the fixtures do not.

The examples serve two readers. An implementer gets a set of conformance targets that exercise many requests together, with every expected status, witness and number stated and recomputed by an independent oracle, and with notes on the algorithms that produce them. A prospective user gets a picture of how MeTTaDinC programs are written and read, and of the kind of design advice the answers carry: a decay rule that silently ruins every summary of a memory, a policy rule that reads a session where it should read an assignment, a learned transfer that is correct only up to a mirror, a market shift that a fill-rate dashboard can never see.

1.2 The four examples

Example 1, belief memory (Section 3). An agent keeps two evidence counts per proposition and serves two consumers: a reasoner that reads a truth value and a controller that reads a three-way status. Under memory pressure the designer wants dormant beliefs stored in fewer bits. The example finds that neither consumer has a useful exact compaction, that the original decay rule makes every compact summary worthless, that a gentler rule bounds the error of a twenty-one-code summary at 1/5, and that "fewer bits" means three different things — an index, a cold store that is rehydrated, and a memory that runs on its codes — with three different claims and three different answers. It extends to six beliefs (2^{48} states) by a product rule and to a ledger of evidence receipts whose forks must be merged, and whose oldest receipts can be compacted to counts only under a delivery protocol that keeps them from arriving again.

Example 2, access policy (Section 4). An administrative role-based access control policy over four users and five roles has forty bits of state. The audit finds two authoring mistakes with two- and three-step witnesses, proves the corrected policy safe with 80 small proofs whose obligations are generated from the module and which pass through the specification's pinned proof-checking pipeline, shows that restoring either mistake in the source changes the verdict, shows a true property returning Unknown until a stronger invariant is supplied, and finds that a targeted monitor needs seventeen bits where a vendor proposed ten.

Example 3, skill sharing (Section 5). Three warehouse robots see the same bay through

cameras in three frames and share a library of navigation skills. A transfer learned from recorded behavior is one of four equally exact candidates, and going around the cycle of transfers returns every cell as its mirror image, so that a skill comes back earning at each cell what it earned at the mirrored cell. Whether that is acceptable depends on the gauge declared at each robot; a gauge copied between robots instead of transported is caught by the intertwining check. A docking requirement breaks a symmetry and exposes the stale transfer, and a pallet makes transfers into one bay approximate.

Example 4, market making (Section 6). A market maker's quoting problem is a discounted decision process in four markets. In calm markets the desk can run on three risk buckets with no loss; toxic flow lowers every value by more than half and changes no decision; one-sided flow breaks the compression and the policy, and the broken symmetry survives as an exact transfer between two markets. A toxicity signal is priced correctly and incorrectly, a news leak is shown to be invisible to a desk that only quotes wide, and each of two monitors is shown to be blind to one of the shifts.

Example	Profile and carriers	Main kinds of claim	Oracle checks
1. Belief memory	graded, deterministic; enumerated, with a product of six	exact quotients, graded and staged defects, frontiers with optimality certificates, induced machines, merge contracts	92
2. Access policy	crisp, deterministic; symbolic forty-bit words	symbolic partitions on a reachable scope, inductive invariants, proof-logged obligations, slices, quotients	69
3. Skill sharing	crisp, deterministic; enumerated subsets of a grid	transfers, candidate families, holonomy, transfer hives with gauges, derived kernel hives	57
4. Market making	stochastic; enumerated, with stored data streams	MDP homomorphisms, Kantorovich defects, policy optimality, averaging, trace and linear claims, e-process monitors	87

Table 1: The four examples.

1.3 How each example is organized

Each example section has the same parts. It opens with the problem in its own terms and the background needed to follow it. It then gives the program: excerpts of the modules with the reasons for each declaration, and pointers to the full file in Appendix A. The results follow, grouped by the question they answer, each with its status, its canonical witness when it is refuted, and its certificate when it is checked. A table of expected results collects them for conformance testing. Notes for implementers describe the algorithms that produced the results and the pitfalls the oracle uncovered, and a final subsection lists the registry additions the example proposes.

Proposed additions are marked in the modules with the comment `PROPOSED`. None of them changes the meaning of an existing request on the inputs that request already accepts; several extend a request to new inputs (Example 4 extends four requests of the Decisions library from one-step decision models to decision processes), and one proposes a change to a status rule (Example 1 allows the optimality of a frontier to be Checked by a replayed lower-bound certificate without exhaustive enumeration). Section 7 collects all of them.

2 Conventions shared by the examples

2.1 Statuses and claims

Every request returns a result in the specification's envelope (specification, Section 5.5), with one of seven statuses (specification, Section 2.2). The examples use all seven, under one policy for what each status requires.

- A result is *Checked* when the claim holds within its scope and is established either by an exact method that the checker runs on the module's machine (finite enumeration, exact rational arithmetic, partition refinement, decision diagrams) or by a certificate that the checker replays: a witness, a coupling, a clique, a table of values, an exact dual solution of a linear program, or a proof of unsatisfiability in the DRAT format that drat-trim converts to LRAT and the verified checker `cake_lpr` accepts, for an obligation that the obligation generator built from the admitted source.
- A result is *Conditional* when the claim holds under named assumptions that the checker has not discharged. The answer of an external solver that has not been replayed, such as a satisfiability solver's "unsatisfiable" without a checked proof, is Conditional, and its assumption names the solver, its version, the logic and the digest of the obligation, as in `(TrustedBackend cadical 1.5.3 SAT sha256:...)`. A second solver or a second encoding that agrees is recorded as evidence, in the field `agreed_by`; it does not discharge the assumption.
- A result is *Refuted* when a counterexample, the *witness*, or a lower-bound certificate shows that the claim fails and the checker replays it by evaluating the admitted source. A witness found by a solver counts once it has been replayed.
- A result is *Unknown* when the claim is unresolved, because a budget ended or a certified interval straddles the bound. The result carries what was learned in its `Resources` field.
- A result is *Estimated* when a statistical procedure supports a conclusion about a data source, with its estimand, protocol, sample and guarantee stated. Estimated results appear only in Example 4.
- A result is *InvalidInput* when a declaration or request violates the admitted language, for example a transfer whose state pullback is not total, and *Unsupported* when the request lies outside the capability manifest of the running release.

A result derived from other results, such as a claim about six beliefs lifted from one belief by the product rule of Example 1, carries the union of their assumptions, so a derivation from a Conditional premise is Conditional. A consumer declares which results it accepts: `(Accepts Checked)` accepts only Checked results, and `(Accepts Checked (Conditional (TrustedBackend cadical)))` also accepts results whose only open assumptions are trusted-backend assumptions for CaDiCaL. A Conditional result that a consumer does not accept counts, for that consumer, as a claim not yet covered.

Only Examples 1 and 2 call a solver. Their bundles keep every unsatisfiable obligation together with its DRAT proof, and their oracles replay the proofs when drat-trim and `cake_lpr` are installed. The results that rest on the proofs are then Checked; without the two checkers the oracles report them as Conditional, with the assumption above. Every other result in the four examples is an exact computation or a replayed certificate.

The specification separates measuring a defect from checking a bound (specification, Section 2.4). A request named `Measure...` returns a certified interval and is Checked when the interval is no wider than the requested `Precision` (zero by default, which asks for the exact value). A request named `Check...` with `(Within b)` is Checked when the upper end of the

interval is at most b, Refuted when the lower end exceeds b (with a witness), and Unknown otherwise. Without `Within`, a check asks for defect zero.

2.2 Defects in one page

An interface is a machine: a carrier X of states, observers o_j with value tolerances β_j, and actions T_a that are deterministic maps, stochastic kernels or nondeterministic relations. A representation is a map q from states to codes, and it is meant to be used in place of the state by some consumer. The specification measures what q loses with four defects (specification, Section 4.2).

- The *current defect* $\delta_{\mathrm{now}}(q)$ is the largest observation distance $1 - \beta_j$ between two states with the same code. It is zero when every observer can be read off the code.
- The *update defect* $\delta_{\mathrm{upd}}(q)$ is the largest distance between the codes of the successors of two states with the same code. For crisp codes and deterministic actions it is zero or one: either every action can be carried out on codes, or some pair of same-code states is sent to different codes. For stochastic actions it is the largest total-variation distance between the laws the two states push onto the codes; defect zero is *strong lumpability*.
- The *behavioral defect* $\delta_{\mathrm{beh}}(q)$ is the largest behavioral distance between two states with the same code. For a deterministic machine the behavioral distance at discount γ is

$$d^{\gamma}_{\mathrm{beh}}(x, y) = \max_{w} \ \gamma^{|w|} \ \max_{j}(1 - \beta_j(o_j T_w x, \ o_j T_w y)),$$

  the largest discounted observation difference that any finite word of actions can produce; through horizon h the maximum ranges over words of length at most h. For stochastic machines the behavioral distance is the Kantorovich bisimulation pseudometric, which replaces the single successor by a coupling of the two successor laws (specification, Section 4.1).
- The *trace defect* compares the laws of observation sequences only. For deterministic machines it equals the behavioral defect; for stochastic machines it can be strictly smaller (specification, Section 4.3).

When the current and update defects are both zero, the representation is an exact quotient: `InduceExact` returns the abstract machine on the codes and a checked quotient arrow. When they are not, `InduceApprox` builds a machine from chosen representatives and reports its *radius*, the largest distance from a state to the representative of its class, and its *arrow defect*, the largest distance between a state and its code in the disjoint union of the concrete and abstract machines (specification, Section 4.6).

In plain terms. A code is a promise that two states are interchangeable for a particular consumer. The current defect says how far the consumer's present readings can differ between states with the same code, the update defect says whether the code can be kept up to date without the state, and the behavioral defect says how far apart the consumer's future readings can drift. These are three different promises, and the examples show each of them failing while the others hold.

2.3 Canonical witnesses

A replay checker accepts any witness that replays. Conformance testing needs more: two correct implementations must report the same witness, so that a manifest can record it. The

specification fixes a canonical witness, the least one in a declared order: shortest word first, then pairs of states in the lexicographic order of carrier declaration, then actions and queries in declaration order, then words in numeric order (specification, Section 4.4 and the registry's `witness_order`). A `Product` carrier enumerates in declared component order with the last component varying fastest, which fixes the order of pairs. Each example adds the ordering rules its own claims need: the least moved cell for a holonomy, the least state and first improving action for a policy optimality check, the least observation sequence for a trace event.

Producing canonical witnesses is mostly a matter of search order. Breadth-first searches expand actions in declaration order; solver models are minimized before they are reported, because a satisfiability solver returns an arbitrary model; and tables are scanned in declaration order.

2.4 Registry patches and the two-pass check

MeTTaDinC is registry driven: every declaration head, field, request, argument and result type that a module may use is listed in a machine-readable registry, and the checker of the validation package rejects anything else. The examples need forms the registry does not have yet. Each example therefore ships a JSON patch that declares exactly those forms — libraries with their primitives, new declarations, new fields on existing declarations, new requests and result types — and a checker, `ex_check.py`, that runs two passes. The first pass applies the patches in memory and checks every form of the module files; it must report no errors. The second pass checks the same files against the unpatched registry and lists every diagnostic, which makes the forms that depend on the patch explicit. The patches compose: Example 3 uses the `NatArith` library of Example 1, and Example 4 uses `NatArith` and the `Families` library of Example 2.

2.5 Oracles

Every number in this document was recomputed by an oracle written in Python with exact rationals. Floating point appears only where a section says so: as a first stage that finds the structure of a fixed point before an exact solve, and in the decimals of a few statistical quantities that are not part of any Checked claim.

The oracles compute from the module files. Each bundle also ships a Python model written from the problem statement, and each oracle first establishes that the module and the model define the same objects, so that the claims are computed on the machines the module declares.

- In Examples 1, 3 and 4 a small reference evaluator, the same file in the three bundles, evaluates the maps of the module, and the oracle compares them with the model cell by cell: the observations, actions and encodings of Example 1 on all 256 states and the receipt maps on all 64 receipt sets; the responses, free cells, transfers, symmetries and skills of Example 3 at every cell and command; and the kernels and reward laws of Example 4 on every state-quote cell. Example 1 builds its distance tables, and hence every frontier obligation, from the successor arrays that the module defines.
- Example 2 cannot enumerate its states, so its oracle builds everything from the source. A reader elaborates `access_policy.metta` into a typed intermediate form, Boolean terms over the bits of a word, and the evaluator, the Boolean encoding and the decision diagrams all consume that form; every obligation, witness and reachable set of the oracle comes from it. On a two-user instance of the same source, a direct interpreter of the source terms, the intermediate form and the generated CNF agree on every word, action, query and closure obligation. Two mutations of the source that restore the example's authoring mistakes must change the verdicts they affect. The bundle's independent model, a hand-written

Python policy and encoder that do not read the module, remains as a differential check (Section 4.8).

Examples 1 and 2 also use a satisfiability solver, CaDiCaL 1.5.3 through python-sat. With drat-trim and `cake_lpr` installed, the two oracles convert every DRAT proof to LRAT and check it; without them, they skip the checks that read the checker's verdicts and report the results that rest on the proofs as Conditional. Table 2 gives the number of checks with the checkers installed.

Example	Checks	Time	Extra dependencies
1. Belief memory	92	about two minutes	numpy, networkx, python-sat; drat-trim and `cake_lpr` to check the nine refutation proofs (without them, 91 checks)
2. Access policy	69	about two minutes	numpy, python-sat, dd; drat-trim and `cake_lpr` to check the 619 proofs (without them, 67 checks)
3. Skill sharing	57	a few seconds	none
4. Market making	87	about two seconds	numpy, scipy

Table 2: The oracles.

2.6 Release scope

The first capability manifest covers deterministic dynamics, crisp and Łukasiewicz values, enumerated and symbolic carriers, the core requests, the `GradedKernels` library and the `BitVectors` and `FiniteSets` primitives (specification, Section 16.1). Example 2 lies entirely within it: its patch adds syntax, a few fields and one primitive, but no new kind of claim. Example 1 uses the graded deterministic core of the first release together with proposed extensions and, for its receipt ledger, the `Contracts` library of a later release. Example 3 needs the `Transfers` and `Hives` libraries, and Example 4 the stochastic profile with the `Decisions` and `Linear` libraries; both are conformance targets for the releases that add those capabilities (milestones M6 and M4 of the guide). A first-release implementation answers their requests with Unsupported.

2.7 Reproducing the results

The four bundles are collected in one package, described in Appendix C. Each bundle runs on its own: `python3 exN_oracle.py` recomputes and checks every number and writes a results file, and `ex_check.py`, pointed at the specification's validation package, checks the modules against the registry and the patches. The bundles of Examples 1 and 2 add `replay_proofs.py`, which replays their shipped proofs with drat-trim and `cake_lpr` alone.

3 Example 1: compacting an agent's belief memory

3.1 The problem

An Omega agent records, for each proposition it tracks, how many observations have supported it and how many have gone against it. These are the counts p and n of the evidence form of a PLN truth value [3]. With $t = p + n$, the *strength* is $s = p/t$, or $1/2$ when there is no evidence, and the *confidence* is $c = t/(t + 4)$; the constant 4 plays the role of PLN's lookahead parameter and is chosen small here. Each count saturates at 15, so a belief occupies eight bits. Evidence arrives as `pos` and `neg` events, each adding one to a count. A periodic `decay` event halves both

counts, so that old evidence weighs less than new; in the memory as first designed, halving rounds down.

Two consumers read the memory.

- Reasoning processes read the truth value (s, c). Two truth values are compared coordinate by coordinate, with tolerance $1 - |x - x'|$, so the distance between two readings is the larger of the differences in strength and in confidence.
- The action-selection controller reads a three-way status. A belief is *Believed* when $t \geq 4$ and $p \geq 2n$, *Disbelieved* when $t \geq 4$ and $n \geq 2p$, and *Uncertain* otherwise.

Under memory pressure the designer wants dormant beliefs — those no process is currently reasoning about — stored in fewer bits, and asks which smaller encodings can be trusted, by which consumer, and with what certified error. A belief has 256 states: small enough that every claim can be checked exactly by enumeration, large enough that the answers are not obvious. The example then extends to six beliefs held together (2^{48} states, which must not be enumerated) and to a ledger of evidence receipts whose forks have to be merged, the setting of the specification's evidence-packet laboratory (specification, Section 12.4).

The findings, in brief:

- Neither consumer has a useful exact compaction. The controller's exact quotient saves a fifth of a bit per belief, and the reasoner's saves nothing.
- The original decay rule makes every compact summary of the truth value worthless: two beliefs that differ by a single observation can, after a few decays and evidence events, report opposite truth values. The defect of every candidate summary is 1, the largest possible.
- A gentler decay rule repairs this, and the checker certifies the repair. Under it, a twenty-one-code summary has defect at most 1/5, and no lossy encoding can do better than 6/35.
- "Store it in fewer bits" means three different things. As an index or summary the twenty-one codes are needed; as a cold store that is restored to a representative when evidence arrives, ten codes suffice; a memory that discards the counts and runs on its codes fails at this error level, because the error compounds with every update.
- Memories that keep only counts cannot be merged without double counting, and cannot tell a receipt that arrives a second time from a new one. Receipts, the identities of the evidence, can be compacted to counts below the oldest open fork only under a delivery protocol that keeps those receipts from arriving again.

3.2 Background: what compaction means here

The memory is a deterministic machine with 256 states and three actions, and each consumer sees it through an interface: `Decide` observes the status, and `Report` observes strength and confidence. An encoding of the memory is a representation q in the sense of Section 2.2: the memory would store $q(p, n)$ instead of (p, n).

An *exact* compaction keeps exactly what the consumer can ever observe. It is computed by partition refinement (specification, Section 4.5): start from the classes of the current observation, split every class whose members are sent by some action into different classes, and repeat until nothing splits. The stable partition is behavioral equivalence for that consumer, and its quotient is the smallest memory that answers every question the consumer could ask, now or after any sequence of events. The savings of a partition are measured in bits, $\log_2$ of the number of classes, and in *weakness*, the probability that two independent uniformly drawn beliefs fall in

the same class (specification, Section 4.7); refinement lowers weakness round by round, and the specification records that fall as a ledger.

When exact compaction saves too little, the question becomes graded: how far apart can two beliefs with the same code drift? For a deterministic machine the behavioral distance between two states is the largest observation difference that any word of actions produces from them, with each step of the word discounted by γ (Section 2.2); at $\gamma = 1$ nothing is discounted. The behavioral defect of an encoding is the largest distance between two states with the same code. Through a horizon h only words of length at most h count, which corresponds to a consumer that looks at most h events ahead. For the reasoner's interface the distance before any event is

$$d_0((p, n), (p', n')) = \max(|s - s'|,\ |c - c'|).$$

All of these are computed on the pair graph, whose nodes are the 65536 ordered pairs of states and whose edges follow the three actions in both components at once.

3.3 The program

The core module declares the memory, its two interfaces and the candidate encodings. The excerpt omits two of the five encoding maps; Appendix A has the full file.

```
(DC.Module BeliefMemory
  (Profile Graded)
  (Use NatArith)                          ; PROPOSED library: Le, Min, Quot
  (Carrier Count  (Of Nat) (Enum 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15))
  (Carrier Belief (Of (Tuple Nat Nat)) (Product Count Count))
  (Carrier Status (Of Symbol) (Enum Believed Disbelieved Uncertain))
  (Map strength (From Belief) (To Rational01)
    (Cases ((Tuple $p $n)
      (If (Eq (Add $p $n) 0) (Rat 1 2) (Div $p (Add $p $n))))))
  (Map confidence (From Belief) (To Rational01)
    (Cases ((Tuple $p $n) (Div (Add $p $n) (Add (Add $p $n) 4)))))
  (Map status (From Belief) (To Status)
    (Cases ((Tuple $p $n)
      (If (And (Le 4 (Add $p $n)) (Le (Mul 2 $n) $p)) Believed
        (If (And (Le 4 (Add $p $n)) (Le (Mul 2 $p) $n)) Disbelieved
            Uncertain)))))
  (Map add-pos (From Belief) (To Belief)
    (Cases ((Tuple $p $n) (Tuple (Min (Add $p 1) 15) $n))))
  (Map add-neg (From Belief) (To Belief)
    (Cases ((Tuple $p $n) (Tuple $p (Min (Add $n 1) 15)))))
  (Map halve (From Belief) (To Belief)
    (Cases ((Tuple $p $n) (Tuple (Quot $p 2) (Quot $n 2)))))
  (Tolerance Dist01 (On Rational01) (OneMinusAbsoluteDifference))
  (Machine Decide (States Belief) (Observe status status)
    (Action pos add-pos) (Action neg add-neg) (Action decay halve))
  (Machine Report (States Belief)
    (Observe strength strength (Values Dist01))
    (Observe confidence confidence (Values Dist01))
    (Action pos add-pos) (Action neg add-neg) (Action decay halve))
  ;; the truth value rounded to twentieths, in integer arithmetic
  (Map tv-grid-20 (From Belief) (To (Tuple Nat Nat))
    (Cases ((Tuple $p $n)
      (Tuple (If (Eq (Add $p $n) 0) 10
                 (Quot (Add (Mul 40 $p) (Add $p $n)) (Mul 2 (Add $p $n))))
             (Quot (Add (Mul 41 (Add $p $n)) 4) (Add (Mul 2 (Add $p $n)) 8))))))
  (Map bucket7 (From Count) (To Nat)
```

```
    (Table (0 0) (1 1) (2 2) (3 3) (4 4) (5 4) (6 5) (7 5) (8 5) (9 5)
           (10 6) (11 6) (12 6) (13 6) (14 6) (15 6)))
  (Map buckets7 (From Belief) (To (Tuple Nat Nat))
    (Cases ((Tuple $p $n) (Tuple (bucket7 $p) (bucket7 $n)))))
  (Map cap7 (From Belief) (To (Tuple Nat Nat))
    (Cases ((Tuple $p $n) (Tuple (Min $p 7) (Min $n 7)))))
  (Representation StatusOnly  (Of Decide) (Encode status))
  (Representation RoundedTV20 (Of Report) (Encode tv-grid-20))
  (Representation Buckets7    (Of Report) (Encode buckets7))
  (Representation Capped7     (Of Report) (Encode cap7))
  (Measure Uniform (On Belief) (Uniform))
  (Derive DecisionQuotient (MinimizeForQueries Decide (Under Uniform)))
  (Derive Staged2                                          ; PROPOSED Until
    (Refine StatusOnly (Preserve Encoding) (Until (Steps 2))))
  (Derive Staged2Machine (InduceExact (Output Staged2 Representation)))
  (Derive FewestAtFifth
    (WeakestWithin Report (Within (Rat 1 5)) (Kind Behavioral) (Under Uniform)
                   (Objective Codes) (Search Exhaustive))))   ; PROPOSED Objective
```

A few choices in this module recur throughout the examples. Every map is unary, so functions of two counts take a tuple and destructure it in a `Cases` pattern. The strength map divides by t only in the branch where $t \neq 0$; this relies on `If` evaluating only the branch it selects, which the proposed `NatArith` library states explicitly, because a strict evaluation of both branches would divide by zero. The encoding `tv-grid-20` rounds strength and confidence to the nearest twentieth with integer arithmetic: $\lfloor (40p + t)/2t \rfloor$ is $20s$ rounded to the nearest integer, and $\lfloor (41t + 4)/(2t + 8) \rfloor$ is $20c$ rounded. The two other grid encodings of the file, `tv-grid-10` and `tv-grid-10-7`, round to tenths, and to tenths in strength and sevenths in confidence. `Buckets7` keeps each count only up to one of the buckets $[0], [1], [2], [3], [4, 5], [6, 9], [10, 15]$, and `Capped7` caps both counts at seven.

The repaired memory imports everything and replaces only the decay map. It also declares two encoding families for the frontier searches of Section 3.8:

```
(DC.Module GentleMemory
  (Profile Graded)
  (Use NatArith)
  (Import BeliefMemory (Only Count Belief Status strength confidence status
                             add-pos add-neg Dist01 tv-grid-20 ...))
  ;; decay only above eight observations, rounding up
  (Map halve-gentle (From Belief) (To Belief)
    (Cases ((Tuple $p $n)
      (If (Le 8 (Add $p $n))
          (Tuple (Quot (Add $p 1) 2) (Quot (Add $n 1) 2))
          (Tuple $p $n)))))
  ;; Decide, Report and the representations as before, with decay = halve-gentle
  (EncodingFamily CountBuckets (Of Report)                    ; PROPOSED
    (Generator (IntervalProduct Count Count (Tied True))))
  (EncodingFamily TVGrids (Of Report)
    (Generator (RoundedGrid (strength 1 40) (confidence 1 40))))
  (Derive BestBuckets
    (WeakestWithin Report (Within (Rat 1 5)) (Under Uniform) (Objective Codes)
                   (Family CountBuckets) (Search Exhaustive)))
  (Derive FewestCenters (WeakestWithin Report (Within (Rat 1 5)) (Kind Radius)
                         (Under Uniform) (Objective Codes) (Search Exhaustive)))
  (Derive ColdStoreCenters
    (InduceApprox (Output FewestCenters Representation)
      (Representatives (Choose (Output FewestCenters Representatives)))))
```

```
  (Derive ColdStore (InduceApprox RoundedTV20)))
```

Six beliefs are declared as a parallel composition with a shared decay, and a product encoding:

```
(DC.Module AgentMemory
  (Profile Graded)
  (Use NatArith Parallel)                                          ; PROPOSED
  (Import GentleMemory (Only Report Grid10x7 Fewest))
  (ParallelMachine Beliefs6 (Component Report) (Copies 6) (Synchronize decay))
  (ProductRepresentation Grid6 (Of Beliefs6) (Component Grid10x7) (Copies 6)))
```

The receipt ledger, finally, records six receipts — $u_1, \ldots, u_4$ supporting the proposition and v_1, v_2 opposing it — as a six-bit set, with set union as the merge, and declares the merges a memory could perform if it had kept only counts, as implementation contracts (specification, Section 9.3):

```
  (Carrier Receipts (Of (BitVector 6)) (All))
  (Map counts (From Receipts) (To Tally)
    (Body ($r) (Tuple (PopCount (Slice $r 0 3)) (PopCount (Slice $r 4 5)))))
  (Map hear-u1 (From Receipts) (To Receipts)          ; hear-u2 ... hear-v2 alike
    (Body ($r) (SetBit $r 0 1)))
  (Map union (From (Product Receipts Receipts)) (To Receipts)
    (Body ($pair) (BitOr (GetTuple $pair 0) (GetTuple $pair 1))))
  (Machine Ledger (States Receipts)
    (Observe strength ledger-strength (Values Dist01))
    (Observe confidence ledger-confidence (Values Dist01))
    (Action hear-u1 hear-u1) ... (Action hear-v2 hear-v2) (Merge merge union))
  (Representation Counts (Of Ledger) (Encode counts))
  (Tolerance TallyDistance (On Tally)
    (Meet (Pullback tally-strength Dist01) (Pullback tally-confidence Dist01)))
  (Implementation AddMerge (Ideal merged-tally) (Actual add-tallies)
    (Audit ForkPair) (InputDistance SameInput) (OutputDistance TallyDistance))
```

3.4 Exact compaction for the controller

`DecisionQuotient` is Checked with 225 classes after six refinement rounds. The number of classes grows as $3, 17, 68, 144, 206, 220, 225$, and the weakness ledger falls from 11931/32768 for the three status classes to 81/16384 for the quotient. The only states the quotient identifies are at saturation: $(k, 14)$ with $(k, 15)$ and $(14, k)$ with $(15, k)$ for $k \leq 13$, and the four states with both counts at least 14. Counts 14 and 15 halve to the same 7 and saturate alike, so the controller can never tell them apart. The quotient takes 7.81 bits instead of 8, which is not worth having.

The status alone cannot be maintained: `CheckUpdates StatusOnly` is Refuted with the pair $((0, 0), (0, 3))$ and the action `pos`. Both beliefs are Uncertain, but `pos` turns the second into $(1, 3)$, which is Disbelieved, and the first into $(1, 0)$, which is still Uncertain.

A planner that looks h events ahead needs only the *staged* partition P_h, agreement through horizon h, which has 17, 68 and 144 classes for $h = 1, 2, 3$. These partitions are snapshot summaries rather than machines. `Staged2Machine` is Refuted on the update obligation with the pair $((0, 0), (0, 1))$ and the action `pos`: the two beliefs agree on every status through two events, but `pos` sends them to $(1, 0)$ and $(1, 1)$, and two more `pos` events give $(3, 0)$, Uncertain, against $(3, 1)$, Believed. The specification's warning that a bounded-horizon certificate supports a staged interface and not an abstract machine (specification, Section 4.5) applies here with a concrete witness.

3.5 Graded summaries under the original decay

For the reasoner nothing exact is worth having either, so the question becomes graded. Table 3 gives the behavioral defect of four candidate encodings: on current reports ("now"), through horizons one to three, over all futures, and with discount 9/10.

Encoding (codes)	now	$h = 1$	$h = 2$	$h = 3$	all	$\gamma = 9/10$
RoundedTV20 (113)	5/104	1/10	1/4	1/2	1	0.9^6
RoundedTV10 (53)	33/340	1/5	1/2	1	1	0.9^3
Buckets7 (49)	1/5	1/5	1/3	1	1	0.9^3
Capped7 (64)	4/11	2/5	1/2	1	1	0.9^3

Table 3: Behavioral defects under the original decay rule (halve, rounding down).

Every update defect is 1 with crisp codes; for `RoundedTV20` the update witness is the pair $((0, 7), (0, 8))$ under `pos`, whose successors have the codes $(3, 13)$ and $(2, 14)$.

`CheckBehavioral RoundedTV20 (Within 1/5)` is Refuted. The witness is the pair $((3, 11), (4, 14))$, both coded $(4, 16)$, with the word `decay decay`: the runs end at $(0, 2)$ and $(1, 3)$, with strengths 0 and 1/4. The weaker claim `(Within 1/2)` is refuted too, at length six: the pair $((11, 12), (12, 11))$ with the word `decay decay pos neg decay decay` ends at $(0, 1)$ and $(1, 0)$, with strengths 0 and 1. The two beliefs began with strengths 11/23 and 12/23.

The mechanism is arithmetic. Rounding down can preserve a difference of one count while halving the total: 11 and 12 become 5 and 6, then 2 and 3. Evidence events between the decays fix the parities so that the difference survives, and each decay doubles its relative size. The distribution of distances shows how complete the damage is. Of the 32640 pairs of distinct states, 34 lie within 1/29 of each other, the least distance being 1/58; these are the saturation pairs. Every other pair is at least 1/2 apart, and 14161 pairs are at distance 1. No distance falls in between, so every lossy encoding that merges anything beyond saturation has defect at least 1/2.

In plain terms. Under the original decay, beliefs that agree almost exactly today can be driven, by a plausible sequence of a few events, to report opposite conclusions. A summary of such a memory cannot be trusted at any useful error level, however many bits it keeps, unless it keeps everything.

3.6 The decay rule is a design parameter

The example therefore compares decay rules. Table 4 reports, for each rule, the undiscounted defect of `RoundedTV20`, the size of the controller's exact quotient, and the *floor*, the least behavioral distance between two distinct states. No lossy encoding can have a defect below the floor.

Decay rule	RoundedTV20	Exact decision classes	Floor
halve, round down	1	225	1/58
halve, round up	1/3	256	1/6
halve, round down, only if $t \geq 8$	4/15	225	1/58
halve, round up, only if $t \geq 8$ (gentle)	1/5	256	6/35
subtract one from each count	1	256	1/2

Table 4: Decay rules compared.

The gentle rule never erases small counts: below eight observations it does nothing, and above it rounds up. The price is that the controller's exact quotient disappears, because the rule tells 14 apart from 15. The third row is a compromise that keeps the 225-class quotient and still bounds the summary at 4/15. Under the gentle rule the defects of Table 5 replace those of Table 3.

Encoding (codes)	now	$h = 1$	$h = 2$	$h = 3$	all	$\gamma = 9/10$
RoundedTV20 (113)	5/104	3/22	1/7	3/20	1/5	27/220
Grid10x7 (43)	3/22	1/6	1/5	1/5	1/5	81/500
RoundedTV10 (53)	33/340	3/20	1/5	1/5	7/30	81/500
Buckets7 (49)	1/5	1/4	1/4	1/4	1/4	9/40
Capped7 (64)	4/11	4/11	4/11	4/11	4/11	4/11

Table 5: Behavioral defects under the gentle decay rule.

`CheckBehavioral RoundedTV20 (Within 1/5)` is now Checked. `(Within 3/20)` is Refuted at length five, with the pair $((4, 10), (5, 11))$ and the word `decay decay pos pos decay`; the runs end at $(2, 3)$ and $(4, 3)$, with strengths 2/5 and 4/7, a distance of 6/35. That is the floor, and 5106 pairs of states attain it. Even $(14, 14)$ and $(15, 15)$, whose reports differ by only 1/136, reach it after a word of eleven events. The cause is runs that straddle the threshold of the rule: at $t = 7$ a belief is left alone while its neighbor at $t = 8$ is halved.

The undiscounted fixed point takes 15 rounds of iteration on the pair graph under the original rule and 23 under the gentle one; with discount 9/10 it stabilizes exactly after 12 rounds in both cases.

3.7 Three uses, three claims

"Store the belief in fewer bits" can mean three different things, and MeTTaDinC has a separate claim for each.

1. *An index or summary.* The code serves as a key — for example, to share planning results across beliefs — while the counts stay available. The criterion is the behavioral defect of the representation: two states with the same code have futures within the bound.
2. *Cold storage with rehydration.* A dormant belief is replaced by its code and restored to a fixed representative of its class when evidence next arrives. The error is the *radius*, the largest distance from a state to the representative of its class (`InduceApprox`, specification, Section 4.6). Each compression cycle adds at most one radius, because at discount one the behavioral distance of two states is never smaller than that of their successors: the words available from the successors are suffixes of words available from the states. Decays missed while the belief was stored are handled by recording a decay epoch with the code and replaying the missed decays on restore; the same bound covers them.
3. *A compact memory that runs on its codes.* The counts are discarded and updates go through the induced abstract machine. The criterion is the arrow defect of the induced machine, and errors compound with every update.

At the bound 1/5 under the gentle rule the three uses give different answers. A summary needs 21 codes (4.39 bits). A cold store needs 10 codes (3.32 bits). Running on codes fails at this level: `RoundedTV20` has radius 6/35, but its induced machine drifts to an arrow defect of 20/63. The drift is 3/22, 3/22, 3/20 and 6/35 at horizons one to four and reaches 20/63 only at horizon 33. `Grid10x7`, with the same radius, drifts to 29/55, and among the 1600 rounding grids of the `TVGrids` family none runs on its codes, with center representatives, within 1/4; the best of

them drifts to 4/15.

In plain terms. A belief that is put away and later restored loses at most a radius each time it is put away. A belief that is kept only as a code and updated as a code loses a little at every update, and the losses add up. The same encoding can be good for the first use and bad for the second, and the language makes the user say which use is meant.

3.8 The frontier and its certificates

With the codes objective, the frontier is a graph coloring problem. Call two states in *conflict* when their behavioral distance exceeds the bound. An encoding whose defect is within the bound is exactly a coloring of the conflict graph, colors being codes, so the fewest codes is the chromatic number of that graph. Coloring is NP-hard in general, and the specification's status discipline applies (specification, Section 4.7): the defect of a returned partition is always certified, and its optimality is Checked only with a certificate. Two certificates make an answer exact. The coloring itself is one; its defect is recomputed from the distance table. A lower bound is the other: either a clique of pairwise conflicting states, each conflict replayable as a word, or a refutation of the problem with one code fewer.

The refutation is a satisfiability obligation in conjunctive normal form: one variable for each state and code, a clause saying that each state receives a code, a clause for each conflicting pair and code, and unit clauses that give the states of a largest clique the first codes, which rules out no coloring up to a renaming of the codes. CaDiCaL finds the obligation unsatisfiable and writes a DRAT proof, the list of clauses it derived in order, each of which a much simpler program can confirm; drat-trim converts the proof to the LRAT format and the verified checker `cake_lpr` checks it. This is the specification's pinned pipeline (guide, Section 1.8). The obligations are built from the distance table of the module's own machine (Section 2.5); the one for twenty codes at 1/5 has 5120 variables and 501,856 clauses. The bundle keeps every refuted CNF with its DRAT proof, every coloring and every clique. With drat-trim and `cake_lpr` installed, the oracle replays the nine refutations of this section, four for codes and five for centers, and every optimum below is Checked. Without them, the optima whose lower bound is a refutation are Conditional on `(TrustedBackend cadical 1.5.3 SAT sha256:...)`, the digest naming the refuted obligation, while the optima certified by a clique stay Checked. Table 6 gives the frontier under the gentle rule at discount one.

Bound	Largest clique	Fewest codes	Proof of optimality
6/35	32	33	refutation with 32 codes
1/5	20	21	refutation with 20 codes
1/4	18	19	refutation with 18 codes
3/10	12	12	clique
1/3	8	8	clique
2/5	6	6	clique
1/2	4	5	refutation with 4 codes

Table 6: Fewest codes for a summary within each bound, gentle rule, discount one. Each refutation is a replayed DRAT proof.

The radius kind of the frontier, used by the cold store, is a minimum dominating set: choose centers so that every state lies within the bound of some center. It needs 12 centers at 6/35, 10 at 1/5 and at 1/4, 4 at 1/3 and 2 at 1/2. Each lower bound is a refutation of one center fewer, with one variable per candidate center, a clause per state listing the centers within the bound of it, and a sequential-counter encoding of "at most k centers"; the nine centers at 1/5,

for instance, are refuted by an obligation of 2479 variables and 4940 clauses. One optimal set at 1/5 is $(0,0)$, $(0,2)$, $(0,8)$, $(1,2)$, $(2,0)$, $(3,11)$, $(4,2)$, $(7,8)$, $(8,0)$, $(11,3)$.

Under the original rule the frontier is flat over most of its range. Below 1/58 no two distinct states lie within the bound, so all 256 states need their own codes. The 34 pairs within 1/29 are the saturation pairs of the exact quotient: 33 of them are exactly 1/58 apart, and the remaining one, $(14,15)$ with $(15,14)$, is 1/29 apart. On $[1/58, 1/29)$ every saturation pair except that one may share a code; the four states with both counts at least 14 then need two codes, because $(14,15)$ and $(15,14)$ still conflict, and a summary needs 226 codes. On $[1/29, 1/2)$ it needs 225, the classes of the exact quotient. Every other pair is at least 1/2 apart, so below 1/2 the fewest codes is a sum over the small groups of states that may share codes, and in each case the lower bound is a clique, of 256, 226 or 225 states, replayed pair by pair. At 1/2 a summary needs 17 codes, again with a clique. With discount 9/10 the gentle rule needs 256 codes at 1/20, 54 at 1/10, 26 at 3/20 and 15 at 1/5, while the original rule needs 225 for each of these bounds; all eight values are certified by cliques.

Declared families are searched exhaustively, so optimality within a family is Checked. Over the 32768 tied bucketings of the `CountBuckets` family the best summary within 1/5 has 49 codes, with buckets $[0], [1], [2], [3,4], [5,7], [8,10], [11,15]$; over the 1600 rounding grids of `TVGrids` the best has 43 codes, strength to tenths and confidence to sevenths. Both lose heavily against the unrestricted 21. The registered weakness objective with greedy search gives 24 classes with weakness 1187/16384, and its optimality is reported as Unknown.

3.9 Six beliefs

The agent holds many beliefs, and the specification's product rule lets claims about one belief carry over to several without enumerating the product. `Beliefs6` has 2^{48} states.

Proposition 3.1 (Product rule)**.** *Let $M_1, \ldots, M_k$ be deterministic machines and let M be their parallel composition, in which every action either acts on all components at once or on exactly one of them, and the observation distance of M is the maximum of the components' observation distances. Then for every discount and every horizon,*

$$d^{M}_{\mathrm{beh}}((x_i)_i, (y_i)_i) = \max_i \; d^{M_i}_{\mathrm{beh}}(x_i, y_i).$$

Consequently a product encoding has the current, behavioral and radius defects of its component encodings, its number of codes is the product of theirs, and its weakness under the product law is the product of their weaknesses.

Proof. A word of M acts on component i through its projection, the subsequence of actions that reach i, which is never longer than the word; discounting by length therefore gives at most the component's own value, so the left side is at most the right. Conversely every word of M_i lifts to a word of M of the same length, by replacing each component action with the corresponding local or global action, and the lifted word produces the same observation difference in component i. The supremum over words therefore commutes with the maximum over components. The same argument applies word length by word length, which gives the horizon form. □

The oracle confirms the proposition on two copies of a sixteen-state memory, for both decay rules and both discounts, over all 65536 pairs. `CheckBehavioral Grid6 (Within 1/5)` is therefore Checked from the component certificate, an exact computation that carries no assumptions. Table 7 gives the cost of six beliefs.

A joint encoding, such as a codebook shared across beliefs, falls outside the product rule. A request about one should return Unknown with a diagnostic that names the rule, rather than attempt to enumerate 2^{48} states.

Encoding of six beliefs	Bits
raw counts	48
exact decision quotient (original rule)	46.9
RoundedTV20	40.9
Grid10x7	32.6
the twenty-one-code summary	26.4
the ten-center cold store	19.9

Table 7: Bits for six beliefs.

3.10 Forks and merges

Agents fork: two processes take copies of a memory, hear different evidence, and later merge. The receipt ledger models this with six receipts and union as the merge.

- `CheckNow Counts` is Checked: the counts determine the truth value.
- `CheckUpdates Counts` is Refuted with the pair $(\{u_1\}, \{u_2\})$ and the action `hear-u1`, giving tallies $(1, 0)$ against $(2, 0)$. Hearing a receipt twice is harmless only if one knows it is the same receipt.
- `MeasureBehavioral Counts` is Checked with defect $1/3$, attained at $(\{u_1, v_1\}, \{u_2, v_2\})$.

A memory that kept only counts can still try to merge two forks, and the contracts measure how badly each attempt fails.

- Adding the tallies has worst error $6/35$. `CheckImplementation AddMerge (Grade 1 1/6)` is Refuted at $(\{u_1, u_2, u_3\}, \{u_1, u_2, u_3\})$, which gives $(6, 0)$ instead of $(3, 0)$.
- Taking the maximum also has worst error $6/35$; the witness is $(\{u_1, u_2, v_1\}, \{u_3, u_4, v_2\})$, which gives $(2, 1)$ instead of $(4, 2)$.
- No rule on counts does better. If both forks hold the tally $(2, 1)$, the true union may be any of six tallies from $(2, 1)$ to $(4, 2)$, and no single tally lies within less than $6/35$ of all of them.
- A three-way merge from the fork base, which adds what each fork heard since the base, is Refuted over all forks with worst error $6/35$; the least witness is the empty base with both forks $\{u_1\}$, with error $2/15$. Restricted to forks whose new receipts are disjoint (`CleanForks`) it is Checked with error 0.

Forgetting an identity is safe only relative to what can happen next. A natural design keeps the identities of receipts newer than the oldest open fork and compacts older receipts to counts. The fork horizon takes care of merges: two forks opened after the compaction share the compacted part, and merging them adds only newer receipts, which still carry their identities. It does not take care of an old receipt that arrives again, through a delayed delivery, a retransmission, or a later operation that names it. The ledger module declares such a compaction, with u_1 and u_2 below the horizon, and a machine whose actions are those of a delivery protocol that never delivers them again:

```
(Map compact (From Receipts) (To Compact)
  (Body ($r) (Tuple (PopCount (Slice $r 0 1)) (Slice $r 2 5))))
(Representation Compacted (Of Ledger) (Encode compact))   ; every hear action
  allowed
(Machine SettledLedger (States Receipts)                   ; u1 and u2 never
  arrive again
  (Observe strength ledger-strength (Values Dist01))
```

```
    (Observe confidence ledger-confidence (Values Dist01))
    (Action hear-u3 hear-u3) (Action hear-u4 hear-u4)
    (Action hear-v1 hear-v1) (Action hear-v2 hear-v2))
  (Representation CompactedSettled (Of SettledLedger) (Encode compact))
```

- On the ledger, where every hear action remains available, `CheckUpdates Compacted` is Refuted with the pair $(\{u_1\}, \{u_2\})$ and the action `hear-u1`. Both sets compact to one old receipt and no newer ones. `hear-u1` leaves the first at one receipt and takes the second to two, so the confidence becomes 1/5 in one run and 1/3 in the other, and no update on the compacted codes can produce both. `CheckBehavioral Compacted (Within 1/10)` is Refuted with the same pair and the same one-action word, a gap of 2/15, and `MeasureBehavioral Compacted` is Checked with defect 1/6.
- `SettledLedger` has exactly the operations of a protocol in which u_1 and u_2 are never delivered again, for instance a delivery watermark at u_2 below which redeliveries are dropped, with forks opened from the compacted state. On it `CheckUpdates CompactedSettled` is Checked and `MeasureBehavioral CompactedSettled` is Checked with defect 0.
- A merge of two compacted forks that reads only their codes, taking the old count of the first and the union of the newer identities (`CompactMerge`), is Refuted over all fork pairs, with worst error 2/3 and least witness $(\emptyset, \{u_1\})$ at error 1/2. Restricted to forks that share the compacted part (`CompactMergeSettled`) it is Checked with error 0.

The design rule is therefore a statement about a protocol. Receipts older than the oldest open fork may be compacted to counts when the protocol also ensures that none of them is delivered or named again: a closed receipt namespace whose old receipts have all been delivered, a delivery watermark that drops redeliveries below it, deduplication tombstones that keep the old identities in a cheaper form, or the exclusion of old hear operations after compaction. The fork horizon is one part of such a protocol, and the certificate holds for the operations declared: `compact` is exact on `SettledLedger` and fails on `Ledger`.

In plain terms. A count cannot tell a receipt that arrives a second time from a new one. Dropping the identities of old evidence is safe only when the rest of the system promises that old evidence will not arrive again, and the check has to be made against the operations that promise leaves open.

3.11 Expected results

Request	Status	Values
`DecisionQuotient` (original rule)	Checked	225 classes, weakness 81/16384
`DecisionQuotient` (gentle rule)	Checked	256 classes
`CheckUpdates StatusOnly`	Refuted	$((0,0),(0,3))$, `pos`
`Staged2`, `Staged2Machine`	Checked, Refuted	68 classes; $((0,0),(0,1))$, `pos`
`Defects RoundedTV20` (original)	Checked	now 5/104, update 1, behavioral 1
`CheckBehavioral RoundedTV20 (Within 1/5)` (original)	Refuted	$((3,11),(4,14))$, `decay decay`
`CheckBehavioral RoundedTV20 (Within 1/2)` (original)	Refuted	$((11,12),(12,11))$, length six

Request	Status	Values
`CheckBehavioral RoundedTV20 (Within 1/5)` (gentle)	Checked	defect 1/5
`CheckBehavioral RoundedTV20 (Within 3/20)` (gentle)	Refuted	$((4, 10), (5, 11))$, length five
`FewestAtFifth`, `FewestAtHalf` (original)	Checked	225 and 17 codes; cliques
`Fewest` (gentle, 1/5)	Checked	21 codes; refutation with 20, proof replayed
`FewestCenters` (gentle, 1/5)	Checked	10 centers; 9 refuted, proof replayed
`BestBuckets`, `BestGrid`	Checked	49 and 43 codes
`WeakestGreedy`	Checked	24 classes, optimality Unknown
`ColdStore` (`InduceApprox RoundedTV20`)	Checked	radius 6/35, arrow defect 20/63
`CheckBehavioral Grid6 (Within 1/5)`	Checked	by the product rule
`CheckUpdates Counts`	Refuted	$(\{u_1\}, \{u_2\})$, `hear-u1`
`MeasureBehavioral Counts`	Checked	1/3
`CheckImplementation AddMerge (Grade 1 6/35)`	Checked	
`CheckImplementation AddMerge (Grade 1 1/6)`	Refuted	$(\{u_1, u_2, u_3\}, \{u_1, u_2, u_3\})$
`CheckImplementation ThreeWay`, `ThreeWayClean`	Refuted, Checked	error 6/35; 0
`CheckUpdates Compacted`	Refuted	$(\{u_1\}, \{u_2\})$, `hear-u1`
`CheckBehavioral Compacted (Within 1/10)`	Refuted	the same pair, `hear-u1`: confidence 1/5, 1/3
`MeasureBehavioral Compacted`	Checked	1/6
`CheckUpdates CompactedSettled`, `MeasureBehavioral CompactedSettled`	Checked	defect 0
`CheckImplementation CompactMerge`, `CompactMergeSettled`	Refuted, Checked	error 2/3; 0

Table 8: Expected results for Example 1. The two optima whose lower bound is a refutation are Checked when the proofs are replayed and Conditional on the solver otherwise.

3.12 Notes for implementers

Undiscounted distances. For a deterministic machine, the undiscounted distance of a pair is the largest observation distance over the pairs reachable from it in the pair graph. A strongly-connected-component pass over the pair graph computes it in linear time (65536 nodes and 3×65536 edges here). Plain iteration also converges, after 15 rounds under the original rule and 23 under the gentle one, but the number of rounds has no useful general bound.

Discounted and horizon distances. Iterate from the observation distances until nothing changes; with discount 9/10 the iteration stops exactly after 12 rounds here. Horizon h is exactly h rounds.

Exact arithmetic. Store distances as ranks into the sorted list of distinct values — 50 values under the original rule and 1445 under the gentle one — and convert to rationals only for results. Comparisons, maxima and the fixed point then run on small integers.

Certificates for a defect. The upper bound is the distance table itself: one pass confirms that each entry is at least the observation distance and at least the entry of each successor pair. The lower bound is a replayed word.

Canonical witnesses. A backward breadth-first search from the violating pairs gives the shortest violating length of every pair in one pass. A forward search from the least pair at that length, expanding actions in declaration order, then gives the canonical word.

Frontiers. A satisfiability solver with clique-based symmetry breaking (the clique's states fixed to distinct colors) solves each coloring instance here in under a second, and drat-trim and `cake_lpr` check each of the nine refutations in under a second as well. Under the original rule no solver is needed below 1/2: the pairs that may share a code fall into small groups, every state of one group conflicts with every state of another, and the fewest codes and the largest clique are sums over the groups. Partitions returned by an optimizer are not canonical, so the manifest should record their properties — the number of codes, the defect within the bound, the kind of optimality certificate — rather than the partition itself. A budget that ends before a certificate is found yields Unknown, with the best clique bound in `Resources`; a refutation whose proof is not replayed yields Conditional, with the solver's trusted-backend assumption.

Merges. A merge reduces to translations. For a union merge, translation by single receipts suffices, and those translations are exactly the `hear` actions: six checks instead of sixty-four. A compaction that forgets identities must be checked on the machine of the declared delivery protocol, whose actions are the operations still possible after compaction; checked on the full ledger it fails.

Behaviors the fixtures pin down. `If` evaluates only its chosen branch, and the $t = 0$ guards depend on it. `Quot` is floor division. The `Center` selector breaks ties by declaration order. `Product` varies the last component fastest, which fixes the order of pairs in witnesses. Codes are crisp by default, so update defects are 0 or 1.

3.13 Proposed registry additions

- A `NatArith` library with `Le`, `Lt`, `Min`, `Max`, `Quot` and `Rem`. A zero divisor returns a tagged failure value, and `If` evaluates only one branch.
- `BitOr`, `BitAnd`, `BitXor` and `PopCount` in `BitVectors`.
- A tolerance body `(Meet t ...)`, the pointwise minimum of tolerances.
- `Refine` with `(Until (Steps k))`, which returns the staged partition.
- For `WeakestWithin`: the arguments `(Objective Weakness|Codes)`, `(Family f)` and `(Kind Radius)`; the result fields `Codes`, `LowerBound` and `Representatives`; and a status rule that lets optimality be Checked without exhaustive enumeration, by a replayed clique or by a refutation whose DRAT proof the checker replays; with a refutation that has not been replayed, optimality is Conditional on the solver.
- An `EncodingFamily` declaration with the generators `IntervalProduct` and `RoundedGrid`.

- A `Parallel` library with `ParallelMachine` and `ProductRepresentation`, whose method is the product rule.

One question is left open: there is no frontier yet for the third use, a memory that runs on its codes. Each candidate costs a fixed-point computation of its induced machine, and the drift results above suggest that it will rarely be the objective anyone wants.

4 Example 2: auditing an access policy without enumeration

4.1 The problem

Many organizations control access with roles. Users are assigned roles, roles carry permissions, and a user exercises a permission by activating one of their roles in a session. In *administrative* role-based access control the assignments themselves are governed by rules: an administrator may assign a role to a user only when preconditions hold, and revoking one role may revoke others that depend on it [4, 5]. Separation of duty is the usual safety requirement: no single person may both approve payments and audit them. Policies of this kind are written by hand, grow by accretion, and are hard to audit. Each rule may look sensible on its own while some sequence of individually legitimate administrative steps still reaches a configuration that violates the requirement, and finding such a sequence by inspection is slow and unreliable. Analysis tools for such policies exist [6]; the example shows how the same questions are posed and answered in MeTTaDinC, with certificates.

The organization has four users (alice, bob, carol, dave) and five roles (staff, clerk, approver, auditor, admin). A user may hold a role (it is *assigned*) and may have it *active* in a session. Permissions come only from active roles: read from staff, clerk, approver or auditor; write from clerk or approver; approve from approver; audit from auditor; configure from admin. The administrative rules are these.

- Two-person rule: assigning or revoking any role for user u requires some *other* user with admin active.
- Preconditions: clerk requires staff; approver requires clerk and not auditor; auditor requires staff and not approver; admin requires staff.
- Cascades: revoking staff removes every role of the user; revoking clerk also removes approver.
- Sessions: a role can be activated only if it is assigned, and approver and auditor are never active together.

The state is a forty-bit word, ten bits per user: five assigned bits followed by five active bits, bit $10u + r$ for "u holds role r" and bit $10u + 5 + r$ for "u has r active". That gives about 1.1×10^{12} configurations, 80 actions (four kinds, four users, five roles) and 20 access observers (five permissions for four users).

The policy as written contains two plausible authoring mistakes. The approver rule checks whether the user has auditor *active* rather than *assigned*, and revoking clerk forgets to remove approver. The start configuration is the word 34672145: alice holds staff and admin, with admin active; bob holds staff and clerk, with clerk active; carol holds staff, active; dave holds nothing.

The findings, in brief:

- The policy as written has two holes, found by canonical witnesses of two and three steps.
- The corrected policy is proved safe without enumeration. It reaches 38,418,640 configurations, and 80 small proofs cover them all; nothing near 2^{40} is ever listed.

- The proof obligations are generated from the module itself, so the audit is about the policy the module declares. Restoring either mistake in the source turns the corresponding proof into a refutation, with the witness found for the policy as written.
- A true property can come back Unknown, when its proof needs a stronger invariant than the property itself. A counterexample to induction is not a refutation.
- For a targeted question, slicing does most of the compaction: the state relevant to "can bob approve" shrinks from forty bits to seventeen, and a vendor's ten-bit slice is refuted.
- The full access interface has no exact compaction at all: forty short tests read off every bit. The only exact saving is restriction to the reachable configurations, about 25.2 bits' worth.

4.2 Background: proofs about a space that cannot be listed

On a symbolic carrier (specification, Section 5.3) states are bit vectors and maps are terms over registered primitives such as bit access, Boolean connectives and bit setting. The checker never enumerates the carrier; it turns each claim into proof obligations over the terms and discharges them with a satisfiability solver or with binary decision diagrams.

Safety as a partition. The audit states each safety requirement as a one-block `BlockPartition` on the reachable scope `(Reach Policy (From start))`: the block is the set of safe configurations, and the coverage obligation says that every reachable configuration lies in it. `CheckBlockPartition` is a first-release request (specification, Section 16.1).

Inductive invariants. The reachable set is not known in advance, so coverage is proved by induction. A set I that contains the start configuration and is closed under every action is an *inductive invariant*; it contains every reachable configuration, and if it lies inside the block, coverage holds. Each closure obligation, "if $x \in I$ then $T_a x \in I$", becomes one satisfiability query: the solver is asked for an x in I whose successor leaves I. An unsatisfiable answer, with a proof, discharges the obligation. A satisfiable answer is a *counterexample to induction*: a configuration from which one step leaves the set. It may be unreachable, so it does not refute the claim (specification, Section 5.3); a refutation needs a word from the start configuration.

Proofs that a second program can check. The obligations are encoded in conjunctive normal form by the Tseitin transformation [7], solved with CaDiCaL, and every unsatisfiability proof is emitted in the DRAT format, converted to LRAT by drat-trim [9] and checked by `cake_lpr`, a proof checker verified in CakeML [10]. This is the specification's pinned pipeline (specification, Section 15.3; guide, Section 1.8); trust rests on the verified checker, not on the solver. A proof certifies the formula it was produced for, so the formula has to be generated from the admitted module by the obligation generator, which the specification places inside the trusted boundary; Section 4.8 describes how the example's oracle generates the obligations and tests the generator. A result whose proofs have not been replayed is Conditional on the solver (Section 2.1).

Exact reachable sets. For counting the scope exactly, binary decision diagrams [8] compute the reachable set as a fixed point of image computations. The image of a set R under a guarded update that sets bit b to v when the guard g holds is $(R \wedge \neg g) \vee ((\exists b.\, R \wedge g) \wedge b = v)$, provided the guard does not read the bits it writes.

In plain terms. A proof by induction says that the policy never leaves the safe region because no single step can leave it. When some single step can leave it, from some configuration, the proof has failed, but the policy may still be safe: that configuration might never occur. Only a concrete sequence of steps from the actual starting configuration shows that the policy is unsafe.

4.3 The program

The module uses the proposed `Families` library of indexed declarations. `Parameters` on a map declares a family of maps indexed by elements of enumerated carriers; `(For $v C)` on an `Observe` or `Action` entry declares one entry per element; `Switch` selects on a parameter, and `Exists` and `ForAll` range over an enumerated carrier. All of these are expanded, and bit indices constant-folded, before checking. The order of the `For` clauses fixes the canonical order of the 80 actions: kind first, then user, then role. An excerpt of the policy as written:

```
(DC.Module AccessPolicy
  (Profile (Dynamics Deterministic) (Values Crisp) (Carriers Symbolic))
  (Use BitVectors Families)                                    ; PROPOSED library
  (Carrier Word (Of (BitVector 40)) (All))
  (Carrier User (Of Symbol) (Enum alice bob carol dave))
  (Carrier Role (Of Symbol) (Enum staff clerk approver auditor admin))
  (Map holds (Parameters ($u User) ($r Role)) (From Word) (To Bool)
    (Body ($x) (Eq (BitAt $x (Add (slot $u) (rank $r))) 1)))
  (Map active (Parameters ($u User) ($r Role)) (From Word) (To Bool)
    (Body ($x) (Eq (BitAt $x (Add (Add (slot $u) 5) (rank $r))) 1)))
  (Map other-admin (Parameters ($u User)) (From Word) (To Bool)
    (Body ($x) (Exists $v User (And (Not (Eq $v $u)) (active $v admin $x)))))
  (Map may-receive (Parameters ($u User) ($r Role)) (From Word) (To Bool)
    (Body ($x)
      (Switch $r
        (staff    True)
        (clerk    (holds $u staff $x))
        ;; as written, the approver rule reads the auditor session
        (approver (And (holds $u clerk $x) (Not (active $u auditor $x))))
        (auditor  (And (holds $u staff $x) (Not (holds $u approver $x))))
        (admin    (holds $u staff $x)))))
  (Map assign (Parameters ($u User) ($r Role)) (From Word) (To Word)
    (Body ($x) (If (And (other-admin $u $x) (may-receive $u $r $x))
                   (grant $u $r $x) $x)))
  (Map revoke (Parameters ($u User) ($r Role)) (From Word) (To Word)
    (Body ($x)
      (If (other-admin $u $x)
          (Switch $r
            (staff    (drop $u staff (drop $u clerk (drop $u approver
                        (drop $u auditor (drop $u admin $x))))))
            (clerk    (drop $u clerk $x))                        ; keeps approver
            (approver (drop $u approver $x))
            (auditor  (drop $u auditor $x))
            (admin    (drop $u admin $x)))
          $x)))
  ;; activate, deactivate, the permission maps and the alarms: Appendix A
  (Machine Policy (States Word)
    (Observe (read $u) (may-read $u) (For $u User))   ; write, approve, ... alike
    (Action (assign $u $r)     (assign $u $r)     (For $u User) (For $r Role))
    (Action (revoke $u $r)     (revoke $u $r)     (For $u User) (For $r Role))
    (Action (activate $u $r)   (activate $u $r)   (For $u User) (For $r Role))
    (Action (deactivate $u $r) (deactivate $u $r) (For $u User) (For $r Role)))
  (Const start Word 34672145)
  (Carrier Live (Of (BitVector 40)) (Reach Policy (From start)))
  (Machine SoDWatch (States Live) (Observe breach sod-breach)       ; PROPOSED field
    (ActionsOf Policy))
  (BlockPartition NoBreach (On Live)
    (Block Quiet (Body ($x) (Not (sod-breach $x))) start)
```

```
    (Successor Any Quiet Quiet))                                    ; PROPOSED Any
  ;; ClerksOnly and WellFormed alike
  ...)
```

The three safety requirements are separation of duty (`sod-breach`: some user holds both approver and auditor), no approval without clerk (`clerkless-approval`: some user has approver active without holding clerk), and well-formedness (`consistent`: every active role is assigned, the role dependencies hold, and nobody holds approver and auditor together). `SoDWatch` observes only its alarm and borrows the 80 actions of the policy through the proposed `ActionsOf` field, and the block partition says that the alarm is constant (off) on the reachable scope, with every action keeping the block in itself.

The corrected module imports everything shared, fixes the two rules, and adds the audit's certificates, slices and quotients:

```
(DC.Module AccessPolicyFixed
  ;; Profile, Use and the import of the shared maps as before
  (Map may-receive ...
       ;; corrected: the approver rule reads the auditor assignment
       (approver (And (holds $u clerk $x) (Not (holds $u auditor $x))))
       ...)
  (Map revoke ...
           (clerk (drop $u clerk (drop $u approver $x)))      ; cascades
           ...)
  (Machine LivePolicy (States Live) (Like Policy))            ; PROPOSED
  (BlockPartition ClerksOnlyProved (On Live)
    (Block Quiet (Body ($x) (Not (clerkless-approval $x))) start)
    (Successor Any Quiet Quiet)
    (Invariant consistent))                                   ; PROPOSED
  (Map assignments (From Word) (To (BitVector 20))
    (Body ($x) (Gather $x 0 1 2 3 4 10 11 12 13 14 20 21 22 23 24
                          30 31 32 33 34)))                   ; PROPOSED
  (Representation AssignmentsOnly (Of Policy) (Encode assignments))
  (Machine BobApproves (States Word) (Observe approve-bob (may-approve bob))
    (ActionsOf Policy))
  (Map bob-cone (From Word) (To (BitVector 17))
    (Body ($x) (Gather $x 0 4 9 10 11 12 13 14 17 18 19 20 24 29 30 34 39)))
  (Map bob-bits (From Word) (To (BitVector 10)) (Body ($x) (Slice $x 10 19)))
  (Representation BobCone (Of BobApproves) (Encode bob-cone))
  (Representation BobOnly (Of BobApproves) (Encode bob-bits))
  (Derive WholeInterface (MinimizeForQueries Policy))
  (Derive LiveInterface  (MinimizeForQueries LivePolicy))
  (Derive BobSlice       (InduceExact BobCone))
  (Derive BobQuotient    (MinimizeForQueries BobApproves)))
```

4.4 The policy as written

Table 9 gives the three partitions on the policy as written, with the canonical witness of each refutation.

In the first witness bob receives auditor without activating it, so the approver rule, which reads the session, lets him receive approver as well: separation of duty is breached in two steps. In the second, revoking clerk leaves approver assigned, and bob then activates it and can approve without being a clerk.

The checker reaches these answers in three stages.

Partition	Status	Canonical witness from `start`
NoBreach	Refuted (coverage)	`(assign bob auditor) (assign bob approver)`
ClerksOnly	Refuted (coverage)	`(assign bob approver) (revoke bob clerk) (activate bob approver)`
WellFormed	Refuted (coverage)	`(assign bob approver) (revoke bob clerk)`

Table 9: The policy as written.

1. It tries to prove coverage by induction, with the block itself as the invariant.
2. The induction step is satisfiable exactly at the faulty rules: at the four approver assignments, and for well-formedness also at the four clerk revocations.
3. A satisfiable step may start from an unreachable configuration, so the checker then searches breadth-first from `start`, expanding actions in declaration order. The first violating configuration it meets yields the least shortest word.

The written policy reaches 436,285,824 configurations, and 397,867,184 of them are ill-formed. The result envelope of the first row:

```
(Result (Status Refuted)
  (Claim (Kind SymbolicPartition) (Partition NoBreach) (Of SoDWatch)
         (Scope (Reach Policy (From start))))
  (Binding (Module "sha256:78379ad6...") (Subject NoBreach) (Machine SoDWatch))
  (Evidence (Witness (ObligationWitness (Obligation Coverage)
              (Word (Do (assign bob auditor) (assign bob approver)))
              (State 34684433))))
  (Assumptions) (Assurance Witness))
```

The witness is replayed by evaluating the module: run from `start` through the module's own `assign` maps, the word ends at 34684433, where `sod-breach` holds.

4.5 The corrected policy

NoBreach is Checked. The block is inductive on its own. The proof consists of 80 closure obligations, one per action, each unsatisfiable — the largest has 59 variables and 62 clauses — together with the check that `start` lies in the block. Each obligation is generated from the elaborated module, and each proof is replayed by drat-trim and `cake_lpr`, so the result carries `(Assumptions)` and `(Assurance Replayed)`. Without the replay it would carry `(Assumptions (TrustedBackend cadical 1.5.3 SAT "sha256:..."))` and the status Conditional.

ClerksOnly is Unknown under `(Budget (Unrollings 6))`. The induction step fails at the four approver activations. The least counterexample to induction is the word 4, in which alice holds approver and nothing else, followed by `(activate alice approver)`. That configuration is unreachable — in the corrected policy nobody holds approver without clerk — and the bounded search finds no reachable violation within six steps, after exploring 26,410 configurations. The property is true, but the block alone cannot prove it, and the result says exactly that:

```
(Result (Status Unknown)
  (Claim (Kind SymbolicPartition) (Partition ClerksOnly) (Of ClerkWatch) ...)
  (Resources (Obligation Coverage)
             (CounterexampleToInduction (Word 4) (Action (activate alice approver)))
             (Search (Unrollings 6) (Configurations 26410))))
```

ClerksOnlyProved is Checked. It supplies the invariant `consistent`, which says that every

active role is assigned, that the role dependencies hold, and that static separation of duty holds. The invariant contains `start` and is closed under all 80 actions (80 proofs, the largest with 104 variables and 269 clauses), and one more proof shows that it implies the block; all 81 proofs are replayed. Strengthening the invariant is the standard way out of a failed induction [11], and the `Invariant` field makes it part of the certificate.

The exact scope. The decision diagrams give the reachable set of the corrected policy: 38,418,640 configurations, which is $79^4 - 27^4$. The formula has a reading. Each user has 79 well-formed configurations, 27 of them without admin, and the reachable set is exactly the well-formed configurations in which someone holds admin. Computing it took 171 diagram nodes, three image rounds and a tenth of a second. The diagrams are built from the guarded updates of the elaborated module, and the reachable sets equal those built from the bundle's independent model. Across the audit 619 unsatisfiable obligations were proved with DRAT proofs, and the bundle replays every one of them through drat-trim and `cake_lpr`.

4.6 Compaction

The whole interface. Whether some part of the configuration can be dropped without changing any access decision is a quotient question.

- `CheckNow AssignmentsOnly` is Refuted with the pair $(0, 32)$: the word 32 has alice's staff session active, so she can read, and the word 0 does not, although both have no assignments.
- `WholeInterface` is Checked, and the quotient is discrete: no two of the 2^{40} words behave alike. The certificate is a set of forty tests, one per bit. Each is a word of at most five actions followed by one query, whose answer equals that bit for every word. For instance, `(deactivate alice auditor) (deactivate alice approver) (activate alice approver)` followed by `approve(alice)` reads alice's approver assignment. Each test is proved by one satisfiability query, generated from the module, whose proof is replayed.
- `LiveInterface` is discrete as well, with 38,418,640 classes, one per reachable configuration.

So the only exact saving is the scope. Storing each user as an index into their 79 well-formed configurations takes seven bits per user, 28 in all, and loses nothing.

A targeted monitor. A monitor that only answers "can bob approve" needs much less.

- The cone of influence of `approve(bob)` has seventeen bits: bob's five assigned bits and his approver, auditor and admin sessions, together with every other user's staff assignment, admin assignment and admin session. The two-person rule is what makes the other users' admin state matter for bob. The cone is the same under both policies. It is computed from the supports of the elaborated update terms, and it is exactly the set of bits that the module's encoding `bob-cone` gathers.
- `BobSlice` is Checked: every update term of a cone bit reads only cone bits, an exact computation on the elaborated terms. The sliced machine has 2^{17} states; 34 of the 80 actions act on it and the others act as the identity. As a second certificate, all 80 two-copy obligations, which say that two words agreeing on the cone have successors agreeing on the cone, are unsatisfiable (the largest has 178 variables and 312 clauses), and their proofs are replayed.
- The vendor's slice `BobOnly`, bob's own ten bits, fails. Its defects are 0 now, 1 for updates and 1 behaviorally. `InduceExact` is Refuted on the update obligation with the pair $(0, 512)$, where 512 is alice's admin session, and the action `(assign bob staff)`: bob's

code becomes 1 in one run and stays 0 in the other. `CheckBehavioral` is Refuted with the pair (131072, 131584) and the word `(revoke bob staff)`: `approve(bob)` is True in one run and False in the other.

- `BobQuotient` has 114,250 classes over the 131,072 cone configurations, after eleven rounds of refinement. `BobLiveQuotient`, on the reachable configurations, has 1160 classes over 1528 reachable cone configurations.

The vendor's slice as an over-approximation. The vendor's ten bits are sound if they are declared for what they are: an over-approximation in which "some other admin is active" is a free choice of the environment, declared as a simulation into a nondeterministic machine (specification, Section 8.2; the `Simulation` library of a later release). For the corrected policy that abstraction reaches 79 configurations of bob, none of them bad, so bob-local safety carries over to the full policy. For the policy as written it reaches 145, and 54 of them breach separation of duty.

4.7 Expected results

Request	Status	Values
written: `NoBreach`, `ClerksOnly`, `WellFormed`	Refuted	the witnesses of Table 9
`NoBreach`	Checked	80 closure proofs, replayed
`ClerksOnly`, `(Unrollings 6)`	Unknown	counterexample to induction: word 4, `(activate alice approver)`; none reachable
`ClerksOnlyProved`	Checked	$80 + 1$ proofs, replayed
`CheckNow AssignmentsOnly`	Refuted	$(0, 32)$, `read(alice)`
`WholeInterface`, `LiveInterface`	Checked	discrete; 2^{40} and 38,418,640 classes; 40 proofs, replayed
`BobSlice`	Checked	2^{17} states, 34 acting actions
`Defects BobOnly`	Checked	0, 1, 1
`InduceExact BobOnly`	Refuted	$(0, 512)$, `(assign bob staff)`
`CheckBehavioral BobOnly`	Refuted	$(131072, 131584)$, `(revoke bob staff)`
`BobQuotient`, `BobLiveQuotient`	Checked	114,250 and 1160 classes
mutation M1 (approver rule reads the auditor session): `NoBreach`	Refuted	`(assign bob auditor) (assign bob approver)`, state 34684433
mutation M2 (revoking clerk keeps approver): `ClerksOnlyProved`	Refuted	`(assign bob approver) (revoke bob clerk) (activate bob approver)`

Table 10: Expected results for Example 2. The results that rest on proofs are Checked when the proofs are replayed, and Conditional on `(TrustedBackend cadical 1.5.3 SAT ...)` otherwise. The two mutations are negative acceptance tests: each edits one rule of `AccessPolicyFixed` back to its form in `AccessPolicy`.

Every request lies in the first release's scope: deterministic dynamics, crisp values, symbolic carriers and bit vectors. The patch adds syntax, fields and one primitive, but no new kind of claim.

4.8 Notes for implementers

Build every obligation from the source. The oracle reads `access_policy.metta` with a small reader, resolves the imports (a name is looked up in the module that declares the map being elaborated), expands the `Families` forms, folds the bit indices, and elaborates every map into `Bool` or `Word` into a typed intermediate form: Boolean terms over the bits of one word, with the declared `From` and `To` types checked, and for each action its normal form "if g then set these bits". The CNF encoder, the evaluator that finds and replays witnesses, and the decision diagrams all read this one form, so a change to the module reaches all three.

Validate the encoding point by point. On the two-user instance of the same source, with the `User` carrier restricted to alice and bob and twenty bits of state, a direct interpreter of the source terms, the intermediate form and the generated CNF agree on every one of the 2^{20} words, for each of the 80 actions of the two modules, 26 queries and alarms, and 240 closure obligations. The CNF side is evaluated by propagating the inputs through the recorded gates, after a structural check that the gate clauses admit exactly one extension of every input assignment, so that whether an obligation is satisfiable from a given word can be read off directly. At forty bits the interpreter and the intermediate form agree on a sample of 50,119 words, which includes every configuration within two steps of `start`, and the closure obligations of `NoBreach` and `ClerksOnlyProved` agree with them on 20,000 of those words.

Mutate the source. Restoring the first mistake in the corrected module, so that the approver rule reads the auditor session again, turns `NoBreach` from Checked into Refuted, with the witness `(assign bob auditor) (assign bob approver)` and the state 34684433 replayed on the mutated source; the closure obligations that change are exactly the four approver assignments. Restoring the second, so that revoking clerk keeps approver, refutes `ClerksOnlyProved` with the three-step witness of the policy as written and leaves `NoBreach` Checked. A separate model of the policy would notice neither mutation, because it does not read the module.

Keep an independent model as a differential check. The bundle's hand-written Python policy and encoder do not read the module. They agree with the source on every word of the two-user instance and on the forty-bit sample, the hand-written closure obligations of the corrected policy are unsatisfiable as well, their decision diagrams give the same reachable sets, and their obligations agree with enumeration: all 240 closure obligations at twenty bits and all 2400 two-copy obligations at ten bits. Under either mutation the comparison fails at the first affected word, for instance at word 524298 under the first, where alice holds clerk and auditor and bob has admin active.

Expand families first. Four action schemas give 80 actions and five observer schemas give 20 observers. The obligation generator only ever sees expanded, constant-folded terms.

Closure obligations are independent and small. None has more than 104 variables, so they can run in parallel. Hash shared structure, and emit byte-identical conjunctive normal form for identical obligations, so that certificates are reproducible.

Report a failed induction step with care. A satisfiable induction step goes in `Resources`, as a counterexample to induction, and never in `Witness`. Refute only with a reachable word found by a search in canonical order.

Minimize solver models before reporting them. Solvers return arbitrary models; for the written policy the first counterexample to induction came back as a thirteen-digit word. Fix bits from the most significant down, which costs at most forty extra calls, to obtain the least model; for `NoBreach` on the policy as written it is the word 10752 with `(assign bob approver)`.

Images for guarded updates. The image formula for guarded set and clear updates holds only when the guard does not read the bits it writes, so check that condition when expanding the terms.

Say which evidence backs each number. Scope sizes come from recomputing the decision diagrams. The satisfiability path proves only that the scope lies inside the invariant, so the result should state which kind of evidence supports each number it reports, and its assurance: `Exact` for the decision diagrams, `Replayed` for replayed proofs, `Witness` for replayed refutations, `TrustedBackend` for a solver answer taken on trust.

Cones of influence. Take the support of each update term and close it under all actions; the result is sound and cheap to compute. Guard bits count as reads, and that is exactly what the vendor's slice missed.

Minimizing an interface on 2^{40} words. Take the cone, enumerate the 2^{17}-state slice, refine it, and lift the blocks back as predicates over the cone bits. Coverage and disjointness of the lifted blocks then hold by construction.

The proof pipeline needs one setting. The default heap and stack of `cake_lpr` (4096 MiB each) could not be allocated on a seven-gigabyte machine; the flags `-CML_HEAP_SIZE=1024 -CML_STACK_SIZE=512` work, and the guide's pipeline command should include them. The proofs are a few hundred bytes of DRAT each, and all 619 were checked in about twenty-five seconds. The solver here is CaDiCaL 1.5.3 through python-sat rather than the pinned binary; since only the obligation generator and `cake_lpr` are trusted, the solver's version appears only in the trusted-backend assumption of a result whose proofs were not replayed.

4.9 Proposed registry additions

- A `Families` library: `(Parameters ($p C) ...)` on `Map`; `(For $v C)` on `Observe` and `Action` entries, with compound labels whose expansion order is the canonical one; `Switch`; and `Exists` and `ForAll` over enumerated carriers.
- The `Machine` fields `(ActionsOf M)` and `(Like M)`.
- On `BlockPartition`: an `(Invariant map)` field, and `(Successor Any B B')` for "every action".
- `(Gather x i ...)` in `BitVectors`.
- The budget unit `Unrollings`, a search depth used as a work limit and never as a semantic horizon.
- Registered `Resources` fields for Unknown results: `CounterexampleToInduction` and `Search`.

5 Example 3: sharing skills between robots with different camera frames

5.1 The problem

Robots that share skills rarely share a body. Their cameras are mounted differently, their grippers differ, and a command that means "forward" for one means "left" for another. A skill learned by one robot can be used by another only through a translation of states and commands, and a library of such translations becomes a network: some are calibrated, some are learned from recorded behavior, and composing them around a loop should bring a skill back unchanged, or at least changed in a way nobody minds. The specification's skill-transfer laboratory states the claims for this setting (specification, Section 13.3); this example works them out on a problem with enough geometry to be instructive.

Three warehouse robots work in identical 5×5 bays. The goal is at the center, cell $(2, 2)$, counting rows from the top and columns from the left, and pillars stand in rows 1 and 3 of the two outer columns. Each robot sees the bay through its own camera and issues the commands *up*, *right*, *down* and *left* in its own image frame. Robot A's camera is aligned with the bay, robot B's is turned a quarter turn clockwise, and robot C's sees the bay transposed. A move earns 1 if it brings the robot closer to the goal, 1/2 if the distance is unchanged (a wall or a pillar blocks the move), and 0 if it moves away. Each robot has 21 free cells.

The shared library has three transfers. AB and BC come from camera calibration. CA was learned by matching recorded responses: the matcher took the first frame change under which every translated command earned the recorded reward.

The findings, in brief:

- A learned transfer is fixed only up to the task's symmetries. The learned CA is one of four candidates that are all exactly right, and going around the cycle A, B, C, A returns every cell as its up-down mirror image.
- Whether that is acceptable depends on the declared gauge, the group of response symmetries each robot is allowed to absorb. With no gauge the library is refuted; with the task's symmetry group it is coherent.
- A gauge copied unchanged from one robot to the next is wrong; it must be transported into each robot's frame. The checker catches the copied gauge on the intertwining condition.
- A skill sent around the loop earns, at each cell, what the original earns at the mirrored cell. A's greedy skill comes back changed at 5 of its 21 cells and earns the same reward at every cell, because its rewards are symmetric under the mirror; the skill that always moves up comes back as the skill that always moves down, and its reward changes at 12 cells.
- A new requirement that breaks a symmetry exposes the stale transfer, with a one-move witness, and re-learning repairs the library up to a smaller gauge.
- A pallet in one bay makes transfers into that bay approximate, and makes the transfer out of it ill-typed. No hive claim covers that case yet.

5.2 Background: transfers, symmetries, holonomy and gauges

A *method system* has states, methods (here, commands) and a response $r(x, m)$, the reward of method m in state x. A *transfer* from system i to system j has two parts: a *state pullback* φ, which maps each state of j to the state of i that it corresponds to, and a *method pushforward* ι,

which maps each method of i to a method of j. The transfer is exact when the target, given the translated command, earns what the source earns at the corresponding state:

$$r_j(y, \iota(m)) = r_i(\varphi(y), m) \quad \text{for every target state } y \text{ and source method } m.$$

Otherwise its defect is the largest difference over all pairs, measured as the total-variation distance between reward laws (`RewardLawTV`, the default) or as the difference of expected rewards (`ExpectedReward`). With these directions a transfer is a morphism of polynomial functors (specification, Section 10.2; [12]), and composition follows that structure: pullbacks compose in the reverse order of the transfers and pushforwards in the same order.

A *symmetry* of a method system is a pair of permutations (g_S, g_M) of states and methods with $r(g_S x, g_M m) = r(x, m)$ for every state and method. The symmetries of a system form a group.

Composing the transfers around a cycle that starts and ends at one member gives a transfer from that member to itself, the *holonomy* of the cycle. If the holonomy is the identity, a skill comes back unchanged. If it is a symmetry (g_S, g_M), a skill π, which chooses a method $\pi(x)$ in each state, comes back as $\pi'(x) = g_M^{-1}\pi(g_S x)$, and the symmetry condition gives

$$r(x, \pi'(x)) = r\big(g_S x, \pi(g_S x)\big) \quad \text{for every state } x:$$

the returned skill earns at x what the original earns at the corresponding state $g_S x$. This is what the `TransportSkill` request certifies, equality at the pulled-back state. It does not give $r(x, \pi'(x)) = r(x, \pi(x))$. Three statements have to be kept apart. Performance at correspondingly transformed states is kept by every transported skill. Performance at the same state is kept by a skill whose reward profile $x \mapsto r(x, \pi(x))$ is invariant under the gauge. Expected performance from a start law that the gauge leaves unchanged is kept by every skill, because such a law gives x and $g_S x$ the same weight. A *gauge* declares, for each member, a group of symmetries that the library is allowed to absorb, and a *transfer hive* is coherent up to its gauge when every edge is exact, every declared gauge element is a symmetry, the gauges agree along the edges, and every holonomy lies in the gauge of its base member (specification, Section 10.4). Agreement along an edge is the *intertwining* condition: conjugating the target's gauge by the edge must give the source's gauge, $\iota^{-1} G_j\, \iota = G_i$ on methods and likewise on states. A gauge is a statement about a frame, so the same symmetry looks different in different frames, and a gauge declared in one robot's frame has to be carried into the others.

Kernel hives are the second kind of hive (specification, Section 10.4). Each member carries a tolerance on its states, here the response tolerance that identifies cells with identical rows of rewards, and the transfers induce cross kernels between members. The hive is coherent when the combined kernel on the disjoint union of the members' states is transitive in the Łukasiewicz sense; the *consensus cost* measures how much closing it under transitivity would identify. A derived kernel hive can be *saturated* by the gauge, which first identifies states related by gauge symmetries. The two kinds of hive make different claims: a transfer hive identifies states, a kernel hive identifies response classes.

In plain terms. A transfer is a dictionary between two robots' points of view. Going around a loop of dictionaries can return a word with a different meaning; the question is whether the difference is one the task cannot see. The gauge is the list of differences each robot has agreed not to care about, and that list has to be written in each robot's own terms. A skill that comes back through such a loop does at each position what the original did at the mirrored position; that is the same as what the original did at the same position only for a skill that looks the same in the mirror.

5.3 The dihedral picture

The eight symmetries of the square — the identity, three rotations, and four mirrors — act on cells and on directions. Write r_{90} for a clockwise quarter turn on the screen, $(r, c) \mapsto (c, 4 - r)$, and m_v, m_h, m_d, m_a for the left-right mirror, the up-down mirror, the transpose and the anti-transpose. The progress task is invariant under four of them,

$$K = \{\mathrm{id},\ r_{180},\ m_v,\ m_h\},$$

because the bay itself has exactly these symmetries: the goal is central and the pillars are placed symmetrically about both axes, but not about the diagonals. K is a subgroup of index two, hence normal, so it is the same set of transformations in every robot's frame. Its two single-mirror subgroups are not frame independent: a quarter turn exchanges the left-right and the up-down mirror.

The candidates for CA. The exact transfers from C to A are exactly the calibrated transfer composed with an element of K, so there are four:

- m_d, the calibrated transfer;
- r_{90}, the learned one, which equals the calibrated pullback composed with A's up-down mirror: $m_d \circ m_h = r_{90}$;
- r_{270} and m_a, the other two.

The holonomy. Composing the transfers A to B to C to A gives the identity with the calibrated CA, and the up-down mirror on both cells and commands with the learned CA.

5.4 The program

The progress-task module declares the group elements as maps on cells and directions, the bay, the three robots as method systems in their own frames, the library of transfers, the symmetries and the hives. The excerpt abbreviates the repeated group elements; Appendix A has the full file.

```
(DC.Module SkillShare
  (Profile Crisp)
  (Use Transfers Hives NatArith)
  (Carrier Coord  (Of Nat) (Enum 0 1 2 3 4))
  (Carrier Grid   (Of (Tuple Nat Nat)) (Product Coord Coord))
  (Carrier Dir    (Of Symbol) (Enum up right down left))
  (Carrier Reward (Of Rational) (Enum 0 (Rat 1 2) 1))
  (Map r90-cell (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple $c (Sub 4 $r)))))
  (Map mh-cell  (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple (Sub 4 $r) $c))))
  (Map md-cell  (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple $c $r))))
  (Map r90-dir  (From Dir) (To Dir)
    (Cases (up right) (right down) (down left) (left up)))
  ;; the other cell and direction maps alike
  (Map pillar (From Grid) (To Bool)
    (Cases ((Tuple $r $c) (And (Or (Eq $r 1) (Eq $r 3)) (Or (Eq $c 0) (Eq $c 4))))))
  (Map step (From (Product Grid Dir)) (To Grid)            ; blocked moves stay put
    (Cases
      ((Tuple (Tuple $r $c) up)
        (If (Eq $r 0) (Tuple $r $c)
          (If (pillar (Tuple (Sub $r 1) $c)) (Tuple $r $c) (Tuple (Sub $r 1) $c))))
      ;; down, left and right alike
      ))
  (Map progress (From (Product Grid Dir)) (To Reward)
```

```
  (Body ($xm) (If (Lt (dist (step $xm)) (dist (GetTuple $xm 0))) 1
                (If (Eq (dist (step $xm)) (dist (GetTuple $xm 0))) (Rat 1 2) 0))))
(Map free-B (From Grid) (To Bool) (Body ($y) (Not (pillar (r270-cell $y)))))
(Carrier FreeB (Of (Tuple Nat Nat)) (Subset Grid free-B))  ; FreeA, FreeC alike
(Map respond-B (From (Product FreeB Dir)) (To Reward)
  (Body ($ym) (progress (Tuple (r270-cell (GetTuple $ym 0))
                               (r270-dir (GetTuple $ym 1))))))
(MethodSystem A (States FreeA) (Methods Dir) (Response respond-A))
(MethodSystem B (States FreeB) (Methods Dir) (Response respond-B))
(MethodSystem C (States FreeC) (Methods Dir) (Response respond-C))
(Transfer AB (Source A) (Target B)
  (StatePullback r270-cell) (MethodPushforward r90-dir))
(Transfer BC (Source B) (Target C)
  (StatePullback mv-cell) (MethodPushforward mv-dir))
(Transfer CA (Source C) (Target A)                          ; learned
  (StatePullback r90-cell) (MethodPushforward r270-dir))
(Transfer CAcal (Source C) (Target A)                       ; calibrated
  (StatePullback md-cell) (MethodPushforward md-dir))
(Transfer NaiveAB (Source A) (Target B) (StatePullback r270-cell)
  (MethodPushforward same-dir))
(Derive CandidatesCA
  (FindTransfers C A                                                ; PROPOSED
    (Candidates (same-cell same-dir) (r90-cell r270-dir) (r180-cell r180-dir)
                (r270-cell r90-dir) (mv-cell mv-dir) (mh-cell mh-dir)
                (md-cell md-dir) (ma-cell ma-dir))))
(Derive RoundTrip (ComposeTransfers (Output ABchecked Transfer)
                    (Output BCchecked Transfer) (Output CAchecked Transfer)))
(Symmetry FlipUD-A (Of A) (States mh-cell) (Methods mh-dir))
(Symmetry FlipLR-A (Of A) (States mv-cell) (Methods mv-dir))     ; and for B and C
(TransferHive Strict (Member A) (Member B) (Member C)
  (Edge AB) (Edge BC) (Edge CA) (Cycle A B C))
(TransferHive FullGauge ... (Gauge A FlipUD-A FlipLR-A)
  (Gauge B FlipUD-B FlipLR-B) (Gauge C FlipUD-C FlipLR-C))
(TransferHive Copied ...
  (Gauge A FlipUD-A) (Gauge B FlipUD-B) (Gauge C FlipUD-C))
(TransferHive Transported ...
  (Gauge A FlipUD-A) (Gauge B FlipLR-B) (Gauge C FlipLR-C))
(TransferHive FromBase    ... (GaugeFrom A FlipUD-A))               ; PROPOSED
(Derive Unsaturated (KernelOf Strict))
(Derive SaturatedFull (KernelOf FullGauge (Saturate Gauge)))
(Map greedy-A (From FreeA) (To Dir) (Table ((Tuple 0 0) right) ...))
(Derive SkillRoundTrip                                          ; PROPOSED
  (TransportSkill (Output RoundTrip Transfer) greedy-A)))
```

Each robot's response is the progress reward read through its own camera: robot B's view of a cell is mapped back to the bay by r_{270}, and its commands likewise. The pillar-free cells of each robot form a `Subset` carrier in that robot's frame, which makes the totality of every state pullback an admission check (Section 5.10). The proposed request `FindTransfers` lists every exact candidate in a declared family, here the eight frame changes, and `TransportSkill` moves a skill table along a checked transfer.

Two further modules complete the example. `SkillShareDock` imports the progress module and replaces the response with *docking*: the charger faces north, so arriving at the goal from any side but the north earns $1/2$. Its library holds `CAold`, the old learned pullback, and `CAnew`, the re-learned one, with docking gauges. `PalletBay` declares C's bay with a pallet at world cell $(0, 2)$, and the transfers `ACp` and `BCp` into that bay.

5.5 Transfers

- AB, BC, CA and CAcal are all exact.
- `FindTransfers C A` returns four exact candidates out of eight: r_{90}, r_{270}, m_d and m_a.
- `NaiveAB` maps states correctly but copies command names. It is Refuted with defect 1. The witness is B's cell $(0, 0)$, where A's *right* is copied as B's *right*: A's command earns 1 and B's earns 1/2, because B's *right* is the bay's *up*, into a pillar.
- `CheckValuePullback NaiveAB` is Checked all the same. Optimal values are 1 everywhere except 0 at the goal, so agreement of optimal values says nothing about whether skills transfer; this is why the specification keeps the value pullback and the transfer certificate apart.
- `RoundTrip` composes to the holonomy: the up-down mirror on both cells and commands, with defect 0.

5.6 The hive under four gauge declarations

Hive	Status	Reason
Strict	Refuted	holonomy: the up-down mirror, which moves $(0, 0)$ to $(4, 0)$
FullGauge	Checked	the group K at every robot
Copied	Refuted	intertwining fails on the edge AB
Transported, FromBase	Checked	the gauge transported into each robot's frame

Table 11: The progress-task library under four gauge declarations.

Why the copied gauge fails. A's up-down mirror is B's left-right mirror, because B's camera is turned a quarter turn. The edge AB therefore maps B's declared up-down mirror onto A's left-right mirror, which is not in A's declared gauge $\{\mathrm{id}, m_h\}$.

The transported gauge. It is the up-down mirror at A and the left-right mirror at B and at C, since transposing also exchanges the two mirrors. Carrying it once more around the cycle returns A's gauge, so the transport is consistent. The proposed field `GaugeFrom` declares the gauge at one member and lets the checker transport it along the edges, including this consistency check.

5.7 Derived kernel hives

Each robot's response tolerance identifies cells with identical rows of rewards: $(0, 0)$, $(2, 0)$ and $(4, 0)$ on one side of the bay and their counterparts on the other. That leaves 17 classes per robot, on 63 nodes in all.

- The unsaturated hive, `Unsaturated`, is incoherent. The least transitivity witness is the triple $((A, (0, 1)), (B, (1, 4)), (C, (1, 0)))$.
- Its holonomy kernel exceeds A's tolerance at 12 pairs, the least being $((0, 1), (4, 1))$.
- The consensus cost of closing it is 4/147.

The transfer-hive witness $(0, 0) \mapsto (4, 0)$ is invisible here, because those two cells have the same responses. This is the difference between the two claims in a concrete case.

Saturated by a gauge, the derived hive is coherent with the full gauge and with the transported gauge, and incoherent with the copied gauge, with the witness $((A, (0, 0)), (B, (0, 0)), (A, (0, 4)))$.

The copied-gauge case is a concrete instance of the specification's statement that the intertwining hypothesis cannot be dropped.

5.8 A skill around the loop

A's greedy skill takes the first improving command in the order *up*, *right*, *down*, *left*. Sent from A to B to C and back to A, it returns changed at 5 of its 21 cells, $(0,1)$, $(1,1)$, $(2,2)$, $(3,1)$ and $(4,1)$, and the transported versions earn the source rewards at B and at C. What `TransportSkill` certifies for the round trip is the identity of Section 5.2: at every cell x of A, the returned skill earns what the greedy skill earns at the mirrored cell $m_h(x)$. The greedy skill earns the same at a cell and at its mirror image, since its reward profile is symmetric under the up-down mirror, so for it the returned skill also earns the same reward at every cell.

That second property is a property of the greedy skill. The skill that always moves up, `always-up` in the module, comes back as the skill that always moves down; `UpRoundTrip` is Checked and reports the skill changed at all 21 cells. At $(3,2)$, just below the goal, the original earns 1 and the returned skill 0; at $(1,2)$, just above it, the reverse; in all, the reward at the same cell changes at 12 of the 21 cells. The certified identity holds all the same: at $(3,2)$ the returned skill earns what `always-up` earns at $(1,2)$. Averaged over the 21 cells with equal weight, a law that the mirror leaves unchanged, both skills earn $1/2$ per move; started from $(3,2)$ alone, they earn 1 and 0. "Coherent up to the gauge" therefore means that the skills that come back are the originals composed with a symmetry of the task: as good as the originals under any start law the symmetry preserves, and cell by cell only when the original's rewards are themselves symmetric.

5.9 Docking breaks a symmetry

The charger faces north, and the docking reward makes the direction of arrival at the goal matter. That shrinks the symmetry group to the left-right mirror in A's frame, and to the up-down mirror in B's and C's frames.

Request	Status	Detail
`CheckTransfer CAold`	Refuted	at A's cell $(1,2)$: A's *down* earns 1, entering from the north, but it is paired with C's *left*, which enters from the south and earns $1/2$
`CAold`, expected reward, within $1/2$	Checked	defect $1/2$
`FindTransfers` again	Checked	two candidates, r_{270} and m_d
`CheckTransfer CAnew` (r_{270})	Checked	holonomy is the left-right mirror, which lies in the docking gauge
`StaleGauge`	Refuted	FlipUD-A is no longer a symmetry: from $(1,2)$, *down* earns 1 and the mirrored move earns $1/2$
`OldLibrary`	Refuted	the edge `CAold`
`Docking`	Checked	coherent with the transported gauge
saturated kernel hive	Checked	coherent
unsaturated kernel hive	Refuted	incoherent; closure cost $4/63$

Table 12: The docking task.

The re-learned CA is again off by a mirror, but the left-right mirror is a symmetry of docking,

so the library is correct up to the new gauge. None of the progress-task certificates carries over, because the response map is part of the interface identity of every transfer and hive certificate.

5.10 A pallet in C's bay

A pallet at world cell $(0, 2)$ blocks one cell of C's bay, which leaves C with 20 states. Transfers into the pallet bay become approximate: under `ACp`, 3 of the 80 state-command pairs differ, the moves into the pallet cell.

Request	Status	Detail
`CheckTransfer ACp`	Refuted	total-variation defect 1; witness: the pallet bay's cell $(1, 0)$, A's *right* becomes the pallet bay's *down*, with rewards 1 and 1/2
`ACp`, expected reward, within 1/2	Checked	defect 1/2
`ACp`, expected reward, within 1/4	Refuted	the same witness
`AtoCp` (A to B to the pallet bay)	Checked	bound $0 + 1/2$, attained; the composite equals `ACp`
a transfer from the pallet bay back to A	InvalidInput	the pullback sends A's cell $(0, 2)$ onto the pallet

Table 13: The pallet bay.

The last row means that the cycle cannot close. Graded transfer hives remain one of the specification's open problems, so no hive claim exists for this library.

5.11 Expected results

Request	Status	Values
`CheckTransfer` AB, BC, CA, CAcal	Checked	defect 0
`CandidatesCA`	Checked	r_{90}, r_{270}, m_d, m_a
`CheckTransfer NaiveAB`	Refuted	B's $(0, 0)$, *right*: 1 against 1/2
`CheckValuePullback NaiveAB`	Checked	
`RoundTrip`	Checked	holonomy (m_h, m_h)
`CheckTransferHive` Strict, FullGauge, Copied, Transported, FromBase	Refuted, Checked, Refuted, Checked, Checked	Table 11
derived kernel hives	as above	12 pairs; cost 4/147
`SkillRoundTrip`	Checked	5 of 21 cells changed; rewards equal at mirrored cells, and here at the same cells
`UpRoundTrip`	Checked	21 of 21 cells changed (always down); same-cell rewards differ at 12 cells
docking requests		Table 12

Request	Status	Values
pallet requests		Table 13

Table 14: Expected results for Example 3.

The `Transfers` and `Hives` libraries come after the first release, so Example 3 is a conformance target for that release.

5.12 Notes for implementers

Group elements. Store each element as a pair of permutations, of states and of commands. Generate a gauge by closure from its declared generators, and check the symmetry condition on the generators only.

Obligation order and witnesses. Check, in this order: edge exactness, edges in declaration order, with the least target state and then the source command; the gauge generators, with the least state and then the command; intertwining, edge by edge, reporting the first target generator whose conjugate leaves the source gauge; and the holonomy, reporting the least moved state.

Holonomy composition order. Follow the polynomial-functor composition order of the specification: around the cycle, pullbacks compose right to left ($AB \circ BC \circ CA$) and pushforwards left to right. The membership test is $(\varphi, \iota^{-1}) \in G_{\text{base}}$, with φ the composite pullback and ι the composite pushforward.

Intertwining for small and large groups. For small groups, compare the conjugated method groups as sets. For large ones, compare generating sets with a membership test such as Schreier–Sims.

Report both holonomy witnesses, typed. The transfer-hive witness $(0, 0) \mapsto (4, 0)$ and the kernel-hive witness $(0, 1) \mapsto (4, 1)$ belong to different claims and must not be confused.

Saturation. Compute the saturated kernel $\alpha \circ c \circ \alpha$ through equivalence classes rather than by the naive four-fold loop. The 63-node closure runs in exact Łukasiewicz arithmetic.

Interface digests. Key every transfer and hive certificate by the digest of its response map, so that the docking module can never reuse a progress certificate.

Pullback totality is an admission check. It belongs to the subset carrier, so the pallet bay's reverse transfer is InvalidInput before any response is compared.

Composing across metrics. `ComposeTransfers` needs a single metric. An exact transfer has defect 0 under every metric, and for rewards in $[0, 1]$ the expected-reward defect is at most the total-variation defect; that conversion should be registered.

Say what a transported skill keeps. `TransportSkill` certifies equality at the pulled-back state. A report that a returned skill earns the same at the same state is a separate claim: the invariance of the skill's reward profile under the holonomy, checked state by state. The greedy skill has it and `always-up` does not.

Learned transfers. Report every exact candidate. Choosing among them consistently around a cycle is gauge fixing; here, choosing m_d for CA makes the holonomy trivial.

5.13 Proposed registry additions

- `FindTransfers`: all exact transfers within a declared family of candidates, with their count and one defect certificate each.
- `TransportSkill`: moves a skill table along a checked transfer, certifies that the transported skill earns at every target state what the source skill earns at the pulled-back state, and counts the states at which the skill changed.
- `(GaugeFrom member symmetry ...)` on `TransferHive`: the gauge is declared at one member and transported by the checker, with the consistency check around each cycle.
- `NatArith`, from Example 1.

Two later additions suggest themselves: a gauge-fixing completion that chooses candidates so that holonomies become trivial, and a graded hive claim with `Within`, which should return Unsupported until the theory exists.

6 Example 4: a market maker across regime shifts

6.1 The problem

A market maker posts a price at which it will buy (the bid) and a price at which it will sell (the ask), and earns the difference when both sides trade. Two forces work against it. Every fill changes its inventory, and inventory is risk: a desk that has bought more than it sold is exposed to the price falling. And some of the traders who take its quotes know more than it does, so that the price tends to move against the desk right after it trades — adverse selection, the subject of the classical models of Glosten and Milgrom [18] and of Kyle [19]. A quoting policy balances spread against both, and the balance depends on the market: how often each side trades, how informed the flow is, whether one side dominates. Markets change regime often, and a policy and its forecasts tuned in one regime are routinely carried into another. After a shift the desk needs to know what changed, whether its policy still holds, what its own data can and cannot reveal, and how to detect shifts without crying wolf. The specification's regime laboratory poses these questions on a two-state example (specification, Section 13.4); the market-making application treats typed regime changes, certified transfer and selective repair at length [21]. This example makes them concrete on a model small enough to solve exactly.

The desk. The desk holds at most two lots either way, so its inventory q lies in $\{-2, \ldots, 2\}$. Each step it posts one of three quotes:

- *tight*: one tick from the mid on both sides;
- *wide*: two ticks on both sides;
- *lean*: one tick on the side that reduces the position and two on the side that grows it; when flat, lean is the same as tight.

At ± 2 lots the side that would grow the position is withdrawn, so at the limits lean is also the same as tight. Each step, a buyer lifts the ask and a seller hits the bid, independently of each other. A fill earns its offset in ticks minus the adverse-selection cost θ of the market, the expected move against the desk after the fill. Holding q' lots after the step costs $|q'|/4$ tick, and the desk maximizes expected discounted profit and loss (P&L) with discount $9/10$.

The markets. In a calm market a one-tick quote fills with probability $1/2$ and a two-tick quote with probability $1/8$, on each side, and fills carry no information. Fill probabilities that fall geometrically with the distance from the mid are the usual simplification in models of limit-order fills [20]. Table 15 lists the four markets of the example.

Market	Ask fills (one, two ticks)	Bid fills (one, two ticks)	θ
calm	1/2, 1/8	1/2, 1/8	0
toxic	1/2, 1/8	1/2, 1/8	1/2
buyers	3/4, 3/16	1/4, 1/16	0
sellers	1/4, 1/16	3/4, 3/16	0

Table 15: The four markets. The toxic market has calm fill rates; the one-sided markets are mirror images of each other.

The toxic market is also given in a finer form: each step independently carries an informed trader with probability $1/2$, whose fills are followed by a one-tick move against the desk ($\theta = 1$), and otherwise only noise traders ($\theta = 0$). On average $\theta = 1/2$.

The findings, in brief:

- In calm markets the desk can run on three risk buckets — flat, one lot, two lots — because the bucket map is an exact quotient of the decision process, and the optimal policy works on it at no loss.
- Toxic flow cuts every value by more than half and changes no decision: the calm policy is certified optimal in the toxic market. A monitor of fill rates is provably blind to this change.
- One-sided buying breaks the compression on both of its obligations, each with a one-line witness, and refutes the calm policy when the desk is one lot long. The broken symmetry survives as an exact transfer between the buyers' and the sellers' markets.
- Knowing which steps carry informed flow is worth about one tick of discounted P&L. A signal that is right nine times in ten, at $1/20$ tick per step, nets about a third of a tick, and evaluating it as if the signal were the truth overstates it by 0.22.
- A desk that only quotes wide cannot tell from its own tape whether news has leaked: the two models are trace equivalent but not bisimilar. A tight quote makes the leak visible, with a two-step witness.
- Each of two monitors is blind to one of the two shifts, and the blindness is an exact claim. Detections on data are Estimated results with guarantees that hold however often the stream is examined.

6.2 Background: decision processes and their quotients

A finite discounted decision process has states, actions, a transition kernel $P(x' \mid x, a)$, a reward law for each state and action, and a discount $\gamma < 1$. A stationary policy π picks one action per

state, and its value solves the linear system

$$V^{\pi}(x) = r(x, \pi(x)) + \gamma \sum_{x'} P(x' \mid x, \pi(x))\, V^{\pi}(x'),$$

where r is the expected reward. With rational data the system can be solved exactly. Policy iteration [22] alternates exact evaluation with improvement — in each state, switch to an action with a larger $Q(x,a) = r(x,a) + \gamma \sum_{x'} P(x' \mid x, a) V^{\pi}(x')$ — and stops at an optimal policy, which is certified by the values together with the table of Q-values: the policy is optimal exactly when no $Q(x,a)$ exceeds $Q(x, \pi(x))$.

The registry's `Decisions` library has one-step decision models only. The example proposes a `DecisionProcess` declaration and gives it a second reading: a decision process *is* a stochastic machine whose observations are the expected rewards of its actions, one query per action. With that reading the core requests apply unchanged. `InduceExact` on a representation of a decision process checks that states with the same code have equal expected rewards for every action and push equal probability onto every code; that is the definition of a stochastic bisimulation of the process [13], or a homomorphism onto the quotient [14], and such a quotient preserves optimal values and optimal policies. `CheckAbstractionMap` between two processes checks a homomorphism, and `MeasureBehavioral` measures the Kantorovich behavioral distance of the reward interface, the stochastic version of the distance of Section 2.2 [15, 16, 17].

Proposition 6.1 (Policies pull back along homomorphisms)**.** *Let h map the states of a decision process M onto those of a process N with the same actions, so that $r_M(x,a) = r_N(hx,a)$ and $\sum_{x' : hx' = y} P_M(x' \mid x, a) = P_N(y \mid hx, a)$ for every x, a and y. Then for every stationary policy π of N, $V_M^{\pi \circ h} = V_N^{\pi} \circ h$; moreover $V_M^{*} = V_N^{*} \circ h$, and $\pi \circ h$ is optimal for M whenever π is optimal for N.*

Proof. Put $W = V_N^{\pi} \circ h$. For every x, with $a = \pi(hx)$,

$$r_M(x,a) + \gamma \sum_{x'} P_M(x' \mid x, a)\, W(x') = r_N(hx,a) + \gamma \sum_{y} P_N(y \mid hx, a)\, V_N^{\pi}(y) = V_N^{\pi}(hx) = W(x),$$

grouping the successors x' by $y = hx'$. So W solves the evaluation equation of $\pi \circ h$ in M, whose solution is unique for $\gamma < 1$. The same computation with a maximum over actions shows that $V_N^{*} \circ h$ solves the Bellman optimality equation of M, so it equals V_M^{*}, and $\pi \circ h$ attains it when π attains V_N^{*}. □

6.3 The program

The model is written with a small proposed algebra of finite rational laws: `(Indep L1 L2)` is the product law, `(Image $v e L)` is the law of the expression `e` when `$v` is drawn from `L`, `(Expect L)` is a mean, and `(LawOf m)` is the law of a declared measure; `Coin` and `Point` remain the elementary laws. Markets are a parameter of the maps, using the `Parameters` and `Switch` forms of Example 2. The core of `MarketDesk`:

```
(DC.Module MarketDesk
  (Profile Stochastic)
  (Use Decisions Laws Families NatArith)          ; Laws, DecisionProcess: PROPOSED
  (Carrier Inv    (Of Int)    (Enum -2 -1 0 1 2))
  (Carrier Quote  (Of Symbol) (Enum tight wide lean))
  (Carrier Market (Of Symbol) (Enum calm toxic buyers sellers informed))
  (Carrier Offset (Of Nat)    (Enum 0 1 2))       ; ticks from the mid; 0: not shown
  (Tolerance Ticks (On Rational) (OneMinusAbsoluteDifference))
  (Map ask-offset (From (Product Inv Quote)) (To Offset)
```

```
  (Cases ((Tuple $q tight) (If (Eq $q -2) 0 1))
         ((Tuple $q wide)  (If (Eq $q -2) 0 2))
         ((Tuple $q lean)  (If (Eq $q -2) 0 (If (Lt $q 0) 2 1)))))
;; bid-offset alike, withdrawn at +2 and wide on the bid when long
(Map ask-fill (Parameters ($m Market)) (From Offset) (To Rational)
  (Cases (0 0)
         (1 (Switch $m (calm (Rat 1 2)) (toxic (Rat 1 2)) (buyers (Rat 3 4))
                       (sellers (Rat 1 4)) (informed (Rat 1 2))))
         (2 (Switch $m (calm (Rat 1 8)) (toxic (Rat 1 8)) (buyers (Rat 3 16))
                       (sellers (Rat 1 16)) (informed (Rat 1 8))))))
(Map theta (Parameters ($m Market)) (From One) (To Rational)
  (Body ($u) (Switch $m (calm 0) (toxic (Rat 1 2)) (buyers 0) (sellers 0)
                        (informed 1))))
(Map edge (Parameters ($m Market)) (From Offset) (To Rational)
  (Body ($o) (Sub $o (theta $m 0))))
(Map fills (Parameters ($m Market)) (From (Product Inv Quote))
  (To (Law (Tuple Bit Bit)))
  (Body ($xa) (Indep (Coin (ask-fill $m (ask-offset $xa)))
                     (Coin (bid-fill $m (bid-offset $xa))))))
(Map step (Parameters ($m Market)) (From (Product Inv Quote)) (To (Law Inv))
  (Body ($xa)
    (Image $f (after (Tuple (GetTuple $xa 0) (GetTuple $f 0) (GetTuple $f 1)))
      (fills $m $xa))))
(Map pnl (Parameters ($m Market)) (From (Product Inv Quote)) (To (Law Rational))
  (Body ($xa)
    (Image $f
      (Sub (Add (Mul (GetTuple $f 0) (edge $m (ask-offset $xa)))
                (Mul (GetTuple $f 1) (edge $m (bid-offset $xa))))
           (Mul (Rat 1 4) (held (Tuple (GetTuple $xa 0)
                                       (GetTuple $f 0) (GetTuple $f 1)))))
      (fills $m $xa))))
(DecisionProcess CalmDesk (States Inv) (Actions Quote)
  (Kernel (step calm)) (RewardLaw (pnl calm))
  (Discount (Rat 9 10)) (Values Ticks))
;; ToxicDesk, BuyersDesk and SellersDesk alike
(Map bucket (From Inv) (To Bucket) (Body ($q) (Max $q (Sub 0 $q))))
(Map mirror (From Inv) (To Inv) (Body ($q) (Sub 0 $q)))
(Representation Risk       (Of CalmDesk)   (Encode bucket))
(Representation RiskBuyers (Of BuyersDesk) (Encode bucket))
(AbstractionMap CalmMirror (Source CalmDesk)   (Target CalmDesk)    (Map mirror))
(AbstractionMap Flip       (Source BuyersDesk) (Target SellersDesk) (Map mirror))
(Map calm-policy (From Inv) (To Quote)
  (Table (-2 tight) (-1 lean) (0 tight) (1 lean) (2 tight))))
```

The fill law is the product of two coins, one per side; the inventory law is its image under $q - A + B$ (`after`), and the P&L law its image under the edges of the fills minus the holding cost (`held` is $|q - A + B|$). Because laws are evaluated on their support, a side that is not shown contributes a coin of probability zero, which never produces the out-of-range inventory ± 3; the admission check for closure of the carrier is a check on supports. The value tolerance `Ticks` compares expected rewards in ticks, capped at one.

The oracle evaluates these maps with a small reference evaluator and compares them, cell by cell, with an independent Python model: kernels and reward laws agree on all 60 state-quote cells of the four markets, and on all 30 cells of the informed-flow process below.

6.4 Calm: three risk buckets are enough

`InduceExact Risk` is Checked. Mirroring the position, $q \mapsto -q$, maps the calm market onto itself (`CheckAbstractionMap CalmMirror` is Checked with kind Quotient), so q and $-q$ have equal expected rewards and push equal probability onto every bucket. The quotient process on buckets has, for one lot:

Quote	Expected reward	To flat	To one lot	To two lots
tight	3/4	1/4	1/2	1/4
wide	1/4	7/64	25/32	7/64
lean	19/32	7/16	1/2	1/16

Table 16: The quotient rows of the one-lot bucket, calm market.

Exact policy iteration converges in two rounds. The optimum quotes tight when flat, leans at one lot and quotes tight at two lots (where tight and lean coincide); ties go to the first action declared. The values are $2513/352 \approx 7.139$ when flat, $217/32 \approx 6.781$ at one lot and $2033/352 \approx 5.776$ at two lots. `OptimizePolicy CalmDesk (Through Risk)` returns the same value as the unrestricted optimum, which Proposition 6.1 predicts: the compression costs nothing.

The table shows why leaning wins at one lot although tight earns more immediately: lean gives up 5/32 of a tick of expected spread to get flat with probability 7/16 instead of 1/4, and to reach the expensive two-lot position with probability 1/16 instead of 1/4. With fill probabilities that fall from 1/2 to 1/8 between one and two ticks, a two-tick quote earns half the expected spread of a one-tick quote, so wide quotes are never chosen in the calm market.

6.5 Two shifts of different types

Toxic flow. The fill laws of the toxic market are those of calm at every cell. Values fall to $2181/704 \approx 3.098$ when flat, $189/64 \approx 2.953$ at one lot and $1541/704 \approx 2.189$ at two, a loss of $2845/704 \approx 4.04$ ticks from flat. `Risk` stays an exact quotient, since the market is still symmetric, and `CheckOptimal ToxicDesk (Policy calm-policy)` is Checked: no action improves on the calm policy's Q-value in any state. The forecasts have to be redone; the quoting rule does not. The margin is limited: at $\theta = 3/4$ instead of 1/2 the optimum changes when flat, to wide, and nowhere else.

One-sided buying. In the buyers' market both obligations of the compression fail.

- `InduceExact RiskBuyers` is Refuted on current adequacy, with the pair $(-2, 2)$ and the query "reward of tight": $-3/16$ against 7/16. Short two lots, only the bid is shown and a seller hits it one time in four; long two lots, a buyer lifts the ask three times in four.
- `CheckUpdates RiskBuyers` is Refuted with the pair $(-2, 2)$, the action tight, the one-lot bucket and the masses 1/4 and 3/4. The total-variation lumpability defect is 1/2.

How far apart. `MeasureBehavioral RiskBuyers (Discount 9/10)` is Checked with the exact interval [56601/73472, 56601/73472], about 0.770, attained at $(-2, 2)$ through tight quotes; the other merged pair, $(-1, 1)$, is at $48015/73472 \approx 0.654$. The immediate parts are only 5/8 and 13/32: most of the distance lies in how the two futures branch. The value is the exact fixed point of the distance equations on the ten pairs of states, and its certificate has two halves.

- One optimal coupling for every pair and action, 30 in all, bounds every term of the equations from above (specification, Section 4.4). For $(-2, 2)$ under tight, the coupling pairs "stay short two" with "unwound to one lot long" (1/2), "stay short two" with "stay long two" (1/4), and "short one" with "long one" (1/4).
- For each of the five pairs whose distance comes from a future step, a Kantorovich potential — a function that changes by at most the distance between any two states — attains the bound. For $(-2, 2)$ it takes the values 1, 48015/73472, 16871/73472 and 0 at -2, -1, 2 and 1; its expected value one tight quote ahead, discounted by 9/10, separates the pair by exactly the distance. The other five pairs are bound by their immediate rewards.

Replay checks the marginals of every coupling, the Lipschitz condition of every potential, and one rational equation or inequality per pair and action. Because the operator is a contraction for $\gamma < 1$, a verified fixed point is the distance.

The policy. `CheckOptimal BuyersDesk (Policy calm-policy)` is Refuted at one lot long: lean has Q-value 489418779/101884448 $\approx$ 4.804 and tight 503574367/101884448 $\approx$ 4.943. Sellers are scarce, so a tight bid earns spread with little risk of getting longer. The resolved optimum quotes tight at one lot long and keeps everything else. From flat it is worth 92970729/21404480 $\approx$ 4.344, against $\approx$ 4.305 for the calm policy; the calm policy's regret is 0.039 from flat and 0.258 from one lot long. Restricted to the buckets (`Through RiskBuyers`) the best policy is the calm one, at $\approx$ 4.305: the buckets cannot hold long apart from short.

The broken symmetry becomes a transfer. The mirror no longer maps the buyers' market to itself, but it maps it exactly onto the sellers' market: `CheckAbstractionMap Flip` is Checked with kind Quotient. `PullbackPolicy Flip (Policy sellers-policy)` is Checked as well. The sellers' optimum, read through the mirror, is exactly the buyers' optimum, with equal values at mirrored positions, as Proposition 6.1 says it must be. A desk that has solved one side of a one-sided market has solved the other. The identity map from calm to buyers, by contrast, is refuted at once, at the state -2 with a tight quote: rewards 1/8 against $-3/16$. The quotes are defined relative to the position, which is why the mirror needs no translation of actions; with quotes skewed in absolute directions it would exchange them.

In plain terms. A calm market cannot tell a long position from a short one, so the desk only needs to know how big its position is. A one-sided market can tell them apart, so the desk needs the sign again — but the market where buyers dominate is the mirror image of the one where sellers dominate, so a policy worked out for one is a policy for the other.

6.6 Toxic flow as a hidden distinction

The toxic market averages two kinds of steps, informed and noise, that the desk cannot tell apart. The informed-flow module puts the flow type into the state and asks what the hidden distinction is worth.

```
(DC.Module InformedFlow
  (Profile Stochastic)
  (Use Decisions Laws Families NatArith)
  (Import MarketDesk (Only Inv Quote Market Bit Ticks step pnl ToxicDesk))
  (Carrier Flow    (Of Symbol) (Enum noise informed))
  (Carrier Book    (Of (Tuple Int Symbol)) (Product Inv Flow))
  (Carrier Reading (Of Symbol) (Enum says-noise says-informed))
  (Measure Half     (On Flow) (Masses (noise (Rat 1 2)) (informed (Rat 1 2))))
  (Measure FlatHalf (On Book)
    (Masses ((Tuple 0 noise) (Rat 1 2)) ((Tuple 0 informed) (Rat 1 2))))
```

```
(Map split-step (From (Product Book Quote)) (To (Law Book))
  (Body ($xa) (Indep (step calm (visible-move $xa))     ; the flow is redrawn
                     (LawOf Half))))
(Map split-pnl (From (Product Book Quote)) (To (Law Rational))
  (Body ($xa) (If (Eq (flow-of (GetTuple $xa 0)) informed)
                  (pnl informed (visible-move $xa))
                  (pnl calm (visible-move $xa)))))
(DecisionProcess Split (States Book) (Actions Quote)
  (Kernel split-step) (RewardLaw split-pnl) (Discount (Rat 9 10)) (Values Ticks))
(Representation Coarse (Of Split) (Encode visible))
(TransitionKernel FlowSignal (From Flow) (To Reading)
  (Row noise    (Mass says-noise (Rat 9 10)) (Mass says-informed (Rat 1 10)))
  (Row informed (Mass says-noise (Rat 1 10)) (Mass says-informed (Rat 9 10))))
(ObservationMethod Signal (Channel FlowSignal) (Cost (Rat 1 20))))
```

The flow is *fresh*: it is redrawn from `Half` at every step, independently of everything else. The desk sees only the inventory, `Coarse`; a toxicity signal, wrong one time in ten and costing $1/20$ tick per step, reads the flow before each quote.

Proposition 6.2 (Averaging a fresh hidden factor)**.** *Let a decision process have states (x, h), and suppose that in every transition the hidden component of the successor is drawn from a fixed law μ, independently of the visible component of the successor and of the current state and action, and that the start law also has $h \sim \mu$ independent of x. Let $\bar{P}(x' \mid x, a) = \sum_h \mu(h) P(x' \mid (x, h), a)$ and $\bar{r}(x, a) = \sum_h \mu(h) r((x, h), a)$. Then for every policy that chooses actions from the visible history:*

1. *the visible states and actions $(x_0, a_0, x_1, a_1, \dots)$ have the same joint law in the process and in the averaged process $(\bar{P}, \bar{r})$;*
2. *given the visible history $(x_0, a_0, \dots, x_t)$ and the action a_t, the conditional mean of the reward at time t is $\bar{r}(x_t, a_t)$ in both processes;*
3. *hence the expected reward at every time, and the expected discounted return, are the same in both.*

In particular the best coarse policy is the optimum of the averaged process, and memoryless coarse policies lose nothing. The law of the full-state expected reward $r((x_t, h_t), a_t)$ is not preserved: it depends on h_t, which the averaged process replaces by its mean.

Proof. By induction on time, the hidden component h_t is independent of the visible history $(x_0, a_0, \dots, x_t)$ and has law μ: at $t = 0$ by the start law, and afterwards because it is redrawn independently at each transition. The action a_t depends only on the visible history, so h_t is independent of that history and a_t together. Given both, the conditional mean of the reward is $\sum_h \mu(h) r((x_t, h), a_t) = \bar{r}(x_t, a_t)$, which is the second statement, and the law of x_{t+1} is $\sum_h \mu(h) P(\cdot \mid (x_t, h), a_t) = \bar{P}(\cdot \mid x_t, a_t)$. By induction the visible process is the averaged chain under the same policy, which is the first statement. The third follows by taking expectations of the second, which depend only on the law of (x_t, a_t), and summing with the discount. The optimum of the averaged process over history-dependent policies is attained by a stationary policy, which is a memoryless coarse policy. Finally, given the visible history the expected reward $r((x_t, h_t), a_t)$ is a function of h_t whose law is the image of μ, while the averaged process has the single value $\bar{r}(x_t, a_t)$; the two laws agree only when $r((x_t, h), a_t)$ is the same for every h in the support of μ. □

In the example the hidden factor is the flow type, with μ the law `Half`. At the state -2 with a tight quote, the expected reward is $1/8$ with noise traders and $-3/8$ with an informed

trader, each with probability 1/2, while the averaged model has the constant −1/8: the means agree and the laws do not. The oracle checks the three statements exactly, for the calm policy and for a policy that quotes wide whenever the inventory has just moved: the joint laws of the visible states and actions through three steps coincide, the conditional mean of the reward given every visible history is the averaged reward, and the expected discounted return of the calm policy from flat is 2181/704 in both processes.

Without freshness the proposition fails: if the hidden factor persists, the visible history carries information about it, and memory or randomization can beat every memoryless coarse policy [23]. The proposed `AverageModel` on processes therefore checks freshness, by enumeration, before it answers.

Averaging. `CheckEqual` applied to `(AverageModel Split (Through Coarse) (Under Half))` and `ToxicDesk`, with `(Compare Means)`, is Checked: the averaged process has exactly the kernels and the expected rewards of the toxic market. With `(Compare Laws)` the check is Refuted: at the state −2 with a tight quote, a P&L of −1/4 has probability 1/4 in the averaged model and 0 in the toxic one. In the toxic description every fill loses exactly half a tick, while in the finer model half the fills lose a full tick and the others nothing. Values and decisions agree, since they depend only on means, but a desk with a variance limit would see the difference.

Averaging is not quotienting. `InduceExact Coarse` is Refuted on current adequacy, with the pair $((-2, \textit{noise}), (-2, \textit{informed}))$ and the query "reward of tight": 1/8 against −3/8. `Coarse` does pass the update obligation, because fills do not depend on the flow, but it is not a quotient. The averaged model is exact for coarse policies by Proposition 6.2, and the quotient does not exist; these are different statements, and the specification keeps them apart (specification, Section 13.4).

Policy	Value from flat	Decimal
sees only the inventory (the toxic optimum)	2181/704	3.098
sees the flow	401605/96926	4.143
uses the signal, net of its cost	165927/48463	3.424
wrong evaluation of the signal policy	176571/48463	3.643

Table 17: What the hidden distinction is worth.

Table 17 reports the requests. `OptimizePolicy Split (Through Coarse)` searches all 243 policies on the inventory, evaluates each in the full process, and finds the calm policy with the toxic optimum, as the proposition says it must. `OptimizePolicy Split` finds that seeing the flow changes the calm policy in a single cell — quote wide when flat and an informed trader is present — and that this is worth 1.045 ticks. `EvaluatePolicy Split (With Signal) (Through Coarse)` finds that the signal supports the same rule applied to its readings; it gains 0.826 before its cost and 0.326 after, so it pays at any price below 0.0826 tick per step. The reduced computation, which works with posteriors given the reading, and a direct evaluation on the hidden states agree exactly.

The last row of the table is the trap the specification's regime laboratory warns about. Evaluating the signal policy as if each reading were the true flow type gives the value of seeing the flow minus the discounted cost of the signal, 3.643; the same policy in fact earns 3.424.

6.7 Did the news leak?

Background. For a stochastic machine, behavioral equivalence and trace equivalence are different claims (specification, Section 4.3 and 7.2). Two states are *trace equivalent* when every sequence of observations has the same probability from both, under every sequence of actions;

equivalently, under every policy that chooses actions from what it has observed. They are *bisimilar* when, in addition, their futures branch alike: a test that reads a predicted probability and reacts to it — a *modal test* — cannot separate them. A consumer that acts only on its own observations can tell apart trace classes and nothing finer. Trace equivalence is decided by linear algebra: the functions $x \mapsto P_x(y_0 a_1 y_1 \cdots a_k y_k)$ span a space whose dimension is the size of the minimal linear realization, the predictive-state dimension of the process [24, 25, 26, 27]. The specification's mandatory seven-state chain has two states that are trace equivalent but not bisimilar, a trace partition of six classes and a minimal linear realization of dimension five.

The model. `NewsLeak` is a tape of nine states. In `leak` an informed trader already knows the direction of an imminent move; in `embargo` nobody knows it yet. The informed trader picks off a tight quote just before the move, and the desk reports `hit-up` or `hit-down`; a wide quote is left alone. An embargoed release moves the price with no informed trade.

```
(TransitionKernel QuoteTight (From Tape) (To Tape)
  (Row leak (Mass knows-up (Rat 1 2)) (Mass knows-down (Rat 1 2)))
  (Row embargo (Mass pending 1))
  (Row knows-up (Mass picked-up 1)) (Row knows-down (Mass picked-down 1))
  (Row pending (Mass moved-up (Rat 1 2)) (Mass moved-down (Rat 1 2)))
  ;; the four outcome states are absorbing
  ...)
;; QuoteWide alike, except knows-up -> moved-up and knows-down -> moved-down
(Machine Desk     (States Tape) (Observe report report)
  (Action tight QuoteTight) (Action wide QuoteWide))
(Machine WideDesk (States Tape) (Observe report report) (Action wide QuoteWide))
(Carrier Seen (Of Symbol) (Reach WideDesk (From leak embargo)))
(Machine WideSeen (States Seen) (Like WideDesk))   ; the desk that only quotes wide
(Representation MergeLeak     (Of Desk)     (Encode merge-leak))
(Representation MergeLeakWide (Of WideSeen) (Encode merge-leak-seen))
```

`WideSeen` is the desk that only ever quotes wide, restricted to the seven states it can reach; the `Reach` carrier makes the restriction part of the declaration.

Results.

- On `WideSeen`, `CheckTrace MergeLeakWide (Policies ObservationBased)` is Checked: merging leak with embargo changes the probability of no sequence of reports, under any policy that acts on the tape. `TracePartition` has six classes, and `MinimalLinear` has dimension 5.
- `CheckBehavioral MergeLeakWide` is Refuted with defect 1/2. The witness is a modal test. Let $\psi(s)$ be the probability that the next report is `up`, and $\varphi(s)$ the expected distance of the next ψ from 1/2. Then $\varphi(\mathit{leak}) = 1/2$ and $\varphi(\mathit{embargo}) = 0$: after a leak, the next state already knows the direction. At discount 9/10 the distance is 81/200; the test, discounted the same way, attains the coupling bound, so the lower and upper certificates meet.
- `WideSeen` is the specification's seven-state chain, state for state, with `leak` and `embargo` in the roles of x and y.
- On `Desk`, which may also quote tight, the merge is refuted for traces too. The canonical witness is the word `(tight tight)` with the event `(quiet quiet up)`, which has probability 0 after a leak and 1/2 after an embargo. The behavioral distance is 1 (81/100 at discount 9/10), `MinimalLinear` has dimension 9, and no two states are trace equivalent.

A desk that always quotes wide cannot learn from its own data whether information leaked: the prior probability of a leak is not identifiable from its tape, and a model that keeps the distinction has a parameter its data cannot fix. Quoting tight is a probe whose price is getting picked off. The interface identity of a trace certificate includes the action set, so the wide-only certificate says nothing about the desk that may also quote tight.

6.8 Monitoring the shift

Background. A monitor watches a stream and should raise an alarm when the stream stops looking like the null law, here the calm market, while keeping false alarms rare however long and however often it looks. Fixed-sample tests do not have that property; anytime-valid procedures do [29, 30]. The simplest is the likelihood-ratio e-process: multiply, observation by observation, the probability of what was seen under a declared alternative divided by its probability under the null. Under the null this product is a nonnegative martingale with initial value one, and Ville's inequality [28] bounds the probability that it *ever* reaches $1/\delta$ by δ. Several monitors run at once combine by a union bound, each with threshold k/δ for k monitors, unless a joint procedure is declared (specification, Section 7.4).

Proposition 6.3 (Monitors are valid under any quoting policy)**.** *Let each observation o_t be drawn, given the past, from the law $N(c_t)$, where the context c_t — for instance the posted quote — is fixed before o_t is drawn, by any rule depending on the past. Let $E_T = \prod_{t \le T} A(c_t)(o_t)/N(c_t)(o_t)$ for a family of alternative laws $A(c)$. Then E_T is a nonnegative supermartingale with $E_0 = 1$, a martingale when each $A(c)$ is absolutely continuous with respect to $N(c)$, and $\Pr\big(\sup_T E_T \ge 1/\delta\big) \le \delta$.*

Proof. Given the past and c_{T+1}, the expected value of the next factor is

$$\sum_{o:\, N(c)(o)>0} N(c)(o)\, \frac{A(c)(o)}{N(c)(o)},$$

the mass of $A(c)$ on the support of $N(c)$, which is at most one and equal to one under absolute continuity. Ville's maximal inequality for nonnegative supermartingales gives the bound. □

The proposition explains the role of the context: each factor must use the law given the quote that was standing when the outcome occurred. A monitor that used, say, the average fill law of the desk's usual quotes would not be a martingale under an adaptive quoting policy.

The monitors. `ShiftMonitors` declares two pairs of monitors, on the markouts after fills and on the fills of each posted quote, against the toxic and the buyers' alternatives, and three stored streams of 4000 steps, generated once by simulation and pinned by digest.

```
(Map markout (Parameters ($m Market)) (From One) (To (Law Markout))
  (Body ($u) (Image $b (If (Eq $b 1) adverse favorable)
               (Coin (Div (Add 1 (theta $m $u)) 2)))))
(Monitor Markouts (Context One)
  (Null (markout calm)) (Alternative (markout toxic))
  (Columns (observation (each "markouts"))))
(Monitor Fills (Context Posted)
  (Null (fills calm)) (Alternative (fills buyers))
  (Columns (context "q" "quote") (observation "ask" "bid")))
;; FillsToxic and MarkoutsBuyers alike
(Stream ToxicTape (Source "streams/toxic_seed1.json")
  (Digest "sha256:ddbb6048...")                  ; the full digest in Appendix A
```

```
  (Provenance "simulated: informed flow on half the steps, calm policy"))
!(dc.run ShiftMonitors
   (Detect Markouts Fills (On ToxicTape) (Level (Rat 1 100))))
```

After a fill the mid moves one tick against the desk with probability $(1 + \theta)/2$: one half in the calm market, three quarters in the toxic one.

Exact claims. `CheckMonitor` is Checked for all four monitors.

- Every factor has null expectation exactly one, in every context.
- `Markouts`: the factors are 3/2 after an adverse move and 1/2 after a favorable one; chi-square 1/4 per markout.
- `Fills`: when flat with a tight quote, the factors are 9/4 (ask filled only), 1/4 (bid only) and 3/4 (both or neither); chi-square 9/16.
- `FillsToxic` is Blind: the fill laws are equal in all 15 contexts, so no statistic of fill rates can ever detect toxicity.
- `MarkoutsBuyers` is Blind: markouts are fair in the buyers' market.

Monitor	Toxic shift	Buyers' shift
markouts	KL ≈ 0.131 nats per markout	Blind (Checked)
fills	Blind (Checked)	KL ≈ 0.262 nats per step when flat and tight

Table 18: Which monitor can see which shift. Blindness is exact; the rates are decimals of irrational quantities and belong to no Checked claim.

Estimated claims. `Detect Markouts Fills` at level 1/100 combines the two monitors by the union bound, so each fires at 200.

- On the toxic stream, `Markouts` fires at step 40, after 35 markouts of which 27 were adverse, with e-value $3^{27}/2^{35} \approx 221.9$; `Fills` never fires.
- On the buyers' stream, `Fills` fires at step 39, with e-value about 252.7; `Markouts` never fires.
- On the calm stream neither fires in 4000 steps; the largest e-values reached are 1.60 and 2.25.

The result record of the first detection:

```
(Result (Status Estimated)
  (Claim (Detect (Markouts Fills) (On ToxicTape)))
  (Value (Fired Markouts) (Step 40) (Observations 35)
         (EValue (Rat 7625597484987 34359738368)) (Threshold 200))
  (Evidence (Stream ToxicTape "sha256:ddbb6048...")
            (Combine Union) (Level (Rat 1 100)))
  (Assumptions "under the null, markouts are fair and independent given the fills,
                and fills follow the calm law given the posted quote")
  (Method LikelihoodRatioEProcess Ville))
```

The claim is about the data source, never about the model: exact arithmetic on a sample does not turn a conclusion about a market into a theorem (specification, Section 7.4). Which monitor

fires types the change, markouts for toxicity and fills for one-sided flow. At about 0.131 nats per markout, Wald's approximation [31] puts the expected number of markouts needed to reach 200 at about 40; that figure is an approximation and belongs to no Checked claim.

6.9 Expected results

Request	Status	Values
`InduceExact Risk`	Checked	three-bucket quotient
`CheckAbstractionMap CalmMirror`	Checked	kind Quotient
`OptimizePolicy CalmDesk (Under Flat)`	Checked	2513/352
`(Through Risk)`	Checked	the same value
`InduceExact RiskToxic`	Checked	
`CheckOptimal ToxicDesk (Policy calm-policy)`	Checked	
`EvaluatePolicy ToxicDesk (Policy calm-policy)`	Checked	2181/704
`InduceExact RiskBuyers`	Refuted	$(-2, 2)$, reward of tight: $-3/16$, $7/16$
`CheckUpdates RiskBuyers`	Refuted	$(-2, 2)$, tight, one lot: $1/4$, $3/4$
`MeasureBehavioral RiskBuyers (Discount 9/10)`	Checked	56601/73472
`CheckOptimal BuyersDesk (Policy calm-policy)`	Refuted	state 1: lean 4.804, tight 4.943
`OptimizePolicy BuyersDesk (Under Flat)`	Checked	92970729/21404480
`(Through RiskBuyers)`	Checked	438585327/101884448
`CheckAbstractionMap Flip`	Checked	kind Quotient
`PullbackPolicy Flip (Policy sellers-policy)`	Checked	the buyers' optimum
`CheckEqual (AverageModel Split ...) ToxicDesk (Compare Means)`	Checked	
with `Compare Laws`	Refuted	$(-2, \text{tight})$, P&L $-1/4$: $1/4$, 0
`InduceExact Coarse`	Refuted	reward of tight: $1/8$, $-3/8$
`OptimizePolicy Split (Through Coarse) (Under FlatHalf)`	Checked	2181/704
`OptimizePolicy Split (Under FlatHalf)`	Checked	401605/96926
`EvaluatePolicy Split (With Signal) (Through Coarse)`	Checked	165927/48463
`CheckTrace MergeLeakWide (Policies ObservationBased)`	Checked	
`CheckBehavioral MergeLeakWide`	Refuted	$1/2$, modal test
`MeasureBehavioral MergeLeakWide (Discount 9/10)`	Checked	81/200
`TracePartition WideSeen`, `MinimalLinear WideSeen`	Checked	six classes; dimension 5
`CheckTrace MergeLeak`	Refuted	`(tight tight)`, `(quiet quiet up)`: 0, 1/2

Request	Status	Values
`CheckBehavioral MergeLeak`	Refuted	1
`MinimalLinear Desk`	Checked	dimension 9
`CheckMonitor`, four monitors	Checked	valid; two Blind
`Detect` on the toxic, buyers' and calm streams	Estimated	Markouts at 40; Fills at 39; none

Table 19: Expected results for Example 4.

Every request is outside the first release: the stochastic profile, the `Decisions` and `Linear` libraries, and the proposed `Monitors` library come later, so Example 4 is a conformance target for the release that adds stochastic dynamics.

6.10 Notes for implementers

Laws are exact finite maps from canonical values to rationals, with zero masses dropped when they are built. `Image` adds the masses of equal values and `Indep` multiplies supports. Carrier closure is checked on supports: when the desk is two lots long the bid's law is `Coin 0`, so the out-of-range inventory 3 is never produced.

Decision processes reuse the machine checks. The machine view's queries are the expected rewards, one per action, with the declared value tolerance. Quotient checks are then MDP homomorphism checks with no new algorithm. Equality of reward laws is a separate and stronger claim (`Compare Laws`).

Policy iteration. Evaluate exactly by Gaussian elimination over the rationals. Improve by keeping the current action on ties and otherwise taking the first best action in declaration order, which makes the reported policy canonical. The certificate is the vector of values and the table of Q-values; replay checks $|S|$ linear equations and $|S||A|$ inequalities. The canonical witness of `CheckOptimal` is the least state, the policy's action there, the first action in declaration order with a larger Q-value, and both Q-values.

Policies through a representation. `OptimizePolicy` enumerates the deterministic code policies (27 for the buckets, 243 for `Coarse`) and evaluates each in the full process. For larger families it should report the best policy found and leave optimality Unknown unless the search was exhaustive. Memoryless code policies are all the coarse policies there are only when the hidden part is fresh; in general the problem is partially observable, memory and randomization can do better, and the result must name its policy class.

Kantorovich distances in three stages. A floating-point value iteration, with one small linear program per pair and action, identifies for each pair the term that sets the distance and the optimal couplings. An exact linear solve gives the candidate distances, and the couplings are re-optimized exactly until they stop changing. Verification then solves an exact transport problem for every pair and action (upper bounds) and an exact dual problem for every pair whose distance comes from a future step (lower bounds). For a discount below one the verified fixed point is the distance. Supports here have at most three points, so a rational simplex with Bland's rule [32] is enough.

The behavioral distance does not bound values. The specification's operator combines the present observation and the future with a maximum. It certifies equal values at distance zero, but no constant c gives $|V^*(x) - V^*(y)| \le c\, d_{\mathrm{beh}}(x, y)$ in general. Consider two chains whose reward gaps are $\delta, \delta/\gamma, \delta/\gamma^2, \ldots$ for $K+1$ steps, with $\delta/\gamma^K \le 1$, and equal afterwards: the distance stays at δ, while the value gap is $(K+1)\delta$, and K can be as large as $\log(1/\delta)/\log(1/\gamma)$. A desk that needs a guarantee on value loss should check values directly, as this example does, or use an additive metric, $d = \max_a(|\Delta r_a| + \gamma W_d)$, which bounds value differences [17]. The registry does not have one yet.

Sensors on processes. Compute values on the hidden states, with a fresh reading drawn each step. When the hidden part is fresh, optimize over policies on the code and the reading using the posteriors given each reading. Never substitute the reading for the hidden state.

Trace claims. Compute trace spaces by Krylov iteration over the operators $M_{a,y} = M_a \operatorname{diag}([o = y])$ with exact rationals. The canonical trace-event witness is the shortest word, words ordered by action declaration, and then the least sequence of observations in the declaration order of the reports. A trace certificate is keyed by its interface, action set included.

Monitors. Fix the context before the observation. Validity, blindness and chi-square are exact results; the KL rate is irrational and stays outside every Checked claim. `Union` gives each of k monitors the threshold k divided by the level, and `Mixture` thresholds the average of the e-values. Store the stream rather than the seed: random generators differ between implementations, and the module pins the stream by its digest. The e-value at a crossing is an exact rational. A null with unknown parameters, such as a calm market with estimated fill rates, needs a different procedure, for example a mixture e-process, which the proposed library does not cover.

Interface identity. Certificates are keyed by the digest of the process — kernel, reward law, discount and value tolerance. The calm policy is optimal in `CalmDesk` and in `ToxicDesk` by two separate certificates.

6.11 Proposed registry additions

- A `Laws` library: `Indep`, `Image` (which binds a variable), `Expect` and `LawOf`.
- A `DecisionProcess` declaration in `Decisions`, which also counts as a `Machine`, with the fields `States`, `Actions`, `Kernel`, `RewardLaw`, `Discount` and `Values`.
- Extensions of four `Decisions` requests: `AverageModel` on processes, with `Through` and the freshness precondition; `CheckEqual` on processes, with `(Compare Laws|Means)`; `OptimizePolicy` on processes, including policies on codes; and `EvaluatePolicy` with either `(Policy p)` or `(With method)`, and with `Through`.
- New `Decisions` requests: `CheckOptimal`, with an `OptimalityVerdict` result, and `PullbackPolicy`, with a `PulledBackPolicy` result.
- A `Monitors` library: the declarations `Monitor` and `Stream`, and the requests `CheckMonitor` (Checked) and `Detect` (Estimated, with a `Detection` result).
- Reused from earlier examples: `NatArith` from Example 1, and `Families` with the `Like` field from Example 2.

Later additions suggested by the example: an additive behavioral metric with value-loss guarantees; declared policy classes, such as finite-memory controllers, for processes with persistent hidden state; monitors for nulls with unknown parameters and confidence sequences for fill rates; and risk-sensitive objectives, such as the variance of discounted P&L, for which `Compare Laws` becomes the relevant check.

7 What the examples ask of the language

7.1 Registry additions across the examples

The four examples propose additions of six kinds. Table 20 collects them; Appendix B lists every entry of the four patches. Most additions are syntax or primitives that let a program say what it already means. A smaller number extend what can be claimed, and those deserve the closest review, because each one needs an algorithm, a certificate format and a status rule before any implementation can support it.

Kind	Addition	Proposed in	Also used in
Elaboration	`Families`: `Parameters`, `For`, `Switch`, `Exists`, `ForAll`	2	4
	`Machine` fields `ActionsOf`, `Like`	2	4 (`Like`)
	`EncodingFamily` with `IntervalProduct`, `RoundedGrid`	1	
	`TransferHive` field `GaugeFrom`	3	
Primitives	`NatArith`: `Le`, `Lt`, `Min`, `Max`, `Quot`, `Rem`	1	3, 4
	`BitOr`, `BitAnd`, `BitXor`, `PopCount`; `Gather`	1; 2	
	tolerance body `Meet`	1	
	`Laws`: `Indep`, `Image`, `Expect`, `LawOf`	4	
Declarations	`ParallelMachine`, `ProductRepresentation` (`Parallel` library)	1	
	`DecisionProcess`, declaring a `Machine`	4	
	`Monitor`, `Stream` (`Monitors` library)	4	
Arguments and fields	`Refine (Until (Steps k))`	1	
	`WeakestWithin`: `Objective`, `Family`, `Kind Radius`; result fields	1	
	`BlockPartition`: `Invariant`, `Successor Any`	2	
	budget unit `Unrollings`; `Resources` fields for Unknown	2	
	`Decisions` requests on processes; `CheckEqual (Compare ...)`	4	
New requests	`FindTransfers`, `TransportSkill`	3	
	`CheckOptimal`, `PullbackPolicy`	4	
	`CheckMonitor`, `Detect`	4	
Status rules	optimality of a frontier Checked by a replayed clique or refutation certificate	1	

Kind	Addition	Proposed in	Also used in

Table 20: Proposed additions, by kind.

The additions that extend what can be claimed are few. `WeakestWithin` with a codes objective and a lower-bound certificate turns frontier optimality from an exhaustive-search claim into a certified one. `FindTransfers` turns "the learned transfer is exact" into "these are all the exact transfers in the family", which is what makes gauge fixing possible. The decision-process requests, `CheckOptimal` and `PullbackPolicy`, bring policies into the claim language, with certificates that are vectors of values and tables of Q-values. `Detect` is the first request whose only status is Estimated, and its result names the data source by digest. Each of these comes with an exact algorithm on the scale of the examples and a replayable certificate, which is the standard every registered claim must meet.

7.2 Lessons for implementers

Some lessons recur across the examples, often in different forms.

Claims that look alike are different claims. Example 1 separates a summary, a cold store and a memory that runs on its codes, with three different answers for the same encoding, and a compaction that is exact for the operations a delivery protocol allows from one that is exact for every operation. Example 2 separates a failed induction step from a refutation. Example 3 separates a value pullback from a transfer, a transfer hive from a kernel hive, with different witnesses for the same cycle, and what a returned skill earns at the mirrored state from what it earns at the same state. Example 4 separates averaging from quotienting, equal means from equal laws, and trace equivalence from bisimulation. In every case the cheaper claim is true and the stronger one false, or the reverse, on the same input. An implementation that lets one request stand in for another produces confident wrong answers, and the registry's job is to keep the requests and their result types distinct.

Every certificate has two halves. An upper bound and a lower bound, a coloring and a clique or a refutation, couplings and potentials, values and a table of Q-values, a proof of closure and a reachable word: each Checked or Refuted result in the examples rests on a pair of objects that a second program can check, with arithmetic or, for a refutation or a proof of closure, with a verified proof checker. Producing them is the hard part; replaying them is cheap, and it is what lets the trusted base stay small. A result whose second half has not been replayed is Conditional, and its assumptions name the solver that was trusted.

Canonical witnesses take deliberate work. They come from breadth-first searches that expand actions in declaration order, from minimizing solver models before reporting them, and from fixed orders on pairs, classes and observation sequences. None of this is automatic, and every implementation that wants to pass a manifest has to do it the same way.

Unknown is an answer. A budget that ends, a property whose proof needs a stronger invariant, a greedy search without a lower bound: each of these returns Unknown with what was learned in `Resources`. Example 2's counterexample to induction is the clearest case, and its result is more useful to the auditor than a false refutation or a silent failure would be.

Interface identity keys every certificate. The docking response of Example 3, each market of Example 4, and the action set of the news-leak desk all change the interface, and certificates for one interface must never be consulted for another. Digests of the response maps, the kernels and the action sets are the mechanism. Example 2 adds the source itself: its obligations are generated from the module, each result records the module's digest, and a mutation of one rule

changes exactly the obligations of that rule.

Admission does real work. A non-total state pullback (Example 3), a hidden factor that is not fresh (Example 4), and a guard that reads the bits it writes (Example 2) are all caught before any claim is evaluated. Checks of this kind are cheap and turn a wrong answer into an InvalidInput.

Avoid enumeration by structure, not by luck. The product rule (Example 1), cones of influence and symbolic proofs (Example 2), and freshness (Example 4) each replace an enumeration that would be impossible or expensive by an argument with a certificate. A request whose structure falls outside such a rule should return Unknown with a diagnostic rather than attempt the enumeration.

Data enters frozen. The streams of Example 4 are stored with digests, and the Estimated results name them. Exact claims are about models and Estimated claims about data, and the result envelope keeps the two apart.

7.3 Open questions the examples raise

- A frontier for memories that run on their codes, where each candidate costs the fixed point of an induced machine (Example 1).
- Joint encodings outside the product rule, such as codebooks shared across beliefs (Example 1).
- Graded transfer hives, for libraries whose transfers are only approximately exact, and a gauge-fixing completion that chooses among exact candidates so that holonomies become trivial (Example 3).
- An additive behavioral metric for decision processes, whose distances bound differences of optimal value (Example 4).
- Declared policy classes for processes whose hidden state persists, where memory and randomization matter (Example 4).
- Monitors for nulls with unknown parameters, and confidence sequences for estimated rates (Example 4).
- Risk-sensitive objectives, for which equality of reward laws, and not only of means, becomes the relevant check (Example 4).
- Declarations of delivery protocols for memories that forget identities, so that the operations a compaction is certified against are declared once instead of being written out as a restricted machine (Example 1).
- A verified obligation generator, which would turn the translation validation of Example 2 into a proof that the generated formulas express the module's obligations (Example 2).

8 Conclusion

The four examples began as tests of a specification and turned into something closer to design reviews. In each of them the most useful output was not a number but a distinction the designer had been collapsing without noticing — between a summary of a memory and a cold store for it, between a property that is false and one whose proof needs more than the property itself, between a skill library that is wrong and one that is right only up to a mirror, between a market whose averages have changed and one in which the things being averaged have changed. Keeping such distinctions apart, in a form that a second program can check, is a modest job for

a language. It is also a job that a great deal of practical reasoning in AI systems currently does without, and the examples suggest that doing it explicitly costs less than one might fear: every claim here was settled exactly or by a certificate that a verified checker replays, most of them in seconds.

The examples are small on purpose, and they show nothing yet about deployed memories, policies, robots or markets. Each would have to be scaled up, connected to real data through frozen snapshots and streams, and run against a running system before it said anything about one. What they do show is how MeTTaDinC programs are written and read, what their answers look like, and where the language runs out. The registry additions they propose are the first concrete evidence about which parts are missing, and the list of open questions is the first concrete evidence about which parts are hard.

The hope, in the end, is that the next person to write a MeTTaDinC program starts from one of these four rather than from the fixtures, and finds that the language says what they need it to say — or, failing that, finds out quickly and precisely what it cannot say yet.

References

[1] *MeTTaDinC: An Executable Language for Observer-Relative Abstraction, Evidence, and Composition.* Specification draft, 4 November 2026.

[2] *Implementing MeTTaDinC in Cetta: A Coding-Agent Specification and Work Plan.* Companion to the MeTTaDinC specification draft, 4 November 2026.

[3] B. Goertzel, M. Iklé, I. F. Goertzel and A. Heljakka. *Probabilistic Logic Networks: A Comprehensive Framework for Uncertain Inference.* Springer, 2008.

[4] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman. Role-based access control models. *IEEE Computer* 29(2):38–47, 1996.

[5] R. Sandhu, V. Bhamidipati and Q. Munawer. The ARBAC97 model for role-based administration of roles. *ACM Transactions on Information and System Security* 2(1):105–135, 1999.

[6] S. D. Stoller, P. Yang, C. R. Ramakrishnan and M. I. Gofman. Efficient policy analysis for administrative role based access control. In *Proceedings of the 14th ACM Conference on Computer and Communications Security*, 2007.

[7] G. S. Tseitin. On the complexity of derivation in propositional calculus. In *Studies in Constructive Mathematics and Mathematical Logic, Part II*, 1968.

[8] R. E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers* C-35(8):677–691, 1986.

[9] N. Wetzler, M. J. H. Heule and W. A. Hunt Jr. DRAT-trim: efficient checking and trimming using expressive clausal proofs. In *SAT 2014*.

[10] Y. K. Tan, M. J. H. Heule and M. O. Myreen. cake_lpr: verified propagation redundancy checking in CakeML. In *TACAS 2021*.

[11] A. R. Bradley. SAT-based model checking without unrolling. In *VMCAI 2011*.

[12] N. Niu and D. I. Spivak. *Polynomial Functors: A Mathematical Theory of Interaction.* arXiv:2312.00990, 2023.

[13] R. Givan, T. Dean and M. Greig. Equivalence notions and model minimization in Markov decision processes. *Artificial Intelligence* 147(1–2):163–223, 2003.

[14] B. Ravindran. *An Algebraic Approach to Abstraction in Reinforcement Learning.* PhD thesis, University of Massachusetts Amherst, 2004.

[15] J. Desharnais, V. Gupta, R. Jagadeesan and P. Panangaden. Metrics for labelled Markov processes. *Theoretical Computer Science* 318(3):323–354, 2004.

[16] F. van Breugel and J. Worrell. An algorithm for quantitative verification of probabilistic transition systems. In *CONCUR 2001*.

[17] N. Ferns, P. Panangaden and D. Precup. Metrics for finite Markov decision processes. In *UAI 2004*, 162–169.

[18] L. R. Glosten and P. R. Milgrom. Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics* 14(1):71–100, 1985.

[19] A. S. Kyle. Continuous auctions and insider trading. *Econometrica* 53(6):1315–1335, 1985.

[20] M. Avellaneda and S. Stoikov. High-frequency trading in a limit order book. *Quantitative Finance* 8(3):217–224, 2008.

[21] B. Goertzel. *Market Making Across Regime Shifts: Typed changes, certified transfer, and selective repair in distinction calculus.* Manuscript, September 2026.

[22] R. A. Howard. *Dynamic Programming and Markov Processes.* MIT Press, 1960.

[23] S. P. Singh, T. Jaakkola and M. I. Jordan. Learning without state-estimation in partially observable Markovian decision processes. In *ICML 1994*.

[24] M.-P. Schützenberger. On the definition of a family of automata. *Information and Control* 4:245–270, 1961.

[25] W.-G. Tzeng. A polynomial-time algorithm for the equivalence of probabilistic automata. *SIAM Journal on Computing* 21(2):216–227, 1992.

[26] M. L. Littman, R. S. Sutton and S. Singh. Predictive representations of state. In *NIPS 2001*.

[27] H. Jaeger. Observable operator models for discrete stochastic time series. *Neural Computation* 12(6):1371–1398, 2000.

[28] J. Ville. *Étude critique de la notion de collectif.* Gauthier-Villars, 1939.

[29] A. Ramdas, P. Grünwald, V. Vovk and G. Shafer. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science* 38(4):576–601, 2023.

[30] S. R. Howard, A. Ramdas, J. McAuliffe and J. Sekhon. Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics* 49(2):1055–1080, 2021.

[31] A. Wald. Sequential tests of statistical hypotheses. *Annals of Mathematical Statistics* 16(2):117–186, 1945.

[32] R. G. Bland. New finite pivoting rules for the simplex method. *Mathematics of Operations Research* 2(2):103–107, 1977.

A Module sources

The four module files follow in full, exactly as they are shipped in the bundles. Lines that do not fit are wrapped, with the continuation marked by an arrow. Comments marked `PROPOSED` flag the forms that need a registry patch.

A.1 Example 1: `belief_memory.metta`

Four modules: `BeliefMemory`, `GentleMemory`, `AgentMemory` and `ReceiptLedger`, with their requests.

```
;; Example 1: compacting an agent's belief memory.
;; Four modules: the memory as first designed (floor-halving decay), the memory with
;; the repaired decay rule, six beliefs in parallel, and an evidence-receipt ledger
;; with forks and merges.  Forms marked PROPOSED are not in the current registry;
;; the example registry patch (ex1_registry_patch.json) adds them.

;; @excerpt belief-memory
(DC.Module BeliefMemory
  (Profile Graded)
  (Use NatArith)                                  ; PROPOSED library: Le, Min, Quot
  (Carrier Count  (Of Nat) (Enum 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15))
  (Carrier Belief (Of (Tuple Nat Nat)) (Product Count Count))
  (Carrier Status (Of Symbol) (Enum Believed Disbelieved Uncertain))

  ;; what the memory reports
  (Map strength (From Belief) (To Rational01)
    (Cases ((Tuple $p $n) (If (Eq (Add $p $n) 0) (Rat 1 2) (Div $p (Add $p $n))))))
  (Map confidence (From Belief) (To Rational01)
    (Cases ((Tuple $p $n) (Div (Add $p $n) (Add (Add $p $n) 4)))))
  (Map status (From Belief) (To Status)
    (Cases ((Tuple $p $n)
      (If (And (Le 4 (Add $p $n)) (Le (Mul 2 $n) $p)) Believed
        (If (And (Le 4 (Add $p $n)) (Le (Mul 2 $p) $n)) Disbelieved Uncertain)))))

  ;; how it changes: evidence saturates at 15, decay halves and rounds down
  (Map add-pos (From Belief) (To Belief) (Cases ((Tuple $p $n) (Tuple (Min (Add $p 1) 15) $n))))
  (Map add-neg (From Belief) (To Belief) (Cases ((Tuple $p $n) (Tuple $p (Min (Add $n 1) 15)))))
  (Map halve   (From Belief) (To Belief) (Cases ((Tuple $p $n) (Tuple (Quot $p 2) (Quot $n 2)))))

  (Tolerance Dist01 (On Rational01) (OneMinusAbsoluteDifference))
  (Machine Decide (States Belief)                 ; the action-selection interface
    (Observe status status)
    (Action pos add-pos) (Action neg add-neg) (Action decay halve))
  (Machine Report (States Belief)                 ; the reasoning interface
    (Observe strength strength (Values Dist01))
    (Observe confidence confidence (Values Dist01))
    (Action pos add-pos) (Action neg add-neg) (Action decay halve))

  ;; candidate encodings: grid indices of the rounded truth value, count buckets, caps
  (Map tv-grid-20 (From Belief) (To (Tuple Nat Nat))
    (Cases ((Tuple $p $n)
      (Tuple (If (Eq (Add $p $n) 0) 10 (Quot (Add (Mul 40 $p) (Add $p $n)) (Mul 2 (Add $p $n))))
             (Quot (Add (Mul 41 (Add $p $n)) 4) (Add (Mul 2 (Add $p $n)) 8))))))
  (Map tv-grid-10 (From Belief) (To (Tuple Nat Nat))
    (Cases ((Tuple $p $n)
      (Tuple (If (Eq (Add $p $n) 0) 5 (Quot (Add (Mul 20 $p) (Add $p $n)) (Mul 2 (Add $p $n))))
             (Quot (Add (Mul 21 (Add $p $n)) 4) (Add (Mul 2 (Add $p $n)) 8))))))
  (Map tv-grid-10-7 (From Belief) (To (Tuple Nat Nat))
    (Cases ((Tuple $p $n)
      (Tuple (If (Eq (Add $p $n) 0) 5 (Quot (Add (Mul 20 $p) (Add $p $n)) (Mul 2 (Add $p $n))))
             (Quot (Add (Mul 15 (Add $p $n)) 4) (Add (Mul 2 (Add $p $n)) 8))))))
  (Map bucket7 (From Count) (To Nat)
    (Table (0 0) (1 1) (2 2) (3 3) (4 4) (5 4) (6 5) (7 5) (8 5) (9 5)
           (10 6) (11 6) (12 6) (13 6) (14 6) (15 6)))
  (Map buckets7 (From Belief) (To (Tuple Nat Nat))
    (Cases ((Tuple $p $n) (Tuple (bucket7 $p) (bucket7 $n)))))
  (Map cap7 (From Belief) (To (Tuple Nat Nat))
    (Cases ((Tuple $p $n) (Tuple (Min $p 7) (Min $n 7))))
```

```
59
60    (Representation StatusOnly  (Of Decide) (Encode status))
61    (Representation RoundedTV20 (Of Report) (Encode tv-grid-20))
62    (Representation RoundedTV10 (Of Report) (Encode tv-grid-10))
63    (Representation Grid10x7    (Of Report) (Encode tv-grid-10-7))
64    (Representation Buckets7    (Of Report) (Encode buckets7))
65    (Representation Capped7     (Of Report) (Encode cap7))
66    (Measure Uniform (On Belief) (Uniform))
67
68    (Derive DecisionQuotient (MinimizeForQueries Decide (Under Uniform)))
69    (Derive StatusRepair (Refine StatusOnly (Preserve Encoding) (Until Stable) (Under Uniform)))
70    (Derive Staged2 (Refine StatusOnly (Preserve Encoding) (Until (Steps 2))))      ; PROPOSED Until
        ↪ form
71    (Derive Staged2Machine (InduceExact (Output Staged2 Representation)))
72    (Derive FewestAtFifth
73      (WeakestWithin Report (Within (Rat 1 5)) (Kind Behavioral) (Under Uniform)
74                     (Objective Codes) (Search Exhaustive)))                       ; PROPOSED Objective
75    (Derive FewestAtHalf
76      (WeakestWithin Report (Within (Rat 1 2)) (Kind Behavioral) (Under Uniform)
77                     (Objective Codes) (Search Exhaustive))))
78  ;; @end
79
80  ;; @excerpt belief-memory-requests
81  !(dc.run BeliefMemory DecisionQuotient)
82  !(dc.run BeliefMemory StatusRepair)
83  !(dc.run BeliefMemory (CheckUpdates StatusOnly))
84  !(dc.run BeliefMemory (CheckBehavioral StatusOnly (Horizon (Steps 0))))
85  !(dc.run BeliefMemory (CheckBehavioral StatusOnly (Horizon (Steps 1))))
86  !(dc.run BeliefMemory Staged2)
87  !(dc.run BeliefMemory Staged2Machine)
88  !(dc.run BeliefMemory (Defects RoundedTV20))
89  !(dc.run BeliefMemory (MeasureBehavioral RoundedTV20 (Horizon (Steps 1))))
90  !(dc.run BeliefMemory (MeasureBehavioral RoundedTV20 (Horizon (Steps 2))))
91  !(dc.run BeliefMemory (MeasureBehavioral RoundedTV20 (Horizon (Steps 3))))
92  !(dc.run BeliefMemory (MeasureBehavioral RoundedTV20 (Horizon (Steps 6))))
93  !(dc.run BeliefMemory (MeasureBehavioral RoundedTV20 (Discount (Rat 9 10))))
94  !(dc.run BeliefMemory (CheckBehavioral RoundedTV20 (Within (Rat 1 5))))
95  !(dc.run BeliefMemory (CheckBehavioral RoundedTV20 (Within (Rat 1 2))))
96  !(dc.run BeliefMemory (CheckBehavioral RoundedTV20 (Within (Rat 1 5)) (Horizon (Steps 1))))
97  !(dc.run BeliefMemory (Defects RoundedTV10))
98  !(dc.run BeliefMemory (Defects Buckets7))
99  !(dc.run BeliefMemory (Defects Capped7))
100 !(dc.run BeliefMemory FewestAtFifth)
101 !(dc.run BeliefMemory FewestAtHalf)
102 ;; @end
103
104 ;; @excerpt gentle-memory
105 (DC.Module GentleMemory
106   (Profile Graded)
107   (Use NatArith)
108   (Import BeliefMemory
109     (Only Count Belief Status strength confidence status add-pos add-neg Dist01
110           tv-grid-20 tv-grid-10 tv-grid-10-7 bucket7 buckets7 cap7))
111   ;; decay only above eight observations, rounding up: small counts are never erased
112   (Map halve-gentle (From Belief) (To Belief)
113     (Cases ((Tuple $p $n)
114       (If (Le 8 (Add $p $n)) (Tuple (Quot (Add $p 1) 2) (Quot (Add $n 1) 2)) (Tuple $p $n)))))
115   (Machine Decide (States Belief) (Observe status status)
116     (Action pos add-pos) (Action neg add-neg) (Action decay halve-gentle))
117   (Machine Report (States Belief)
118     (Observe strength strength (Values Dist01))
119     (Observe confidence confidence (Values Dist01))
120     (Action pos add-pos) (Action neg add-neg) (Action decay halve-gentle))
121   (Representation StatusOnly  (Of Decide) (Encode status))
122   (Representation RoundedTV20 (Of Report) (Encode tv-grid-20))
123   (Representation RoundedTV10 (Of Report) (Encode tv-grid-10))
124   (Representation Grid10x7    (Of Report) (Encode tv-grid-10-7))
125   (Representation Buckets7    (Of Report) (Encode buckets7))
126   (Representation Capped7     (Of Report) (Encode cap7))
127   (Measure Uniform (On Belief) (Uniform))
128
129   (Derive DecisionQuotient (MinimizeForQueries Decide (Under Uniform)))
130   (EncodingFamily CountBuckets (Of Report)                                    ; PROPOSED
```

```
    (Generator (IntervalProduct Count Count (Tied True))))
  (EncodingFamily TVGrids (Of Report)
    (Generator (RoundedGrid (strength 1 40) (confidence 1 40))))
  (Derive BestBuckets
    (WeakestWithin Report (Within (Rat 1 5)) (Under Uniform) (Objective Codes)
                   (Family CountBuckets) (Search Exhaustive)))
  (Derive BestGrid
    (WeakestWithin Report (Within (Rat 1 5)) (Under Uniform) (Objective Codes)
                   (Family TVGrids) (Search Exhaustive)))
  (Derive Fewest
    (WeakestWithin Report (Within (Rat 1 5)) (Under Uniform) (Objective Codes)
                   (Search Exhaustive)))
  (Derive WeakestGreedy
    (WeakestWithin Report (Within (Rat 1 5)) (Under Uniform) (Search Greedy)))
  (Derive FewestDiscounted
    (WeakestWithin Report (Within (Rat 1 10)) (Discount (Rat 9 10)) (Under Uniform)
                   (Objective Codes) (Search Exhaustive)))
  (Derive FewestCenters
    (WeakestWithin Report (Within (Rat 1 5)) (Kind Radius) (Under Uniform)
                   (Objective Codes) (Search Exhaustive)))                      ; PROPOSED Kind
  (Derive ColdStoreCenters
    (InduceApprox (Output FewestCenters Representation)
                  (Representatives (Choose (Output FewestCenters Representatives)))))
  (Derive ColdStore     (InduceApprox RoundedTV20))
  (Derive ColdStoreGrid (InduceApprox Grid10x7))
  (Derive ColdStoreBest (InduceApprox (Output Fewest Representation))))
;; @end

;; @excerpt gentle-memory-requests
!(dc.run GentleMemory DecisionQuotient)
!(dc.run GentleMemory (Defects RoundedTV20))
!(dc.run GentleMemory (MeasureBehavioral RoundedTV20 (Horizon (Steps 1))))
!(dc.run GentleMemory (MeasureBehavioral RoundedTV20 (Horizon (Steps 4))))
!(dc.run GentleMemory (MeasureBehavioral RoundedTV20 (Horizon (Steps 5))))
!(dc.run GentleMemory (MeasureBehavioral RoundedTV20 (Discount (Rat 9 10))))
!(dc.run GentleMemory (CheckBehavioral RoundedTV20 (Within (Rat 1 5))))
!(dc.run GentleMemory (CheckBehavioral RoundedTV20 (Within (Rat 3 20))))
!(dc.run GentleMemory (CheckBehavioral Grid10x7 (Within (Rat 1 5))))
!(dc.run GentleMemory (Defects RoundedTV10))
!(dc.run GentleMemory (Defects Buckets7))
!(dc.run GentleMemory (Defects Capped7))
!(dc.run GentleMemory BestBuckets)
!(dc.run GentleMemory BestGrid)
!(dc.run GentleMemory Fewest)
!(dc.run GentleMemory WeakestGreedy)
!(dc.run GentleMemory FewestDiscounted)
!(dc.run GentleMemory ColdStore)
!(dc.run GentleMemory ColdStoreGrid)
!(dc.run GentleMemory (InduceApprox RoundedTV20 (Horizon (Steps 4))))
!(dc.run GentleMemory ColdStoreBest)
!(dc.run GentleMemory FewestCenters)
!(dc.run GentleMemory ColdStoreCenters)
;; @end

;; @excerpt agent-memory
(DC.Module AgentMemory
  (Profile Graded)
  (Use NatArith Parallel)                          ; PROPOSED library
  (Import GentleMemory (Only Report Grid10x7 Fewest))
  (ParallelMachine Beliefs6 (Component Report) (Copies 6) (Synchronize decay))
  (ProductRepresentation Grid6 (Of Beliefs6) (Component Grid10x7) (Copies 6))
  (ProductRepresentation Fewest6 (Of Beliefs6) (Component (Output Fewest Representation)) (Copies 6)))
;; @end
;; @excerpt agent-memory-requests
!(dc.run AgentMemory (CheckBehavioral Grid6 (Within (Rat 1 5))))
!(dc.run AgentMemory (CheckBehavioral Fewest6 (Within (Rat 1 5))))
!(dc.run AgentMemory (MeasureBehavioral Grid6 (Discount (Rat 9 10))))
;; @end

;; @excerpt receipt-ledger
(DC.Module ReceiptLedger
  (Profile Graded)
  (Use BitVectors Contracts NatArith)
```

```
;; bit i records receipt i: u1 u2 u3 u4 support the proposition, v1 v2 oppose it
(Carrier Receipts (Of (BitVector 6)) (All))
(Carrier Upto8 (Of Nat) (Enum 0 1 2 3 4 5 6 7 8))
(Carrier Upto4 (Of Nat) (Enum 0 1 2 3 4))
(Carrier Tally (Of (Tuple Nat Nat)) (Product Upto8 Upto4))
(Map counts (From Receipts) (To Tally)
  (Body ($r) (Tuple (PopCount (Slice $r 0 3)) (PopCount (Slice $r 4 5)))))   ; PROPOSED PopCount
(Map tally-strength (From Tally) (To Rational01)
  (Cases ((Tuple $p $n) (If (Eq (Add $p $n) 0) (Rat 1 2) (Div $p (Add $p $n))))))
(Map tally-confidence (From Tally) (To Rational01)
  (Cases ((Tuple $p $n) (Div (Add $p $n) (Add (Add $p $n) 4)))))
(Map ledger-strength   (From Receipts) (To Rational01) (Body ($r) (tally-strength (counts $r))))
(Map ledger-confidence (From Receipts) (To Rational01) (Body ($r) (tally-confidence (counts $r))))
(Map hear-u1 (From Receipts) (To Receipts) (Body ($r) (SetBit $r 0 1)))
(Map hear-u2 (From Receipts) (To Receipts) (Body ($r) (SetBit $r 1 1)))
(Map hear-u3 (From Receipts) (To Receipts) (Body ($r) (SetBit $r 2 1)))
(Map hear-u4 (From Receipts) (To Receipts) (Body ($r) (SetBit $r 3 1)))
(Map hear-v1 (From Receipts) (To Receipts) (Body ($r) (SetBit $r 4 1)))
(Map hear-v2 (From Receipts) (To Receipts) (Body ($r) (SetBit $r 5 1)))
(Map union (From (Product Receipts Receipts)) (To Receipts)
  (Body ($pair) (BitOr (GetTuple $pair 0) (GetTuple $pair 1))))               ; PROPOSED BitOr
(Tolerance Dist01 (On Rational01) (OneMinusAbsoluteDifference))
(Machine Ledger (States Receipts)
  (Observe strength ledger-strength (Values Dist01))
  (Observe confidence ledger-confidence (Values Dist01))
  (Action hear-u1 hear-u1) (Action hear-u2 hear-u2) (Action hear-u3 hear-u3)
  (Action hear-u4 hear-u4) (Action hear-v1 hear-v1) (Action hear-v2 hear-v2)
  (Merge merge union))
(Representation Counts (Of Ledger) (Encode counts))

;; merging two forks that kept only counts
(Carrier ForkPair (Of (Tuple (BitVector 6) (BitVector 6))) (Product Receipts Receipts))
(Map merged-tally (From ForkPair) (To Tally) (Body ($pair) (counts (union $pair))))
(Map add-tallies (From ForkPair) (To Tally)
  (Body ($pair) (Tuple (Add (GetTuple (counts (GetTuple $pair 0)) 0) (GetTuple (counts (GetTuple
    ↪ $pair 1)) 0))
                       (Add (GetTuple (counts (GetTuple $pair 0)) 1) (GetTuple (counts (GetTuple
    ↪ $pair 1)) 1)))))
(Map max-tallies (From ForkPair) (To Tally)
  (Body ($pair) (Tuple (Max (GetTuple (counts (GetTuple $pair 0)) 0) (GetTuple (counts (GetTuple
    ↪ $pair 1)) 0))
                       (Max (GetTuple (counts (GetTuple $pair 0)) 1) (GetTuple (counts (GetTuple
    ↪ $pair 1)) 1)))))
(Tolerance SameInput (On ForkPair) (Discrete))
(Tolerance TallyDistance (On Tally)
  (Meet (Pullback tally-strength Dist01) (Pullback tally-confidence Dist01)))   ; PROPOSED Meet
(Implementation AddMerge (Ideal merged-tally) (Actual add-tallies)
  (Audit ForkPair) (InputDistance SameInput) (OutputDistance TallyDistance))
(Implementation MaxMerge (Ideal merged-tally) (Actual max-tallies)
  (Audit ForkPair) (InputDistance SameInput) (OutputDistance TallyDistance))

;; three-way merge from the fork base: exact only when the forks heard different receipts
(Carrier Triple (Of (Tuple (BitVector 6) (BitVector 6) (BitVector 6))) (Product Receipts Receipts
    ↪ Receipts))
(Map based (From Triple) (To Bool)
  (Body ($t) (And (Eq (BitAnd (GetTuple $t 0) (GetTuple $t 1)) (GetTuple $t 0))
                  (Eq (BitAnd (GetTuple $t 0) (GetTuple $t 2)) (GetTuple $t 0)))))
(Carrier Forks (Of (Tuple (BitVector 6) (BitVector 6) (BitVector 6))) (Subset Triple based))
(Map fresh-news (From Forks) (To Bool)
  (Body ($t) (IsZero (BitAnd (BitXor (GetTuple $t 1) (GetTuple $t 0))
                             (BitXor (GetTuple $t 2) (GetTuple $t 0))))))
(Carrier CleanForks (Of (Tuple (BitVector 6) (BitVector 6) (BitVector 6))) (Subset Forks
    ↪ fresh-news))
(Map fork-union (From Forks) (To Tally)
  (Body ($t) (counts (BitOr (GetTuple $t 1) (GetTuple $t 2)))))
(Map three-way (From Forks) (To Tally)
  (Body ($t) (Tuple (Sub (Add (GetTuple (counts (GetTuple $t 1)) 0) (GetTuple (counts (GetTuple $t
    ↪ 2)) 0))
                         (GetTuple (counts (GetTuple $t 0)) 0))
                    (Sub (Add (GetTuple (counts (GetTuple $t 1)) 1) (GetTuple (counts (GetTuple $t
    ↪ 2)) 1))
                         (GetTuple (counts (GetTuple $t 0)) 1)))))
(Tolerance SameTriple (On Forks) (Discrete))
```

```
  (Implementation ThreeWay (Ideal fork-union) (Actual three-way)
    (Audit Forks) (InputDistance SameTriple) (OutputDistance TallyDistance))
  (Implementation ThreeWayClean (Ideal fork-union) (Actual three-way)
    (Audit CleanForks) (InputDistance SameTriple) (OutputDistance TallyDistance))

  ;; compaction below a fork horizon: u1 and u2 are older than the oldest open fork, so only
  ;; their number is kept; the identities of u3, u4, v1 and v2 are kept
  (Carrier Upto2 (Of Nat) (Enum 0 1 2))
  (Carrier Recent (Of (BitVector 4)) (All))
  (Carrier Compact (Of (Tuple Nat (BitVector 4))) (Product Upto2 Recent))
  (Map compact (From Receipts) (To Compact)
    (Body ($r) (Tuple (PopCount (Slice $r 0 1)) (Slice $r 2 5))))
  (Representation Compacted (Of Ledger) (Encode compact))   ; every hear action still allowed
  ;; the delivery protocol after compaction: u1 and u2 are never delivered again (a delivery
  ;; watermark at u2 that drops redeliveries), and forks are opened from the compacted state, so
  ;; a merge adds only newer receipts; the machine has exactly the operations the protocol allows
  (Machine SettledLedger (States Receipts)
    (Observe strength ledger-strength (Values Dist01))
    (Observe confidence ledger-confidence (Values Dist01))
    (Action hear-u3 hear-u3) (Action hear-u4 hear-u4)
    (Action hear-v1 hear-v1) (Action hear-v2 hear-v2))
  (Representation CompactedSettled (Of SettledLedger) (Encode compact))
  ;; merging two forks from their compacted codes: the old count of the first, the union of the
  ;; newer identities; exact only for forks that share the compacted part
  (Map compact-merge (From ForkPair) (To Tally)
    (Body ($pair)
      (Tuple (Add (PopCount (Slice (GetTuple $pair 0) 0 1))
                  (PopCount (BitOr (Slice (GetTuple $pair 0) 2 3) (Slice (GetTuple $pair 1) 2 3))))
             (PopCount (BitOr (Slice (GetTuple $pair 0) 4 5) (Slice (GetTuple $pair 1) 4 5))))))
  (Map same-old (From ForkPair) (To Bool)
    (Body ($pair) (Eq (Slice (GetTuple $pair 0) 0 1) (Slice (GetTuple $pair 1) 0 1))))
  (Carrier SettledForks (Of (Tuple (BitVector 6) (BitVector 6))) (Subset ForkPair same-old))
  (Tolerance SameSettled (On SettledForks) (Discrete))
  (Implementation CompactMerge (Ideal merged-tally) (Actual compact-merge)
    (Audit ForkPair) (InputDistance SameInput) (OutputDistance TallyDistance))
  (Implementation CompactMergeSettled (Ideal merged-tally) (Actual compact-merge)
    (Audit SettledForks) (InputDistance SameSettled) (OutputDistance TallyDistance)))
;; @end

;; @excerpt receipt-ledger-requests
!(dc.run ReceiptLedger (CheckNow Counts))
!(dc.run ReceiptLedger (CheckUpdates Counts))
!(dc.run ReceiptLedger (MeasureBehavioral Counts))
!(dc.run ReceiptLedger (CheckImplementation AddMerge (Grade 1 (Rat 6 35))))
!(dc.run ReceiptLedger (CheckImplementation AddMerge (Grade 1 (Rat 1 6))))
!(dc.run ReceiptLedger (CheckImplementation MaxMerge (Grade 1 (Rat 1 6))))
!(dc.run ReceiptLedger (CheckImplementation ThreeWay (Grade 1 0)))
!(dc.run ReceiptLedger (CheckImplementation ThreeWayClean (Grade 1 0)))
!(dc.run ReceiptLedger (CheckUpdates Compacted))                        ; Refuted without the protocol
!(dc.run ReceiptLedger (CheckBehavioral Compacted (Within (Rat 1 10))))
!(dc.run ReceiptLedger (CheckUpdates CompactedSettled))                 ; Checked under the protocol
!(dc.run ReceiptLedger (MeasureBehavioral CompactedSettled))
!(dc.run ReceiptLedger (CheckImplementation CompactMerge (Grade 1 0)))
!(dc.run ReceiptLedger (CheckImplementation CompactMergeSettled (Grade 1 0)))
;; @end
```

A.2 Example 2: `access_policy.metta`

Two modules: the policy as written, `AccessPolicy`, and the corrected policy with the audit's certificates, slices and quotients, `AccessPolicyFixed`.

```
;; Example 2: auditing an administrative access-control policy on forty-bit words.
;; Two modules: the policy as written, and the corrected policy with the audit's
;; certificates, slices and quotients.  Forms marked PROPOSED need ex2_registry_patch.json.

;; @excerpt access-policy
(DC.Module AccessPolicy
  (Profile (Dynamics Deterministic) (Values Crisp) (Carriers Symbolic))
  (Use BitVectors Families)                          ; PROPOSED library: Parameters, For, Switch,
```

```
    ↪ Exists
(Carrier Word (Of (BitVector 40)) (All))
(Carrier User (Of Symbol) (Enum alice bob carol dave))
(Carrier Role (Of Symbol) (Enum staff clerk approver auditor admin))
(Map slot (From User) (To Nat) (Table (alice 0) (bob 10) (carol 20) (dave 30)))
(Map rank (From Role) (To Nat) (Table (staff 0) (clerk 1) (approver 2) (auditor 3) (admin 4)))

;; bit 10u+r says that u is assigned r; bit 10u+5+r says that u has r active
(Map holds (Parameters ($u User) ($r Role)) (From Word) (To Bool)
  (Body ($x) (Eq (BitAt $x (Add (slot $u) (rank $r))) 1)))
(Map active (Parameters ($u User) ($r Role)) (From Word) (To Bool)
  (Body ($x) (Eq (BitAt $x (Add (Add (slot $u) 5) (rank $r))) 1)))
(Map other-admin (Parameters ($u User)) (From Word) (To Bool)
  (Body ($x) (Exists $v User (And (Not (Eq $v $u)) (active $v admin $x)))))

;; the can-assign rules; as written, the approver rule reads the auditor session
(Map may-receive (Parameters ($u User) ($r Role)) (From Word) (To Bool)
  (Body ($x)
    (Switch $r
      (staff    True)
      (clerk    (holds $u staff $x))
      (approver (And (holds $u clerk $x) (Not (active $u auditor $x))))
      (auditor  (And (holds $u staff $x) (Not (holds $u approver $x))))
      (admin    (holds $u staff $x)))))
(Map grant (Parameters ($u User) ($r Role)) (From Word) (To Word)
  (Body ($x) (SetBit $x (Add (slot $u) (rank $r)) 1)))
(Map drop (Parameters ($u User) ($r Role)) (From Word) (To Word)
  (Body ($x) (SetBit (SetBit $x (Add (slot $u) (rank $r)) 0) (Add (Add (slot $u) 5) (rank $r)) 0)))
(Map assign (Parameters ($u User) ($r Role)) (From Word) (To Word)
  (Body ($x) (If (And (other-admin $u $x) (may-receive $u $r $x)) (grant $u $r $x) $x)))
;; revocation cascades along role dependencies; as written, revoking clerk keeps approver
(Map revoke (Parameters ($u User) ($r Role)) (From Word) (To Word)
  (Body ($x)
    (If (other-admin $u $x)
        (Switch $r
          (staff    (drop $u staff (drop $u clerk (drop $u approver (drop $u auditor (drop $u admin
    ↪ $x))))))
          (clerk    (drop $u clerk $x))
          (approver (drop $u approver $x))
          (auditor  (drop $u auditor $x))
          (admin    (drop $u admin $x)))
        $x)))
;; sessions: approver and auditor are never active together
(Map activate (Parameters ($u User) ($r Role)) (From Word) (To Word)
  (Body ($x)
    (If (And (holds $u $r $x)
             (Switch $r
               (staff True) (clerk True) (admin True)
               (approver (Not (active $u auditor $x)))
               (auditor  (Not (active $u approver $x)))))
        (SetBit $x (Add (Add (slot $u) 5) (rank $r)) 1)
        $x)))
(Map deactivate (Parameters ($u User) ($r Role)) (From Word) (To Word)
  (Body ($x) (SetBit $x (Add (Add (slot $u) 5) (rank $r)) 0)))

;; access decisions and the auditor's alarms
(Map may-read (Parameters ($u User)) (From Word) (To Bool)
  (Body ($x) (Or (Or (active $u staff $x) (active $u clerk $x))
                 (Or (active $u approver $x) (active $u auditor $x)))))
(Map may-write     (Parameters ($u User)) (From Word) (To Bool)
  (Body ($x) (Or (active $u clerk $x) (active $u approver $x))))
(Map may-approve   (Parameters ($u User)) (From Word) (To Bool) (Body ($x) (active $u approver $x)))
(Map may-audit     (Parameters ($u User)) (From Word) (To Bool) (Body ($x) (active $u auditor $x)))
(Map may-configure (Parameters ($u User)) (From Word) (To Bool) (Body ($x) (active $u admin $x)))
(Map sod-breach (From Word) (To Bool)
  (Body ($x) (Exists $u User (And (holds $u approver $x) (holds $u auditor $x)))))
(Map clerkless-approval (From Word) (To Bool)
  (Body ($x) (Exists $u User (And (active $u approver $x) (Not (holds $u clerk $x))))))
(Map well-formed (Parameters ($u User)) (From Word) (To Bool)
  (Body ($x)
    (And (ForAll $r Role (Or (Not (active $u $r $x)) (holds $u $r $x)))
      (And (Or (Not (holds $u clerk $x)) (holds $u staff $x))
        (And (Or (Not (holds $u approver $x)) (holds $u clerk $x))
```

```
            (And (Or (Not (holds $u auditor $x)) (holds $u staff $x))
              (And (Or (Not (holds $u admin $x)) (holds $u staff $x))
                (Not (And (holds $u approver $x) (holds $u auditor $x))))))))))
  (Map consistent (From Word) (To Bool) (Body ($x) (ForAll $u User (well-formed $u $x))))

  (Machine Policy (States Word)
    (Observe (read $u)      (may-read $u)      (For $u User))
    (Observe (write $u)     (may-write $u)     (For $u User))
    (Observe (approve $u)   (may-approve $u)   (For $u User))
    (Observe (audit $u)     (may-audit $u)     (For $u User))
    (Observe (configure $u) (may-configure $u) (For $u User))
    (Action (assign $u $r)     (assign $u $r)     (For $u User) (For $r Role))
    (Action (revoke $u $r)     (revoke $u $r)     (For $u User) (For $r Role))
    (Action (activate $u $r)   (activate $u $r)   (For $u User) (For $r Role))
    (Action (deactivate $u $r) (deactivate $u $r) (For $u User) (For $r Role)))

  ;; alice: staff and admin, admin active; bob: staff and clerk, clerk active; carol: staff, active
  (Const start Word 34672145)
  (Carrier Live (Of (BitVector 40)) (Reach Policy (From start)))
  (Machine SoDWatch   (States Live) (Observe breach sod-breach) (ActionsOf Policy))         ; PROPOSED
  (Machine ClerkWatch (States Live) (Observe clerkless clerkless-approval) (ActionsOf Policy))
  (Machine Hygiene    (States Live) (Observe wellformed consistent) (ActionsOf Policy))
  (BlockPartition NoBreach (On Live)
    (Block Quiet (Body ($x) (Not (sod-breach $x))) start) (Successor Any Quiet Quiet))       ;
    ↪ PROPOSED Any
  (BlockPartition ClerksOnly (On Live)
    (Block Quiet (Body ($x) (Not (clerkless-approval $x))) start) (Successor Any Quiet Quiet))
  (BlockPartition WellFormed (On Live)
    (Block Good (Body ($x) (consistent $x)) start) (Successor Any Good Good)))
;; @end

;; @excerpt access-policy-requests
!(dc.run AccessPolicy (CheckBlockPartition NoBreach (Of SoDWatch)))
!(dc.run AccessPolicy (CheckBlockPartition ClerksOnly (Of ClerkWatch)))
!(dc.run AccessPolicy (CheckBlockPartition WellFormed (Of Hygiene)))
;; @end

;; @excerpt access-policy-fixed
(DC.Module AccessPolicyFixed
  (Profile (Dynamics Deterministic) (Values Crisp) (Carriers Symbolic))
  (Use BitVectors Families)
  (Import AccessPolicy
    (Only Word User Role slot rank holds active other-admin grant drop activate deactivate
          may-read may-write may-approve may-audit may-configure sod-breach clerkless-approval
          well-formed consistent start))
  (Map may-receive (Parameters ($u User) ($r Role)) (From Word) (To Bool)
    (Body ($x)
      (Switch $r
        (staff    True)
        (clerk    (holds $u staff $x))
        (approver (And (holds $u clerk $x) (Not (holds $u auditor $x))))       ; the assignment
        (auditor  (And (holds $u staff $x) (Not (holds $u approver $x))))
        (admin    (holds $u staff $x)))))
  (Map assign (Parameters ($u User) ($r Role)) (From Word) (To Word)
    (Body ($x) (If (And (other-admin $u $x) (may-receive $u $r $x)) (grant $u $r $x) $x)))
  (Map revoke (Parameters ($u User) ($r Role)) (From Word) (To Word)
    (Body ($x)
      (If (other-admin $u $x)
          (Switch $r
            (staff    (drop $u staff (drop $u clerk (drop $u approver (drop $u auditor (drop $u admin
    ↪ $x))))))
            (clerk    (drop $u clerk (drop $u approver $x)))                    ; cascades
            (approver (drop $u approver $x))
            (auditor  (drop $u auditor $x))
            (admin    (drop $u admin $x)))
          $x)))
  (Machine Policy (States Word)
    (Observe (read $u)      (may-read $u)      (For $u User))
    (Observe (write $u)     (may-write $u)     (For $u User))
    (Observe (approve $u)   (may-approve $u)   (For $u User))
    (Observe (audit $u)     (may-audit $u)     (For $u User))
    (Observe (configure $u) (may-configure $u) (For $u User))
    (Action (assign $u $r)     (assign $u $r)     (For $u User) (For $r Role))
```

```
    (Action (revoke $u $r)     (revoke $u $r)     (For $u User) (For $r Role))
    (Action (activate $u $r)   (activate $u $r)   (For $u User) (For $r Role))
    (Action (deactivate $u $r) (deactivate $u $r) (For $u User) (For $r Role)))
  (Carrier Live (Of (BitVector 40)) (Reach Policy (From start)))
  (Machine LivePolicy (States Live) (Like Policy))                                    ; PROPOSED
  (Machine SoDWatch   (States Live) (Observe breach sod-breach) (ActionsOf Policy))
  (Machine ClerkWatch (States Live) (Observe clerkless clerkless-approval) (ActionsOf Policy))

  ;; safety certificates on the reachable configurations
  (BlockPartition NoBreach (On Live)
    (Block Quiet (Body ($x) (Not (sod-breach $x))) start) (Successor Any Quiet Quiet))
  (BlockPartition ClerksOnly (On Live)
    (Block Quiet (Body ($x) (Not (clerkless-approval $x))) start) (Successor Any Quiet Quiet))
  (BlockPartition ClerksOnlyProved (On Live)
    (Block Quiet (Body ($x) (Not (clerkless-approval $x))) start) (Successor Any Quiet Quiet)
    (Invariant consistent))                                                           ; PROPOSED

  ;; compaction: assignments alone, the whole interface, and one targeted question
  (Map assignments (From Word) (To (BitVector 20))
    (Body ($x) (Gather $x 0 1 2 3 4 10 11 12 13 14 20 21 22 23 24 30 31 32 33 34)))       ; PROPOSED
  (Representation AssignmentsOnly (Of Policy) (Encode assignments))
  (Machine BobApproves     (States Word) (Observe approve-bob (may-approve bob)) (ActionsOf Policy))
  (Machine BobApprovesLive (States Live) (Observe approve-bob (may-approve bob)) (ActionsOf Policy))
  (Map bob-cone (From Word) (To (BitVector 17))
    (Body ($x) (Gather $x 0 4 9 10 11 12 13 14 17 18 19 20 24 29 30 34 39)))
  (Map bob-bits (From Word) (To (BitVector 10)) (Body ($x) (Slice $x 10 19)))
  (Representation BobCone (Of BobApproves) (Encode bob-cone))
  (Representation BobOnly (Of BobApproves) (Encode bob-bits))
  (Derive WholeInterface (MinimizeForQueries Policy))
  (Derive LiveInterface  (MinimizeForQueries LivePolicy))
  (Derive BobSlice       (InduceExact BobCone))
  (Derive BobQuotient    (MinimizeForQueries BobApproves))
  (Derive BobLiveQuotient (MinimizeForQueries BobApprovesLive)))
;; @end

;; @excerpt access-policy-fixed-requests
!(dc.run AccessPolicyFixed (CheckBlockPartition NoBreach (Of SoDWatch)))
!(dc.run AccessPolicyFixed (CheckBlockPartition ClerksOnly (Of ClerkWatch) (Budget (Unrollings 6))))
!(dc.run AccessPolicyFixed (CheckBlockPartition ClerksOnlyProved (Of ClerkWatch)))
!(dc.run AccessPolicyFixed (CheckNow AssignmentsOnly))
!(dc.run AccessPolicyFixed WholeInterface)
!(dc.run AccessPolicyFixed LiveInterface)
!(dc.run AccessPolicyFixed BobSlice)
!(dc.run AccessPolicyFixed (Defects BobOnly))
!(dc.run AccessPolicyFixed (InduceExact BobOnly))
!(dc.run AccessPolicyFixed (CheckBehavioral BobOnly))
!(dc.run AccessPolicyFixed BobQuotient)
!(dc.run AccessPolicyFixed BobLiveQuotient)
;; @end
```

A.3 Example 3: `skill_share.metta`

Three modules: the progress task `SkillShare`, the docking task `SkillShareDock`, and `PalletBay`.

```
;; Example 3: sharing navigation skills between robots whose cameras see the bay in
;; different frames.  Three modules: the progress task, the docking task, and a bay with
;; an extra pallet.  Forms marked PROPOSED need ex3_registry_patch.json (and NatArith from
;; ex1_registry_patch.json).

;; @excerpt skill-share
(DC.Module SkillShare
  (Profile Crisp)
  (Use Transfers Hives NatArith)
  (Carrier Coord  (Of Nat) (Enum 0 1 2 3 4))
  (Carrier Grid   (Of (Tuple Nat Nat)) (Product Coord Coord))
  (Carrier Dir    (Of Symbol) (Enum up right down left))
  (Carrier Reward (Of Rational) (Enum 0 (Rat 1 2) 1))

```

```
;; the dihedral group of the square, on cells (row from the top, column from the left)
;; and on image directions; r90 is a clockwise quarter turn on the screen
(Map r90-cell  (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple $c (Sub 4 $r)))))
(Map r270-cell (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple (Sub 4 $c) $r))))
(Map mv-cell   (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple $r (Sub 4 $c)))))   ; mirror
  ↪ left-right
(Map mh-cell   (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple (Sub 4 $r) $c))))   ; mirror
  ↪ up-down
(Map md-cell   (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple $c $r))))           ; transpose
(Map ma-cell   (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple (Sub 4 $c) (Sub 4 $r)))))
(Map r180-cell (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple (Sub 4 $r) (Sub 4 $c)))))
(Map same-cell (From Grid) (To Grid) (Cases ((Tuple $r $c) (Tuple $r $c))))
(Map r90-dir   (From Dir) (To Dir) (Cases (up right) (right down) (down left) (left up)))
(Map r270-dir  (From Dir) (To Dir) (Cases (up left) (right up) (down right) (left down)))
(Map mv-dir    (From Dir) (To Dir) (Cases (up up) (right left) (down down) (left right)))
(Map mh-dir    (From Dir) (To Dir) (Cases (up down) (right right) (down up) (left left)))
(Map md-dir    (From Dir) (To Dir) (Cases (up left) (right down) (down right) (left up)))
(Map ma-dir    (From Dir) (To Dir) (Cases (up right) (right up) (down left) (left down)))
(Map r180-dir  (From Dir) (To Dir) (Cases (up down) (right left) (down up) (left right)))
(Map same-dir  (From Dir) (To Dir) (Cases (up up) (right right) (down down) (left left)))

;; the bay: goal at the centre, pillars at rows 1 and 3 of the outer columns
(Map pillar (From Grid) (To Bool)
  (Cases ((Tuple $r $c) (And (Or (Eq $r 1) (Eq $r 3)) (Or (Eq $c 0) (Eq $c 4))))))
(Map dist (From Grid) (To Nat)
  (Cases ((Tuple $r $c) (Add (If (Le $r 2) (Sub 2 $r) (Sub $r 2)) (If (Le $c 2) (Sub 2 $c) (Sub $c
  ↪ 2))))))
(Map step (From (Product Grid Dir)) (To Grid)       ; blocked moves stay put
  (Cases
    ((Tuple (Tuple $r $c) up)
      (If (Eq $r 0) (Tuple $r $c)
        (If (pillar (Tuple (Sub $r 1) $c)) (Tuple $r $c) (Tuple (Sub $r 1) $c))))
    ((Tuple (Tuple $r $c) down)
      (If (Eq $r 4) (Tuple $r $c)
        (If (pillar (Tuple (Add $r 1) $c)) (Tuple $r $c) (Tuple (Add $r 1) $c))))
    ((Tuple (Tuple $r $c) left)
      (If (Eq $c 0) (Tuple $r $c)
        (If (pillar (Tuple $r (Sub $c 1))) (Tuple $r $c) (Tuple $r (Sub $c 1)))))
    ((Tuple (Tuple $r $c) right)
      (If (Eq $c 4) (Tuple $r $c)
        (If (pillar (Tuple $r (Add $c 1))) (Tuple $r $c) (Tuple $r (Add $c 1)))))))
(Map progress (From (Product Grid Dir)) (To Reward)  ; 1 closer, 1/2 no change, 0 farther
  (Body ($xm)
    (If (Lt (dist (step $xm)) (dist (GetTuple $xm 0))) 1
      (If (Eq (dist (step $xm)) (dist (GetTuple $xm 0))) (Rat 1 2) 0))))

;; three robots: A's camera is aligned with the bay, B's is turned a quarter turn
;; clockwise, C's sees the bay transposed; each commands moves in its own image frame
(Map free-A (From Grid) (To Bool) (Body ($y) (Not (pillar $y))))
(Map free-B (From Grid) (To Bool) (Body ($y) (Not (pillar (r270-cell $y)))))
(Map free-C (From Grid) (To Bool) (Body ($y) (Not (pillar (md-cell $y)))))
(Carrier FreeA (Of (Tuple Nat Nat)) (Subset Grid free-A))
(Carrier FreeB (Of (Tuple Nat Nat)) (Subset Grid free-B))
(Carrier FreeC (Of (Tuple Nat Nat)) (Subset Grid free-C))
(Map respond-A (From (Product FreeA Dir)) (To Reward) (Body ($ym) (progress $ym)))
(Map respond-B (From (Product FreeB Dir)) (To Reward)
  (Body ($ym) (progress (Tuple (r270-cell (GetTuple $ym 0)) (r270-dir (GetTuple $ym 1))))))
(Map respond-C (From (Product FreeC Dir)) (To Reward)
  (Body ($ym) (progress (Tuple (md-cell (GetTuple $ym 0)) (md-dir (GetTuple $ym 1))))))
(MethodSystem A (States FreeA) (Methods Dir) (Response respond-A))
(MethodSystem B (States FreeB) (Methods Dir) (Response respond-B))
(MethodSystem C (States FreeC) (Methods Dir) (Response respond-C))

;; the library: AB and BC from camera calibration, CA learned by matching recorded
;; responses; a pullback maps target cells to source cells, a pushforward source commands
;; to target commands
(Transfer AB      (Source A) (Target B) (StatePullback r270-cell) (MethodPushforward r90-dir))
(Transfer BC      (Source B) (Target C) (StatePullback mv-cell)   (MethodPushforward mv-dir))
(Transfer CA      (Source C) (Target A) (StatePullback r90-cell)  (MethodPushforward r270-dir))
(Transfer CAcal   (Source C) (Target A) (StatePullback md-cell)   (MethodPushforward md-dir))
(Transfer NaiveAB (Source A) (Target B) (StatePullback r270-cell) (MethodPushforward same-dir))
;; every exact C-to-A transfer among the eight frame changes (pullback, pushforward)
(Derive CandidatesCA
```

```
    (FindTransfers C A                                                                  ; PROPOSED
      (Candidates (same-cell same-dir) (r90-cell r270-dir) (r180-cell r180-dir) (r270-cell r90-dir)
                  (mv-cell mv-dir) (mh-cell mh-dir) (md-cell md-dir) (ma-cell ma-dir))))
  (Derive ABchecked (CheckTransfer AB))
  (Derive BCchecked (CheckTransfer BC))
  (Derive CAchecked (CheckTransfer CA))
  (Derive RoundTrip (ComposeTransfers (Output ABchecked Transfer) (Output BCchecked Transfer)
                                      (Output CAchecked Transfer)))

  ;; response symmetries, each in its robot's own frame
  (Symmetry FlipUD-A (Of A) (States mh-cell) (Methods mh-dir))
  (Symmetry FlipLR-A (Of A) (States mv-cell) (Methods mv-dir))
  (Symmetry FlipUD-B (Of B) (States mh-cell) (Methods mh-dir))
  (Symmetry FlipLR-B (Of B) (States mv-cell) (Methods mv-dir))
  (Symmetry FlipUD-C (Of C) (States mh-cell) (Methods mh-dir))
  (Symmetry FlipLR-C (Of C) (States mv-cell) (Methods mv-dir))
  (TransferHive Strict (Member A) (Member B) (Member C)
    (Edge AB) (Edge BC) (Edge CA) (Cycle A B C))
  (TransferHive FullGauge (Member A) (Member B) (Member C)
    (Edge AB) (Edge BC) (Edge CA) (Cycle A B C)
    (Gauge A FlipUD-A FlipLR-A) (Gauge B FlipUD-B FlipLR-B) (Gauge C FlipUD-C FlipLR-C))
  (TransferHive Copied (Member A) (Member B) (Member C)
    (Edge AB) (Edge BC) (Edge CA) (Cycle A B C)
    (Gauge A FlipUD-A) (Gauge B FlipUD-B) (Gauge C FlipUD-C))
  (TransferHive Transported (Member A) (Member B) (Member C)
    (Edge AB) (Edge BC) (Edge CA) (Cycle A B C)
    (Gauge A FlipUD-A) (Gauge B FlipLR-B) (Gauge C FlipLR-C))
  (TransferHive FromBase (Member A) (Member B) (Member C)                               ; PROPOSED
    (Edge AB) (Edge BC) (Edge CA) (Cycle A B C) (GaugeFrom A FlipUD-A))
  (Derive Unsaturated (KernelOf Strict))
  (Derive SaturatedFull (KernelOf FullGauge (Saturate Gauge)))
  (Derive SaturatedCopied (KernelOf Copied (Saturate Gauge)))
  (Derive SaturatedTransported (KernelOf Transported (Saturate Gauge)))

  ;; a skill: the greedy move, ties broken by command order (as a frozen table)
  (Map greedy-A (From FreeA) (To Dir)
    (Table ((Tuple 0 0) right) ((Tuple 0 1) right) ((Tuple 0 2) down) ((Tuple 0 3) down) ((Tuple 0 4)
    ↪ left)
           ((Tuple 1 1) right) ((Tuple 1 2) down) ((Tuple 1 3) down)
           ((Tuple 2 0) right) ((Tuple 2 1) right) ((Tuple 2 2) up) ((Tuple 2 3) left) ((Tuple 2 4)
    ↪ left)
           ((Tuple 3 1) up) ((Tuple 3 2) up) ((Tuple 3 3) up)
           ((Tuple 4 0) right) ((Tuple 4 1) up) ((Tuple 4 2) up) ((Tuple 4 3) up) ((Tuple 4 4) left)))
  (Derive SkillRoundTrip (TransportSkill (Output RoundTrip Transfer) greedy-A))        ; PROPOSED
  ;; a skill whose rewards are not mirror-symmetric: always move up
  (Map always-up (From FreeA) (To Dir) (Body ($x) up))
  (Derive UpRoundTrip (TransportSkill (Output RoundTrip Transfer) always-up)))
;; @end

;; @excerpt skill-share-requests
!(dc.run SkillShare ABchecked)
!(dc.run SkillShare BCchecked)
!(dc.run SkillShare CAchecked)
!(dc.run SkillShare (CheckTransfer CAcal))
!(dc.run SkillShare CandidatesCA)
!(dc.run SkillShare (CheckTransfer NaiveAB))
!(dc.run SkillShare (CheckValuePullback NaiveAB))
!(dc.run SkillShare RoundTrip)
!(dc.run SkillShare (CheckTransferHive Strict))
!(dc.run SkillShare (CheckTransferHive FullGauge))
!(dc.run SkillShare (CheckTransferHive Copied))
!(dc.run SkillShare (CheckTransferHive Transported))
!(dc.run SkillShare (CheckTransferHive FromBase))
!(dc.run SkillShare (CheckKernelCoherence (Output Unsaturated Hive)))
!(dc.run SkillShare (Holonomy (Output Unsaturated Hive) (Cycle A B C)))
!(dc.run SkillShare (ConsensusCost (Output Unsaturated Hive)))
!(dc.run SkillShare (CheckKernelCoherence (Output SaturatedFull Hive)))
!(dc.run SkillShare (CheckKernelCoherence (Output SaturatedCopied Hive)))
!(dc.run SkillShare (CheckKernelCoherence (Output SaturatedTransported Hive)))
!(dc.run SkillShare SkillRoundTrip)
!(dc.run SkillShare UpRoundTrip)
;; @end

```

```
;; @excerpt skill-share-dock
(DC.Module SkillShareDock
  (Profile Crisp)
  (Use Transfers Hives NatArith)
  (Import SkillShare
    (Only Grid Dir Reward FreeA FreeB FreeC r90-cell r270-cell mv-cell mh-cell md-cell
          r90-dir r270-dir mv-dir mh-dir md-dir step progress))
  ;; the charger faces north: arriving at the goal from any other side earns 1/2
  (Map docking (From (Product Grid Dir)) (To Reward)
    (Body ($xm)
      (If (And (Eq (step $xm) (Tuple 2 2))
               (And (Not (Eq (GetTuple $xm 0) (Tuple 2 2))) (Not (Eq (GetTuple $xm 1) down))))
          (Rat 1 2)
          (progress $xm))))
  (Map dock-A (From (Product FreeA Dir)) (To Reward) (Body ($ym) (docking $ym)))
  (Map dock-B (From (Product FreeB Dir)) (To Reward)
    (Body ($ym) (docking (Tuple (r270-cell (GetTuple $ym 0)) (r270-dir (GetTuple $ym 1))))))
  (Map dock-C (From (Product FreeC Dir)) (To Reward)
    (Body ($ym) (docking (Tuple (md-cell (GetTuple $ym 0)) (md-dir (GetTuple $ym 1))))))
  (MethodSystem A (States FreeA) (Methods Dir) (Response dock-A))
  (MethodSystem B (States FreeB) (Methods Dir) (Response dock-B))
  (MethodSystem C (States FreeC) (Methods Dir) (Response dock-C))
  (Transfer AB    (Source A) (Target B) (StatePullback r270-cell) (MethodPushforward r90-dir))
  (Transfer BC    (Source B) (Target C) (StatePullback mv-cell)   (MethodPushforward mv-dir))
  (Transfer CAold (Source C) (Target A) (StatePullback r90-cell)  (MethodPushforward r270-dir))
  (Transfer CAnew (Source C) (Target A) (StatePullback r270-cell) (MethodPushforward r90-dir))
  (Symmetry FlipUD-A (Of A) (States mh-cell) (Methods mh-dir))
  (Symmetry FlipLR-A (Of A) (States mv-cell) (Methods mv-dir))
  (Symmetry FlipUD-B (Of B) (States mh-cell) (Methods mh-dir))
  (Symmetry FlipLR-B (Of B) (States mv-cell) (Methods mv-dir))
  (Symmetry FlipUD-C (Of C) (States mh-cell) (Methods mh-dir))
  (Symmetry FlipLR-C (Of C) (States mv-cell) (Methods mv-dir))
  (TransferHive StaleGauge (Member A) (Member B) (Member C)
    (Edge AB) (Edge BC) (Edge CAnew) (Cycle A B C)
    (Gauge A FlipUD-A FlipLR-A) (Gauge B FlipUD-B FlipLR-B) (Gauge C FlipUD-C FlipLR-C))
  (TransferHive OldLibrary (Member A) (Member B) (Member C)
    (Edge AB) (Edge BC) (Edge CAold) (Cycle A B C)
    (Gauge A FlipLR-A) (Gauge B FlipUD-B) (Gauge C FlipUD-C))
  (TransferHive Docking (Member A) (Member B) (Member C)
    (Edge AB) (Edge BC) (Edge CAnew) (Cycle A B C)
    (Gauge A FlipLR-A) (Gauge B FlipUD-B) (Gauge C FlipUD-C))
  (Derive DockSaturated (KernelOf Docking (Saturate Gauge))))
;; @end

;; @excerpt skill-share-dock-requests
!(dc.run SkillShareDock (CheckTransfer CAold))
!(dc.run SkillShareDock (CheckTransfer CAold (Metric ExpectedReward) (Within (Rat 1 2))))
!(dc.run SkillShareDock (CheckTransfer CAnew))
!(dc.run SkillShareDock (CheckTransferHive StaleGauge))
!(dc.run SkillShareDock (CheckTransferHive OldLibrary))
!(dc.run SkillShareDock (CheckTransferHive Docking))
!(dc.run SkillShareDock (CheckKernelCoherence (Output DockSaturated Hive)))
;; @end

;; @excerpt pallet-bay
(DC.Module PalletBay
  (Profile Crisp)
  (Use Transfers NatArith)
  (Import SkillShare
    (Only Grid Dir Reward FreeA FreeB pillar dist md-cell md-dir mv-cell mv-dir r270-cell r90-dir
          A B AB ABchecked))
  ;; C's bay has a pallet on the north side of the goal's column (world cell (0,2))
  (Map blocked (From Grid) (To Bool) (Body ($y) (Or (pillar $y) (Eq $y (Tuple 0 2)))))
  (Map step-p (From (Product Grid Dir)) (To Grid)
    (Cases
      ((Tuple (Tuple $r $c) up)
        (If (Eq $r 0) (Tuple $r $c)
          (If (blocked (Tuple (Sub $r 1) $c)) (Tuple $r $c) (Tuple (Sub $r 1) $c))))
      ((Tuple (Tuple $r $c) down)
        (If (Eq $r 4) (Tuple $r $c)
          (If (blocked (Tuple (Add $r 1) $c)) (Tuple $r $c) (Tuple (Add $r 1) $c))))
      ((Tuple (Tuple $r $c) left)
        (If (Eq $c 0) (Tuple $r $c)
```

```
          (If (blocked (Tuple $r (Sub $c 1))) (Tuple $r $c) (Tuple $r (Sub $c 1)))))
      ((Tuple (Tuple $r $c) right)
        (If (Eq $c 4) (Tuple $r $c)
          (If (blocked (Tuple $r (Add $c 1))) (Tuple $r $c) (Tuple $r (Add $c 1)))))))
  (Map progress-p (From (Product Grid Dir)) (To Reward)
    (Body ($xm)
      (If (Lt (dist (step-p $xm)) (dist (GetTuple $xm 0))) 1
        (If (Eq (dist (step-p $xm)) (dist (GetTuple $xm 0))) (Rat 1 2) 0))))
  (Map free-Cp (From Grid) (To Bool) (Body ($y) (Not (blocked (md-cell $y)))))
  (Carrier FreeCp (Of (Tuple Nat Nat)) (Subset Grid free-Cp))
  (Map respond-Cp (From (Product FreeCp Dir)) (To Reward)
    (Body ($ym) (progress-p (Tuple (md-cell (GetTuple $ym 0)) (md-dir (GetTuple $ym 1))))))
  (MethodSystem Cp (States FreeCp) (Methods Dir) (Response respond-Cp))
  (Transfer ACp (Source A) (Target Cp) (StatePullback md-cell) (MethodPushforward md-dir))
  (Transfer BCp (Source B) (Target Cp) (StatePullback mv-cell) (MethodPushforward mv-dir))
  (Derive BCpChecked (CheckTransfer BCp (Metric ExpectedReward) (Within (Rat 1 2))))
  (Derive AtoCp (ComposeTransfers (Output ABchecked Transfer) (Output BCpChecked Transfer))))
;; @end

;; @excerpt pallet-bay-requests
!(dc.run PalletBay (CheckTransfer ACp))
!(dc.run PalletBay (CheckTransfer ACp (Metric ExpectedReward) (Within (Rat 1 2))))
!(dc.run PalletBay (CheckTransfer ACp (Metric ExpectedReward) (Within (Rat 1 4))))
!(dc.run PalletBay AtoCp)
;; @end
```

A.4 Example 4: `market_maker.metta`

Four modules: `MarketDesk`, `InformedFlow`, `NewsLeak` and `ShiftMonitors`.

```
;; Example 4: a market maker across regime shifts.  MeTTaDinC worked example, 4 November 2026.
;;
;; Four modules:
;;   MarketDesk     the quoting problem in four regimes, the risk-bucket compression, the
;;                  mirror between the two one-sided regimes, optimality of the calm policy
;;   InformedFlow   toxic flow as a hidden distinction: averages, coarse and refined policies,
;;                  a noisy toxicity signal with a price
;;   NewsLeak       did the news leak?  trace and behavioral claims about the desk's own tape
;;   ShiftMonitors  likelihood-ratio monitors: exact validity and blindness, Estimated detections
;;
;; Forms marked PROPOSED need ex4_registry_patch.json; NatArith comes from Example 1 and
;; Families (Parameters, Switch, Like) from Example 2.

;; ------------------------------------------------------------------------------ the desk
(DC.Module MarketDesk
  (Profile Stochastic)
  (Use Decisions Laws Families NatArith)            ; Laws, DecisionProcess: PROPOSED
  (Carrier Inv    (Of Int)    (Enum -2 -1 0 1 2))     ; inventory in lots
  (Carrier Quote  (Of Symbol) (Enum tight wide lean))
  (Carrier Market (Of Symbol) (Enum calm toxic buyers sellers informed))
  (Carrier Bit    (Of Nat)    (Enum 0 1))
  (Carrier Offset (Of Nat)    (Enum 0 1 2))           ; ticks from the mid; 0: side not shown
  (Carrier Bucket (Of Nat)    (Enum 0 1 2))           ; lots held, on either side
  (Carrier One    (Of Nat)    (Enum 0))
  (Tolerance Ticks (On Rational) (OneMinusAbsoluteDifference))
  (Measure Flat (On Inv) (Masses (0 1)))

  ;; the quoting rule: lean is tight on the side that reduces the position; at the limits
  ;; the side that would grow the position is withdrawn
  (Map ask-offset (From (Product Inv Quote)) (To Offset)
    (Cases ((Tuple $q tight) (If (Eq $q -2) 0 1))
           ((Tuple $q wide)  (If (Eq $q -2) 0 2))
           ((Tuple $q lean)  (If (Eq $q -2) 0 (If (Lt $q 0) 2 1)))))
  (Map bid-offset (From (Product Inv Quote)) (To Offset)
    (Cases ((Tuple $q tight) (If (Eq $q 2) 0 1))
           ((Tuple $q wide)  (If (Eq $q 2) 0 2))
           ((Tuple $q lean)  (If (Eq $q 2) 0 (If (Lt 0 $q) 2 1)))))

  ;; a market is a fill probability per side and offset, and an adverse-selection cost per
  ;; fill (the expected move against the maker after the fill, in ticks)
```

```
  (Map ask-fill (Parameters ($m Market)) (From Offset) (To Rational)
    (Cases (0 0)
           (1 (Switch $m (calm (Rat 1 2)) (toxic (Rat 1 2)) (buyers (Rat 3 4)) (sellers (Rat 1 4))
                         (informed (Rat 1 2))))
           (2 (Switch $m (calm (Rat 1 8)) (toxic (Rat 1 8)) (buyers (Rat 3 16)) (sellers (Rat 1 16))
                         (informed (Rat 1 8))))))
  (Map bid-fill (Parameters ($m Market)) (From Offset) (To Rational)
    (Cases (0 0)
           (1 (Switch $m (calm (Rat 1 2)) (toxic (Rat 1 2)) (buyers (Rat 1 4)) (sellers (Rat 3 4))
                         (informed (Rat 1 2))))
           (2 (Switch $m (calm (Rat 1 8)) (toxic (Rat 1 8)) (buyers (Rat 1 16)) (sellers (Rat 3 16))
                         (informed (Rat 1 8))))))
  (Map theta (Parameters ($m Market)) (From One) (To Rational)
    (Body ($u) (Switch $m (calm 0) (toxic (Rat 1 2)) (buyers 0) (sellers 0) (informed 1))))
  (Map edge (Parameters ($m Market)) (From Offset) (To Rational)
    (Body ($o) (Sub $o (theta $m 0))))

  ;; one step: a buyer lifts the ask and a seller hits the bid, independently
  (Map fills (Parameters ($m Market)) (From (Product Inv Quote)) (To (Law (Tuple Bit Bit)))
    (Body ($xa) (Indep (Coin (ask-fill $m (ask-offset $xa))) (Coin (bid-fill $m (bid-offset $xa))))))
  (Map after (From (Product Inv Bit Bit)) (To Int)
    (Body ($t) (Add (Sub (GetTuple $t 0) (GetTuple $t 1)) (GetTuple $t 2))))
  (Map held (From (Product Inv Bit Bit)) (To Nat)
    (Body ($t) (Max (after $t) (Sub 0 (after $t)))))
  (Map step (Parameters ($m Market)) (From (Product Inv Quote)) (To (Law Inv))
    (Body ($xa)
      (Image $f (after (Tuple (GetTuple $xa 0) (GetTuple $f 0) (GetTuple $f 1)))
        (fills $m $xa))))
  (Map pnl (Parameters ($m Market)) (From (Product Inv Quote)) (To (Law Rational))
    (Body ($xa)
      (Image $f
        (Sub (Add (Mul (GetTuple $f 0) (edge $m (ask-offset $xa)))
                  (Mul (GetTuple $f 1) (edge $m (bid-offset $xa))))
             (Mul (Rat 1 4) (held (Tuple (GetTuple $xa 0) (GetTuple $f 0) (GetTuple $f 1)))))
        (fills $m $xa))))

  ;; four regimes as decision processes (each is also a machine observing expected rewards)
  (DecisionProcess CalmDesk    (States Inv) (Actions Quote) (Kernel (step calm))    (RewardLaw (pnl
    ↪ calm))
    (Discount (Rat 9 10)) (Values Ticks))
  (DecisionProcess ToxicDesk   (States Inv) (Actions Quote) (Kernel (step toxic))   (RewardLaw (pnl
    ↪ toxic))
    (Discount (Rat 9 10)) (Values Ticks))
  (DecisionProcess BuyersDesk  (States Inv) (Actions Quote) (Kernel (step buyers))  (RewardLaw (pnl
    ↪ buyers))
    (Discount (Rat 9 10)) (Values Ticks))
  (DecisionProcess SellersDesk (States Inv) (Actions Quote) (Kernel (step sellers)) (RewardLaw (pnl
    ↪ sellers))
    (Discount (Rat 9 10)) (Values Ticks))

  ;; the risk-bucket compression and the mirror
  (Map bucket (From Inv) (To Bucket) (Body ($q) (Max $q (Sub 0 $q))))
  (Map mirror (From Inv) (To Inv) (Body ($q) (Sub 0 $q)))
  (Representation Risk       (Of CalmDesk)   (Encode bucket))
  (Representation RiskToxic  (Of ToxicDesk)  (Encode bucket))
  (Representation RiskBuyers (Of BuyersDesk) (Encode bucket))
  (AbstractionMap CalmMirror (Source CalmDesk)   (Target CalmDesk)    (Map mirror))
  (AbstractionMap Flip       (Source BuyersDesk) (Target SellersDesk) (Map mirror))

  ;; policies the desk has on file
  (Map calm-policy    (From Inv) (To Quote) (Table (-2 tight) (-1 lean)  (0 tight) (1 lean)  (2
    ↪ tight)))
  (Map sellers-policy (From Inv) (To Quote) (Table (-2 tight) (-1 tight) (0 tight) (1 lean)  (2
    ↪ tight)))

  (Derive CalmQuotient (InduceExact Risk))
  (Derive FlipChecked  (CheckAbstractionMap Flip)))

;; calm: the risk buckets are an exact quotient, and the optimum lives on them
!(dc.run MarketDesk CalmQuotient)
!(dc.run MarketDesk (CheckAbstractionMap CalmMirror))
!(dc.run MarketDesk (OptimizePolicy CalmDesk (Under Flat)))
!(dc.run MarketDesk (OptimizePolicy CalmDesk (Through Risk) (Under Flat)))
```

```
;; toxic flow: every value moves, no decision does
!(dc.run MarketDesk (InduceExact RiskToxic))
!(dc.run MarketDesk (CheckOptimal ToxicDesk (Policy calm-policy)))
!(dc.run MarketDesk (EvaluatePolicy ToxicDesk (Policy calm-policy) (Under Flat)))
;; one-sided flow: the compression breaks, and so does the policy
!(dc.run MarketDesk (InduceExact RiskBuyers))
!(dc.run MarketDesk (CheckUpdates RiskBuyers))
!(dc.run MarketDesk (MeasureBehavioral RiskBuyers (Discount (Rat 9 10))))
!(dc.run MarketDesk (CheckOptimal BuyersDesk (Policy calm-policy)))
!(dc.run MarketDesk (OptimizePolicy BuyersDesk (Under Flat)))
!(dc.run MarketDesk (OptimizePolicy BuyersDesk (Through RiskBuyers) (Under Flat)))
;; the broken symmetry survives as a transfer between the two one-sided markets
!(dc.run MarketDesk FlipChecked)
!(dc.run MarketDesk (PullbackPolicy Flip (Policy sellers-policy)))

;; ------------------------------------------------------------------------------ informed flow
(DC.Module InformedFlow
  (Profile Stochastic)
  (Use Decisions Laws Families NatArith)
  (Import MarketDesk (Only Inv Quote Market Bit Ticks step pnl ToxicDesk))
  (Carrier Flow    (Of Symbol) (Enum noise informed))
  (Carrier Book    (Of (Tuple Int Symbol)) (Product Inv Flow))
  (Carrier Reading (Of Symbol) (Enum says-noise says-informed))
  (Measure Half     (On Flow) (Masses (noise (Rat 1 2)) (informed (Rat 1 2))))
  (Measure FlatHalf (On Book) (Masses ((Tuple 0 noise) (Rat 1 2)) ((Tuple 0 informed) (Rat 1 2))))

  ;; each step is informed with probability 1/2, independently of everything else: an
  ;; informed fill is followed by a one-tick move against the maker; fills are as in calm
  (Map visible (From Book) (To Inv) (Body ($x) (GetTuple $x 0)))
  (Map flow-of (From Book) (To Flow) (Body ($x) (GetTuple $x 1)))
  (Map visible-move (From (Product Book Quote)) (To (Product Inv Quote))
    (Body ($xa) (Tuple (visible (GetTuple $xa 0)) (GetTuple $xa 1))))
  (Map split-step (From (Product Book Quote)) (To (Law Book))
    (Body ($xa) (Indep (step calm (visible-move $xa)) (LawOf Half))))
  (Map split-pnl (From (Product Book Quote)) (To (Law Rational))
    (Body ($xa) (If (Eq (flow-of (GetTuple $xa 0)) informed)
                    (pnl informed (visible-move $xa))
                    (pnl calm (visible-move $xa)))))
  (DecisionProcess Split (States Book) (Actions Quote) (Kernel split-step) (RewardLaw split-pnl)
    (Discount (Rat 9 10)) (Values Ticks))
  (Representation Coarse (Of Split) (Encode visible))

  ;; a toxicity signal that is wrong one time in ten and costs 1/20 tick per step
  (TransitionKernel FlowSignal (From Flow) (To Reading)
    (Row noise    (Mass says-noise (Rat 9 10)) (Mass says-informed (Rat 1 10)))
    (Row informed (Mass says-noise (Rat 1 10)) (Mass says-informed (Rat 9 10))))
  (ObservationMethod Signal (Channel FlowSignal) (Cost (Rat 1 20))))

!(dc.run InformedFlow (CheckEqual (AverageModel Split (Through Coarse) (Under Half)) ToxicDesk
    ↪ (Compare Means)))
!(dc.run InformedFlow (CheckEqual (AverageModel Split (Through Coarse) (Under Half)) ToxicDesk))
!(dc.run InformedFlow (InduceExact Coarse))
!(dc.run InformedFlow (OptimizePolicy Split (Through Coarse) (Under FlatHalf)))
!(dc.run InformedFlow (OptimizePolicy Split (Under FlatHalf)))
!(dc.run InformedFlow (EvaluatePolicy Split (With Signal) (Through Coarse) (Under FlatHalf)))

;; ------------------------------------------------------------------------------ did the news leak?
(DC.Module NewsLeak
  (Profile Stochastic)
  (Use Linear)                                          ; the Like field: PROPOSED with Example 2
  (Carrier Tape   (Of Symbol)
    (Enum leak embargo knows-up knows-down pending moved-up moved-down picked-up picked-down))
  (Carrier Report (Of Symbol) (Enum quiet up down hit-up hit-down))
  (Map report (From Tape) (To Report)
    (Cases (leak quiet) (embargo quiet) (knows-up quiet) (knows-down quiet) (pending quiet)
           (moved-up up) (moved-down down) (picked-up hit-up) (picked-down hit-down)))
  ;; leak: an informed trader already knows the direction; embargo: nobody does yet.
  ;; The informed trader picks off a tight quote just before the move and leaves a wide one.
  (TransitionKernel QuoteTight (From Tape) (To Tape)
    (Row leak (Mass knows-up (Rat 1 2)) (Mass knows-down (Rat 1 2)))
    (Row embargo (Mass pending 1))
    (Row knows-up (Mass picked-up 1))
    (Row knows-down (Mass picked-down 1))
```

```
    (Row pending (Mass moved-up (Rat 1 2)) (Mass moved-down (Rat 1 2)))
    (Row moved-up (Mass moved-up 1)) (Row moved-down (Mass moved-down 1))
    (Row picked-up (Mass picked-up 1)) (Row picked-down (Mass picked-down 1)))
  (TransitionKernel QuoteWide (From Tape) (To Tape)
    (Row leak (Mass knows-up (Rat 1 2)) (Mass knows-down (Rat 1 2)))
    (Row embargo (Mass pending 1))
    (Row knows-up (Mass moved-up 1))
    (Row knows-down (Mass moved-down 1))
    (Row pending (Mass moved-up (Rat 1 2)) (Mass moved-down (Rat 1 2)))
    (Row moved-up (Mass moved-up 1)) (Row moved-down (Mass moved-down 1))
    (Row picked-up (Mass picked-up 1)) (Row picked-down (Mass picked-down 1)))
  (Machine Desk     (States Tape) (Observe report report) (Action tight QuoteTight) (Action wide
    ↪ QuoteWide))
  (Machine WideDesk (States Tape) (Observe report report) (Action wide QuoteWide))
  (Carrier Seen (Of Symbol) (Reach WideDesk (From leak embargo)))
  (Machine WideSeen (States Seen) (Like WideDesk))        ; the desk that only ever quotes wide
  (Map merge-leak (From Tape) (To Tape)
    (Cases (leak leak) (embargo leak) (knows-up knows-up) (knows-down knows-down) (pending pending)
           (moved-up moved-up) (moved-down moved-down) (picked-up picked-up) (picked-down
    ↪ picked-down)))
  (Map merge-leak-seen (From Seen) (To Seen)
    (Cases (leak leak) (embargo leak) (knows-up knows-up) (knows-down knows-down) (pending pending)
           (moved-up moved-up) (moved-down moved-down)))
  (Representation MergeLeak     (Of Desk)     (Encode merge-leak))
  (Representation MergeLeakWide (Of WideSeen) (Encode merge-leak-seen))
  (Derive WideLinear (MinimalLinear WideSeen))
  (Derive DeskLinear (MinimalLinear Desk))
  (Derive WideTraceClasses (TracePartition WideSeen)))

!(dc.run NewsLeak (CheckTrace MergeLeakWide (Policies ObservationBased)))
!(dc.run NewsLeak (CheckBehavioral MergeLeakWide))
!(dc.run NewsLeak (MeasureBehavioral MergeLeakWide (Discount (Rat 9 10))))
!(dc.run NewsLeak WideTraceClasses)
!(dc.run NewsLeak WideLinear)
!(dc.run NewsLeak (CheckTrace MergeLeak))
!(dc.run NewsLeak (CheckBehavioral MergeLeak))
!(dc.run NewsLeak DeskLinear)

;; ------------------------------------------------------------------------ monitors
(DC.Module ShiftMonitors
  (Profile Stochastic)
  (Use Decisions Laws Families Monitors)                ; Monitors: PROPOSED
  (Import MarketDesk (Only Inv Quote Market One theta fills))
  (Carrier Markout (Of Symbol) (Enum adverse favorable))
  (Carrier Posted  (Of (Tuple Int Symbol)) (Product Inv Quote))     ; the quote standing at a step
  ;; after a fill the mid moves one tick against the maker with probability (1 + theta)/2
  (Map markout (Parameters ($m Market)) (From One) (To (Law Markout))
    (Body ($u) (Image $b (If (Eq $b 1) adverse favorable) (Coin (Div (Add 1 (theta $m $u)) 2)))))
  (Monitor Markouts       (Context One) (Null (markout calm)) (Alternative (markout toxic))
    (Columns (observation (each "markouts"))))
  (Monitor MarkoutsBuyers (Context One) (Null (markout calm)) (Alternative (markout buyers))
    (Columns (observation (each "markouts"))))
  (Monitor Fills      (Context Posted) (Null (fills calm)) (Alternative (fills buyers))
    (Columns (context "q" "quote") (observation "ask" "bid")))
  (Monitor FillsToxic (Context Posted) (Null (fills calm)) (Alternative (fills toxic))
    (Columns (context "q" "quote") (observation "ask" "bid")))
  (Stream ToxicTape  (Source "streams/toxic_seed1.json")  (Digest
    ↪ "sha256:ddbb604841be3c26ae4b5af8ab260f0a7aa995bd2a0963c994b58fee71257d00")
    (Provenance "simulated: informed flow on half the steps, calm policy, 4000 steps, seed 1"))
  (Stream BuyersTape (Source "streams/buyers_seed1.json") (Digest
    ↪ "sha256:7f3ca6594cc85c5cfc3833861eb12a27c7136ab01fc4ef3e3151797632254cc0")
    (Provenance "simulated: one-sided buying, calm policy, 4000 steps, seed 1"))
  (Stream CalmTape   (Source "streams/calm_seed1.json")   (Digest
    ↪ "sha256:075bbc19f30524d642a07b57de44daf1618b043cc273e0f49357ee6b1e1af3ab")
    (Provenance "simulated: calm market, calm policy, 4000 steps, seed 1")))

!(dc.run ShiftMonitors (CheckMonitor Markouts))
!(dc.run ShiftMonitors (CheckMonitor Fills))
!(dc.run ShiftMonitors (CheckMonitor FillsToxic))
!(dc.run ShiftMonitors (CheckMonitor MarkoutsBuyers))
!(dc.run ShiftMonitors (Detect Markouts Fills (On ToxicTape)  (Level (Rat 1 100))))
!(dc.run ShiftMonitors (Detect Markouts Fills (On BuyersTape) (Level (Rat 1 100))))
!(dc.run ShiftMonitors (Detect Markouts Fills (On CalmTape)   (Level (Rat 1 100))))
```

B Registry patches

Each example's patch is a JSON file whose entries are applied in memory by `ex_check.py`. The tables below are generated from the four patch files and list every entry: libraries and their primitives, primitives added to existing libraries, declarations, fields added to existing declarations, requests (new or extended), request arguments, result types and result fields, and budget units. A request listed in a patch either is new or replaces the registered entry of the same name with an extended one.

B.1 Example 1

Entry	Name	Description
library	`NatArith`	primitives Le, Lt, Min, Max, Quot, Rem. Order and integer division on Nat and Int; Quot and Rem need a positive divisor and return a tagged failure value otherwise; If evaluates only the branch it selects
library	`Parallel`	Parallel composition of machines with synchronized and interleaved actions, and product representations; claims about products are answered by the product rule from component certificates
primitives	`BitVectors`	adds BitOr, BitAnd, BitXor, PopCount
primitives	`Core`	adds IntervalProduct, Tied, RoundedGrid, Meet
tolerance body	`Meet`	registered as a tolerance body
declaration	`EncodingFamily`	library Core; fields Of, Generator. A finite declared family of representations of one machine; frontier optimality may be Checked relative to it
declaration	`ParallelMachine`	library Parallel; fields Component, Copies, Synchronize; counts as a Machine. States are tuples of component states; observers are label.i for each copy
declaration	`ProductRepresentation`	library Parallel; fields Of, Component, Copies; counts as a Representation. The componentwise encoding of a parallel machine
argument	`Refine Until`	type stable-or-horizon, default Stable. Stable, or (Steps k) for the staged partition P_k, which need not be update-compatible
argument	`WeakestWithin Objective`	type enum:Weakness|Codes, default Weakness
argument	`WeakestWithin Family`	type ref:EncodingFamily. Search only the declared family; Optimality is then relative to it
argument	`WeakestWithin Kind`	type enum:Behavioral|Trace|Now|Updates|Radius, default Behavioral. Radius: the largest behavioral distance from a state to the representative of its class; the representatives are part of the answer
result fields	`Frontier`	Codes: Nat, LowerBound: Certificate?, Representatives: Map?

Table 21: Registry patch of Example 1.

B.2 Example 2

Entry	Name	Description
library	`Families`	primitives Switch, Exists, ForAll, For; constants Any; binders Exists, ForAll. Indexed families elaborated before checking: a Map with (Parameters ($p Carrier) ...) is a family of unary maps, applied as (f p1 ... x); (For $v Carrier) on an Observe or Action entry declares one entry per element, labelled by the instantiated label term, in the order of the For clauses and of each carrier's enumeration; (Switch $p (element expr)+) selects on a parameter and must be total and exclusive over its carrier; (Exists $v C e) and (ForAll $v C e) range over an enumerated carrier; Any in a Successor entry stands for every action label
primitives	`BitVectors`	adds Gather
field	`Map Parameters`	type expr. Elaboration-time parameters, each ($p Carrier) over an enumerated carrier
field	`Machine ActionsOf`	type ref:Machine. The action entries of another machine on a carrier of the same element type
field	`Machine Like`	type ref:Machine. The observe and action entries of another machine; States may differ (a scope)
field	`BlockPartition Invariant`	type ref:Map. A candidate inductive invariant for a reachability scope: checked to contain the initial states and to be closed under every action, then used to discharge the universal obligations
budget unit	`Unrollings`	Bounded search depth for reachable counterexamples when an inductive argument fails; a work limit, never a semantic horizon
note	`Gather`	(Gather x i1 ... ik) is the k-bit word whose bit j is bit i_j of x

Table 22: Registry patch of Example 2.

B.3 Example 3

Entry	Name	Description
field	`TransferHive GaugeFrom`	type expr. (GaugeFrom member symmetry+): declare the gauge at one member; the checker transports it along the declared edges by conjugation with the method pushforwards, reports the transported gauges, and checks that the transport around each cycle returns the declared gauge
request	`TransportSkill`	library Transfers; result TransportedSkill; statuses Checked. The transported skill y -> iota(skill(phi y)) earns, at every target state, the labelled response the source skill earns at the pulled-back state; the result also lists where it differs from a declared comparison skill
request	`FindTransfers`	library Transfers; result TransferCandidates; statuses Checked; arguments Candidates, Within, Metric. The listed candidates are exactly those of the declared family whose defect is at most Within; a learner that keeps one of several reports the others, since the choice among them is a gauge choice

Entry	Name	Description
result type	`TransportedSkill`	Skill: Map, Changed: Nat, Certificate: Certificate. The transported skill table, the number of states where it differs from the skill it was transported from when source and target coincide, and the response-equality certificate
result type	`TransferCandidates`	Exact: TransferList, Count: Nat, Certificate: Certificate. The qualifying candidates in declaration order, with one defect certificate per candidate

Table 23: Registry patch of Example 3.

B.4 Example 4

Entry	Name	Description
library	`Laws`	primitives Indep, Image, Expect, LawOf; binders Image. Finite rational laws as values: (Indep L1 ... Lk) is the product law, whose values are k-tuples; (Image $v expr L) is the law of expr when $v is drawn from L, masses of equal values added; (Expect L) is the mean of a rational-valued law; (LawOf m) is the law of a declared Measure. Coin and Point remain the elementary laws. Laws are evaluated on their support, so a value of mass zero is never produced and carrier closure is checked on supports.
library	`Monitors`	primitives Columns, context, observation, each; constants Union, Mixture. Likelihood-ratio monitors for streams: a Monitor multiplies Alternative(c)(o) / Null(c)(o) over the observations o of a Stream, each read in the context c posted before it. Exact claims about a monitor are Checked; detections on data are Estimated.
declaration	`DecisionProcess`	library Decisions; fields States, Actions, Kernel, RewardLaw, Discount, Values; counts as a Machine. A finite discounted decision process. It is also a stochastic Machine whose observations are the expected rewards of its actions, one query (reward a) per action in declaration order, so InduceExact, CheckUpdates, MeasureBehavioral and CheckAbstractionMap apply to it unchanged; a quotient in that sense is an MDP homomorphism.
declaration	`Monitor`	library Monitors; fields Context, Null, Alternative, Columns
declaration	`Stream`	library Monitors; fields Source, Digest, Provenance. Frozen observation data with provenance; a different digest is a different stream
request	`AverageModel`	library Decisions; result AveragedModel; statuses Checked; arguments Under, Through. Prior-averaged reward law per action. For a DecisionProcess with Through: the process on the codes whose rewards and pushed rows are averaged over each fiber under Under; admitted only when the hidden part is fresh (redrawn from Under at every step, independently of the rest), and then exact for every policy, history-dependent or not, that sees only the codes; InvalidInput otherwise

Entry	Name	Description
request	`CheckEqual`	library Decisions; result Verdict; statuses Checked, Refuted; arguments Compare. The two averaged reward laws are equal; for decision processes on the same states and actions (declared or averaged), equal kernels and equal reward laws (Compare Laws) or equal expected rewards (Compare Means), with witness the least state, the first action and the first successor or reward value, in increasing order, whose masses differ
request	`OptimizePolicy`	library Decisions; result PolicyValue; statuses Checked; arguments Through, Under. Optimal expected reward over policies that see only the representation. For a DecisionProcess: the optimal discounted value from the start law Under, over stationary deterministic policies on states (no Through) or on codes (Through), evaluated in the concrete process, with the policy table and a Bellman certificate; on codes the class is memoryless code policies, which is every coarse policy when the hidden part is fresh
request	`EvaluatePolicy`	library Decisions; result PolicyValue; statuses Checked; arguments With, Policy, Through, Under. Exactly one of With and Policy. With: expected reward of the best policy using the observation method, net of its cost; for a DecisionProcess the method reads the state component whose carrier is its channel's input once per step before the action, the policy also sees the code of Through, and values are computed on the hidden states, never by substituting the reading for the state. Policy: the exact discounted value of the declared stationary policy from Under
request	`CheckOptimal`	library Decisions; result OptimalityVerdict; statuses Checked, Refuted; arguments Policy. The stationary policy is optimal: its exact values V satisfy Q(x, pi x) >= Q(x, a) for every state and action, Q(x, a) = r(x, a) + gamma E V; Refuted with the least state, the policy's action, the first action in declaration order with a larger Q-value and the two exact Q-values
request	`PullbackPolicy`	library Decisions; result PulledBackPolicy; statuses Checked, Refuted; arguments Policy. The arrow h is a homomorphism of decision processes (expected rewards and pushed rows preserved) and the policy x -> pi(h x) has at every source state the value pi has at h x; when h is surjective and pi is optimal for the target, the pulled-back policy is optimal for the source; Refuted with the homomorphism witness otherwise
request	`CheckMonitor`	library Monitors; result MonitorCheck; statuses Checked. For every context the null expectation of the likelihood-ratio factor is at most one (exactly one when the alternative is absolutely continuous with respect to the null), so the product is a nonnegative supermartingale under the null for every policy that fixes the context before the observation; Blind when the two laws agree in every context; exact chi-square per context

Entry	Name	Description
request	`Detect`	library Monitors; result Detection; statuses Estimated; arguments On, Level, Combine. Estimated: if the stream follows the null laws, the probability that a listed monitor ever reaches its threshold is at most Level (Ville's inequality; Union gives each of k monitors the threshold k/Level, Mixture thresholds the average of the e-values at 1/Level); the result names the first monitor and observation to reach its threshold, or reports that none did, with the stream digest, the protocol and the dependence assumptions
result type	`OptimalityVerdict`	Values: Table, QValues: Table, Witness: Witness?, Certificate: Certificate. The exact values of the policy and the Q-table that certifies or refutes optimality
result type	`PulledBackPolicy`	Policy: Map, Values: Table, Certificate: Certificate
result type	`MonitorCheck`	Valid: Bool, Blind: Bool, ChiSquare: Table, Certificate: Certificate
result type	`Detection`	Fired: Monitor?, Step: Nat?, Observations: Nat, EValue: Rational, Threshold: Rational, Estimand: Expr, Procedure: Expr, Assumptions: Expr. An Estimated result: the claim is about the data source, never about the model
result fields	`AveragedModel`	Process: DecisionProcess?
result fields	`PolicyValue`	Values: Table?, Certificate: Certificate

Table 24: Registry patch of Example 4.

C Reproducing the results

The four bundles are collected in `mettadinc_worked_examples_package.zip`, one directory per example (`ex1_belief_memory`, `ex2_access_policy`, `ex3_skill_share` and `ex4_market_maker`) under a top-level README. Each bundle contains its module file, its registry patch (and the patches of earlier examples that it reuses), the static checker `ex_check.py`, the Python model, the oracle and the oracle's results. The bundles of Examples 1, 3 and 4 add the reference evaluator `metta_eval.py`, and those of Examples 1 and 2 their proof obligations and a replay script. Python 3.11 or later is assumed.

Running an oracle. In a bundle directory, `python3 exN_oracle.py` recomputes every number of the example from the module file, prints one line per check, writes `exN_results.json`, and exits with a nonzero status if any check fails. Table 2 gives the running times and dependencies. The oracles of Examples 1 and 2 write every unsatisfiable obligation, with its CNF and its DRAT proof, to `obligations/`, and check the proofs when the environment variable `MDINC_TOOLS` names a directory holding `drat-trim/drat-trim` and `cake_lpr/cake_lpr`. Without the tools they skip the checks that read the proof checker's verdicts and report the results that rest on the proofs as Conditional, with a trusted-backend assumption. The pinned versions are drat-trim at commit `2e3b2dc` and `cake_lpr` at commit `033aad5`, run as

```
drat-trim obligation.cnf obligation.drat -L obligation.lrat
cake_lpr --CML_HEAP_SIZE=1024 --CML_STACK_SIZE=512 obligation.cnf obligation.lrat
```

Replaying the shipped proofs. `ex1_obligations.tar.xz` and `ex2_obligations.tar.xz` hold the obligations of the runs that wrote the shipped results: the CNF and DRAT proof of every unsatisfiable obligation, the colorings, cliques and center sets of Example 1, and an index

of names, claims, CNF digests and the digest of the module file. `replay_proofs.py` needs neither python-sat nor the oracle. It extracts an archive, compares the module digest with the module file it is given and every CNF with its digest, and runs the two commands above on every proof:

```
MDINC_TOOLS=/path/to/tools python3 replay_proofs.py ex2_obligations.tar.xz
    access_policy.metta
```

Checking the modules. The checker needs the specification's validation package, which contains the registry and the base checker:

```
MDINC_PACKAGE=/path/to/mettadinc_validation_package python3 ex_check.py \
    ex1_registry_patch.json ex2_registry_patch.json ex4_registry_patch.json \
    -- market_maker.metta
```

The patches before `--` are applied in order. The first pass must report that all forms conform to the registry with the proposed patches; the second pass lists the diagnostics against the unpatched registry, which are exactly the forms that depend on the patches. The commands for the other examples take the patch lists of Table 25.

Example	Module file	Patches, in order
1	`belief_memory.metta`	`ex1_registry_patch.json`
2	`access_policy.metta`	`ex2_registry_patch.json`
3	`skill_share.metta`	`ex1_registry_patch.json`
		`ex3_registry_patch.json`
4	`market_maker.metta`	`ex1_registry_patch.json`
		`ex2_registry_patch.json`
		`ex4_registry_patch.json`

Table 25: Static checks.

The reference evaluator. `metta_eval.py`, the same file in the bundles of Examples 1, 3 and 4, reads a module file with its own small reader, resolves imports, and evaluates the maps of the module: `Cases`, `Body` and `Table` bodies, parameterized maps, the arithmetic and bit-vector primitives used in the files, and the proposed law primitives. The oracles use it to compare each module with its Python model cell by cell. It is a test aid for these examples and not an implementation of MeTTaDinC.

The source route of Example 2. `policy_ir.py` reads `access_policy.metta` into the typed intermediate form of Section 4.8, and `obligations.py`, `bdd_reach.py` and the oracle build every obligation, reachable set and witness from it; `validate_small_lib.py` holds the point-by-point validation on the two-user instance. `rbac.py`, `circuits.py` and `sat_obligations.py` are the independent model, kept as a differential check. The oracle also applies the two source mutations of Table 10 and requires the verdicts they affect to change.

The streams of Example 4. The three streams in `streams/` were generated once by `monitors.simulate` with seed 1 and are stored as data; the module pins each by its SHA-256 digest. Reproducing a detection means replaying the stored stream, not rerunning the generator, whose output depends on the random number generator of the language it runs in.